

# 鸿蒙小时达项目

- 小时达司机端设计图: <https://codesign.qq.com/s/P4VIZMyVoRZq6wL/ALwE9V4JwNRZX1D/inspect>
- 小时达接口地址: <https://apifox.com/apidoc/shared-4b036830-59b1-4526-b00a-61df2b3d4ae1>
- 小时达司机端账号注册地址: <https://fe-slwl-manager.itheima.net/#/driver-register>
- git仓库地址- <https://gitee.com/shuiruohanyu/quick-delivery>

## 分层架构逻辑模型

- HarmonyOS应用的分层架构主要包括三个层次：产品定制层products、基础特性层features和公共能力层commons，为开发者构建了一个清晰、高效、可扩展的设计架构。
- 不同设备意味着不同的入口。products是个入口 可能 包含手机 + 平板 + 2in1 + 手表 + Car

Hap-entry  
Hsp-共享包  
Har包-静态共享包

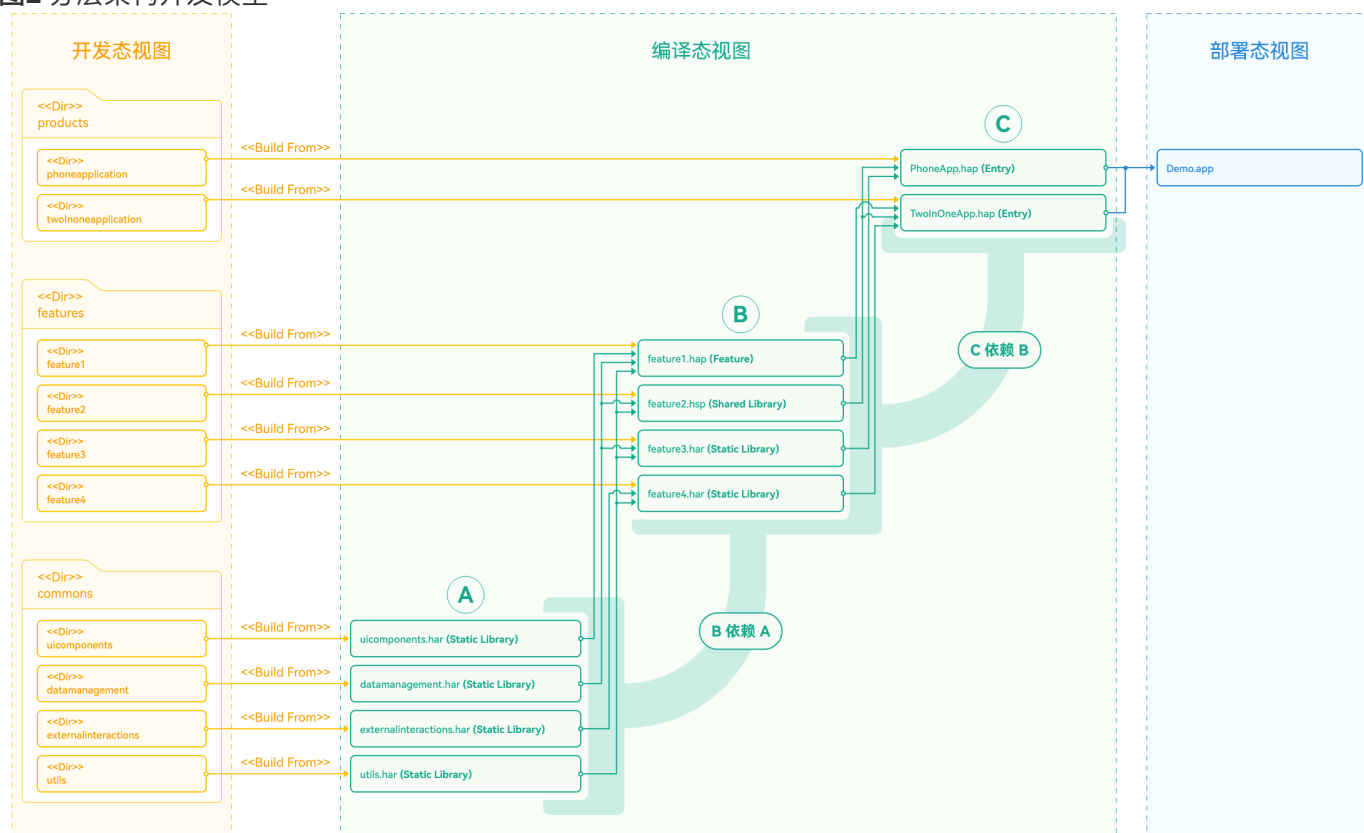
products- hap包  
features- hap-hsp包  
common- hsp

- **产品定制层**产品定制层专注于满足不同设备或使用场景（如应用）的个性化需求，包括UI设计、资源和配置，以及针对特定场景的交互逻辑和功能特性。产品定制层的功能模块独立运作，同时依赖基础特性层和公共能力层来实现具体功能。作为应用的入口，产品定制层是用户直接互动的界面。为满足特定产品需求，产品定制层可灵活地调整和扩展，从而满足各种使用场景。
- **基础特性层**基础特性层位于公共能力层之上，用于存放基础特性集合，例如相对独立的功能UI和业务逻辑实现。该层的每个功能模块都具有高内聚、低耦合、可定制的特点，以支持产品的灵活部署。基础特性层为上层的产品定制层提供稳健且丰富的基础功能支持，包括UI组件、基础服务等。同时依赖于下层的公共能力层为其提供通用功能和服务。为了增强系统的可扩展性和维护性，基础特性层将功能进行模块化处理。例如，一个应用的底部导航栏中的每个选项都可能是一个独立的业务模块。
- **公共能力层**公共功能层用于存放公共基础能力，集中了例如公共UI组件、数据管理、外部交互以及工具库等的共享功能。应用可以共享和调用这些公共能力。公共能力层为上层的基础特性层和产品定制层提供稳定可靠的功能支持，确保整个应用的稳定性和可维护性。公共能力层包括但不限于以下组成：
  - **公共UI组件**：这些组件被设计成通用且高度可复用的，确保在不同的应用程序模块间保持一致的用户体验。公共UI组件提供了标准化且友好的界面，帮助开发者快速实现常见的用户交互需求，例如提示、警告、加载状态显示等，从而提高开发效率和用户满意度。
  - **数据管理**：负责应用程序中数据的存储和访问，包括应用数据、系统数据等，提供了统一的数据管理接口，简化数据的读写操作。通过集中式的数据管理方式不仅使得数据的维护更为简单，而且能够保证数据的一致性和安全性。
  - **外部交互**：负责应用程序与外部系统的交互，包括网络请求、文件I/O、设备I/O等，提供统一的外部交互接口，简化应用程序与外部系统的交互。开发者可以更为方便地实现应用程序的网络通信、数据存储和硬件接入等功能，从而加速开发流程并保证程序的稳定性和性能。

- 工具库：提供一系列常用工具函数和类，例如字符串处理、日期时间处理、加密解密、数据压缩解压等，帮助开发者提高效率和代码质量。

## 开发模型

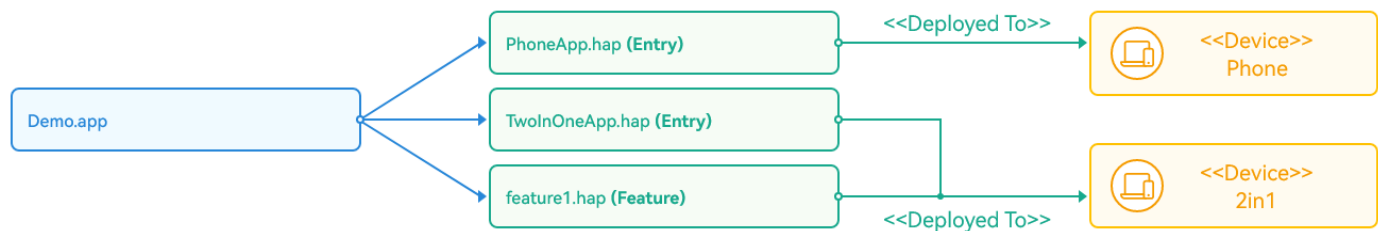
图2 分层架构开发模型



- **产品定制层**产品定制层的各个子目录会被编译成一个Entry类型的HAP，作为应用的主入口。该层主要针对跨多种设备，为各种设备形态集成相应的功能和特性。产品定制层被划分为多个功能模块，每个功能模块都针对特定的设备或使用场景设计，并根据具体的产品需求进行功能及交互的定制开发。**说明**
  - 在产品定制层，开发者可以从不同设备对应应用的UX设计和功能两个维度，结合具体的业务场景，选择一次编译生成相同或者不同的HAP（或其组合）。
  - 通过使用定制多目标构建产物的定制功能，可以将应用所对应的HAP编译成各自的.app文件，用于上架到应用市场。
- **基础特性层**在基础特性层中，功能模块根据部署需求被分为两类。对于需要通过Ability承载的功能，可以设计为Feature类型的HAP，而对于不需要通过Ability承载的功能，根据是否需要实现按需加载，可以选择设计为HAR模块或者HSP模块，编译后对应HAR包或者HSP包。
- **公共能力层**公共能力层的各子目录将被编译成HAR包，而他们只能被产品定制层和基础特性层所依赖，不允许存在反向依赖。该层旨在提取模块化公共基础能力，为上层提供标准接口和协议，从而提高整体的复用率和开发效率。

## 部署模型

图3 分层架构部署模型（不同设备的定制）



应用程序（.app文件）在流水线或应用市场上被解包为n \* Entry类型的HAP + n \* Feature类型的HAP，根据设备类型和使用场景将应用部署到不同类型的设备上，实现多端的统一用户体验。

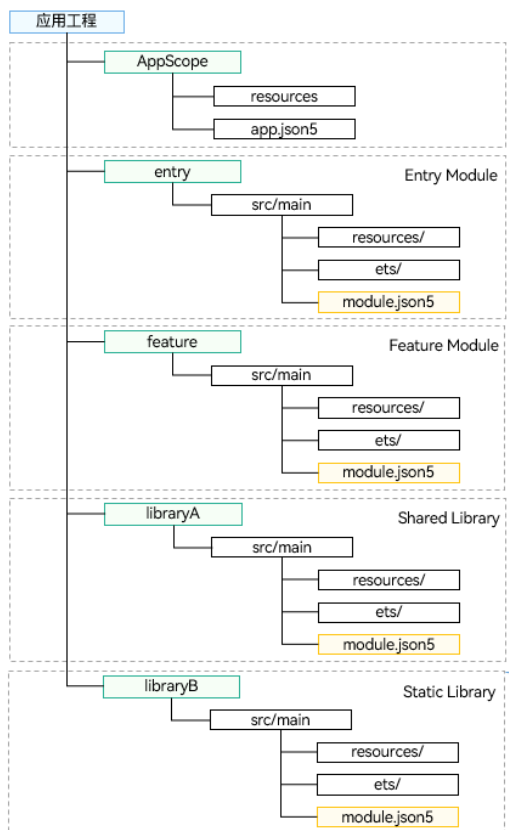
说明

当Entry类型的HAP和Feature类型的HAP被分发并部署到相应的设备时，他们所依赖的HSP也会一同被分发并部署到相应的设备上。

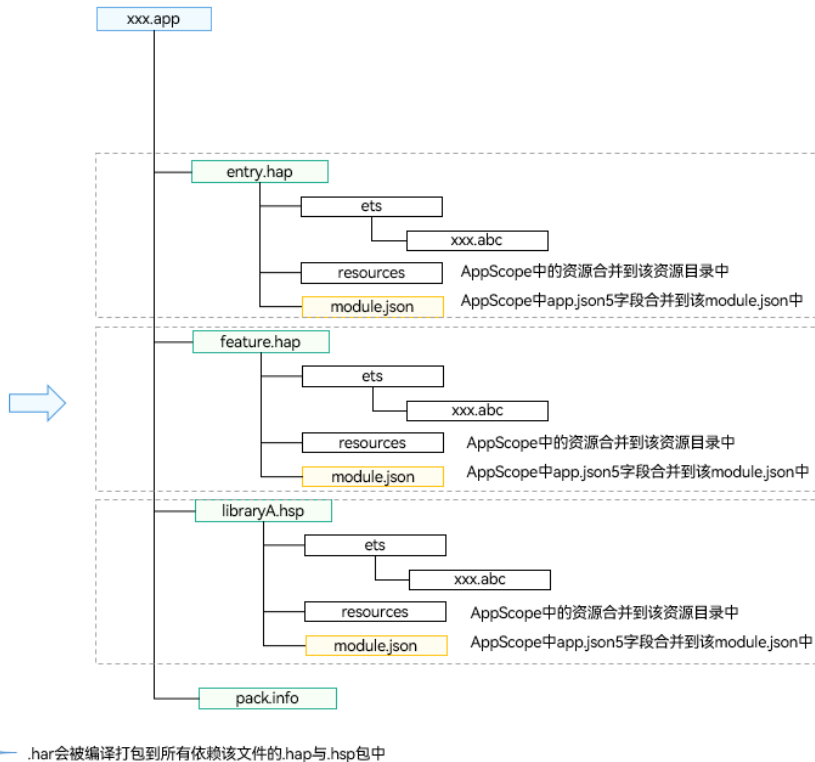
在部署模型中，每个Entry类型的HAP代表了应用的入口点，而Feature类型的HAP则包含了应用的特定功能模块。允许应用能够以模块化的方式适配和部署，从而满足不同设备和场景的需求。

该部署模型不仅优化了应用的组织结构，也为保持应用在各种设备和场景中的一致性提供了支持。通过按照设备类型和使用场景来区分和部署不同的HAP，能确保无论在何种设备或场景中，用户都能获得统一且高质量

开发态的视图



编译打包后的视图



.har会被编译打包到所有依赖该文件的.hap与.hsp包中

## 选择合适的包类型

HAP、HAR、HSP三者的功能和使用场景总结对比如下：

| Module类型       | 包类型 | 说明                                                                                                                                                              |
|----------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ability        | HAP | 应用的功能模块，可以独立安装和运行，必须包含一个entry类型的HAP，可选包含一个或多个feature类型的HAP。                                                                                                     |
| Static Library | HAR | 静态共享包，编译态复用。<br>- 支持应用内共享，也可以发布后供其他应用使用。<br>- 作为三方库，发布到OHPM私仓，供公司内部其他应用使用。<br>- 作为三方库，发布到OHPM中心仓，供其他应用使用。<br>- 多包（HAP/HSP）引用相同的HAR时，会造成多包间代码和资源的重复拷贝，从而导致应用包膨大。 |
| Shared Library | HSP | 动态共享包，运行时复用。<br>- 当前仅支持应用内共享。<br>- 当多包（HAP/HSP）同时引用同一个共享包时，采用HSP替代HAR，可以避免HAR造成的多包间代码和资源的重复拷贝，从而减小应用包大小。                                                        |

腾讯的面试官问了这么个问题：har会多份拷贝，hsp不会，但是hsp不会进行资源共享，有什么方案吗？  
腾讯用hsp包了一个har，向外暴露



开发者可以根据实际场景所需的能力，选择相应类型的包进行开发。在后续的章节中还会针对如何使用HAP、HAR、HSP分别展开详细介绍。

| 规格                                       | HAP | HAR | HSP |
|------------------------------------------|-----|-----|-----|
| 支持在配置文件中声明UIAbility组件与ExtensionAbility组件 | √   | ×   | ×   |
| 支持在配置文件中声明pages页面                        | √   | ×   | √   |
| 支持包含资源文件与.so文件                           | √   | √   | √   |
| 支持依赖其他HAR文件                              | √   | √   | √   |
| 支持依赖其他HSP文件                              | √   | √   | √   |
| 支持在设备上独立安装运行                             | √   | ×   | ×   |

**i** 说明

- HAR虽然不支持在配置文件中声明pages页面，但是可以包含pages页面，并通过命名路由的方式进行跳转。
- 由于HSP仅支持应用内共享，如果HAR依赖了HSP，则该HAR文件仅支持应用内共享，不支持发布到二方仓或三方仓供其他应用使用，否则会导致编译失败。
- HAR和HSP均不支持循环依赖，也不支持依赖传递。

## 单层项目架构

```
ets
├── common                               // 通用模块
│   ├── builders                       // - 通用组件 @Builder
│   ├── components                    // - 通用组件 @Component
│   ├── constants                     // - 常量数据
│   ├── images                        // - 图片资源
│   └── utils                          // - 工具类
├── entryability                       // 入口UIAbility
│   └── EntryAbility.ts
├── models                             // - 数据模型
├── pages                              // - 页面组件
│   ├── HomePage.ets
│   └── Index.ets
└── views                             // - 页面对应自定义组件
    ├── HomePage
    └── Index
```

三层项目架构代码实例  
[优秀实践-HMOS世界 \(ArkTS\) .zip](#)

## 三层架构

```
project-root
├── commons                // 基础层（通用组件、工具类、资源）
│   └── basic              // - 通用模块
├── features               // 需求层（业务组件）
│   ├── home              // - 需求模块
│   └── mine               // - 需求模块
└── products              // 入口层（页面）
    └── phone              // - 设备模块
```

## 三层架构模式

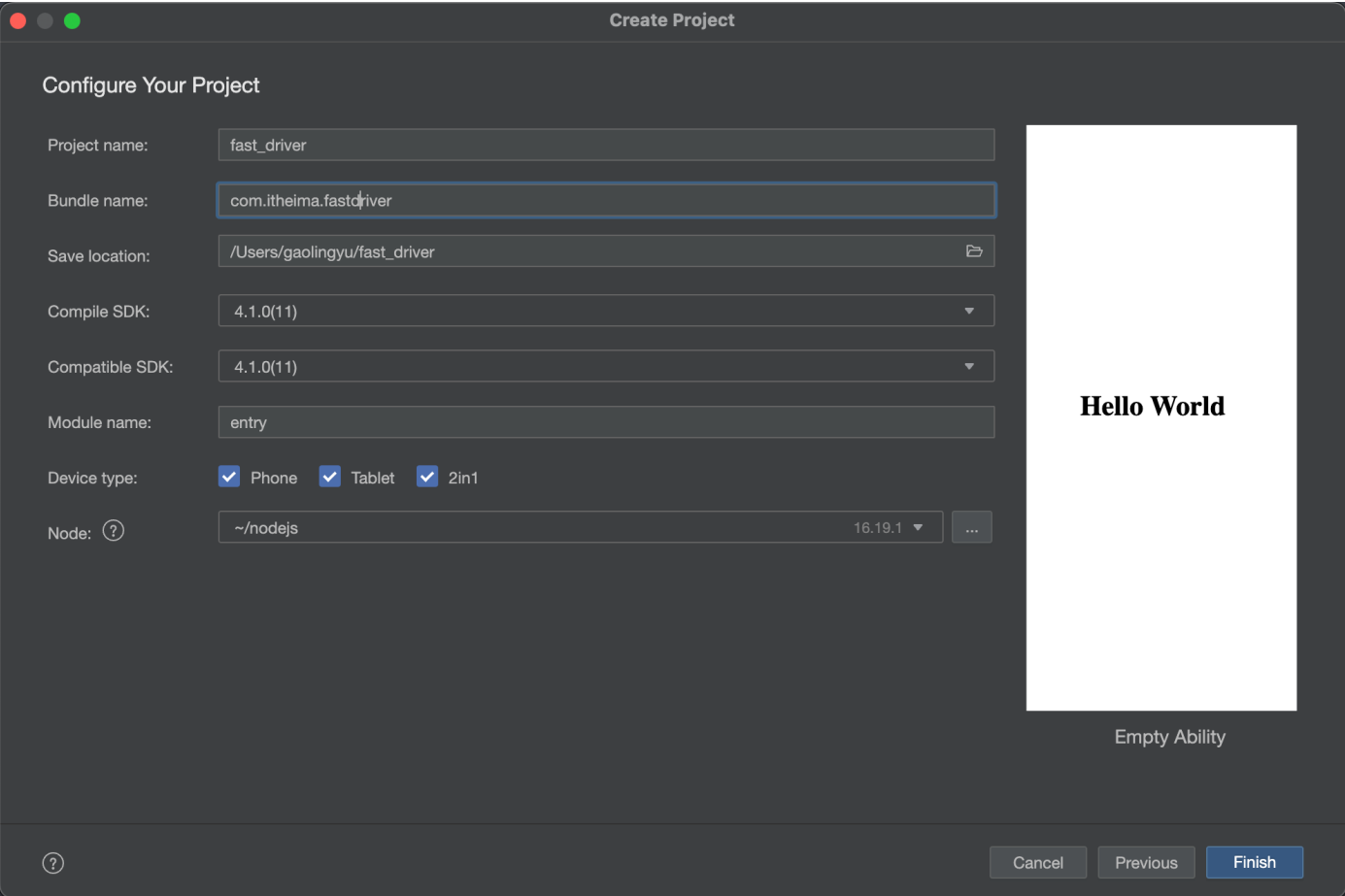
HMOS世界应用采用三层架构模式，整体架构如图所示：

- products层（产品定制层）：用于针对不同设备形态进行功能和特性集成。products层各个子目录分别编译为一个Entry类型的HAP包，作为应用的主入口，products层不可以横向调用。目前仅支持phone设备，后期会支持更多设备类型。
- features层（基础特性层）：用于放置不同的业务模块，例如探索、学习、活动、我的、登录等，供产品灵活部署。分别编译成HSP包，features层可以横向调用及依赖common层，同时可以被products层不同设备形态的HAP所依赖，但是不能反向依赖products层。
- common层（公共能力层）：用于存放项目的工具类、公共常量等，例如公共UI组件、日志管理、通用工具方法等。编译成HAR包，只可以被products和features依赖，不可以反向依赖。

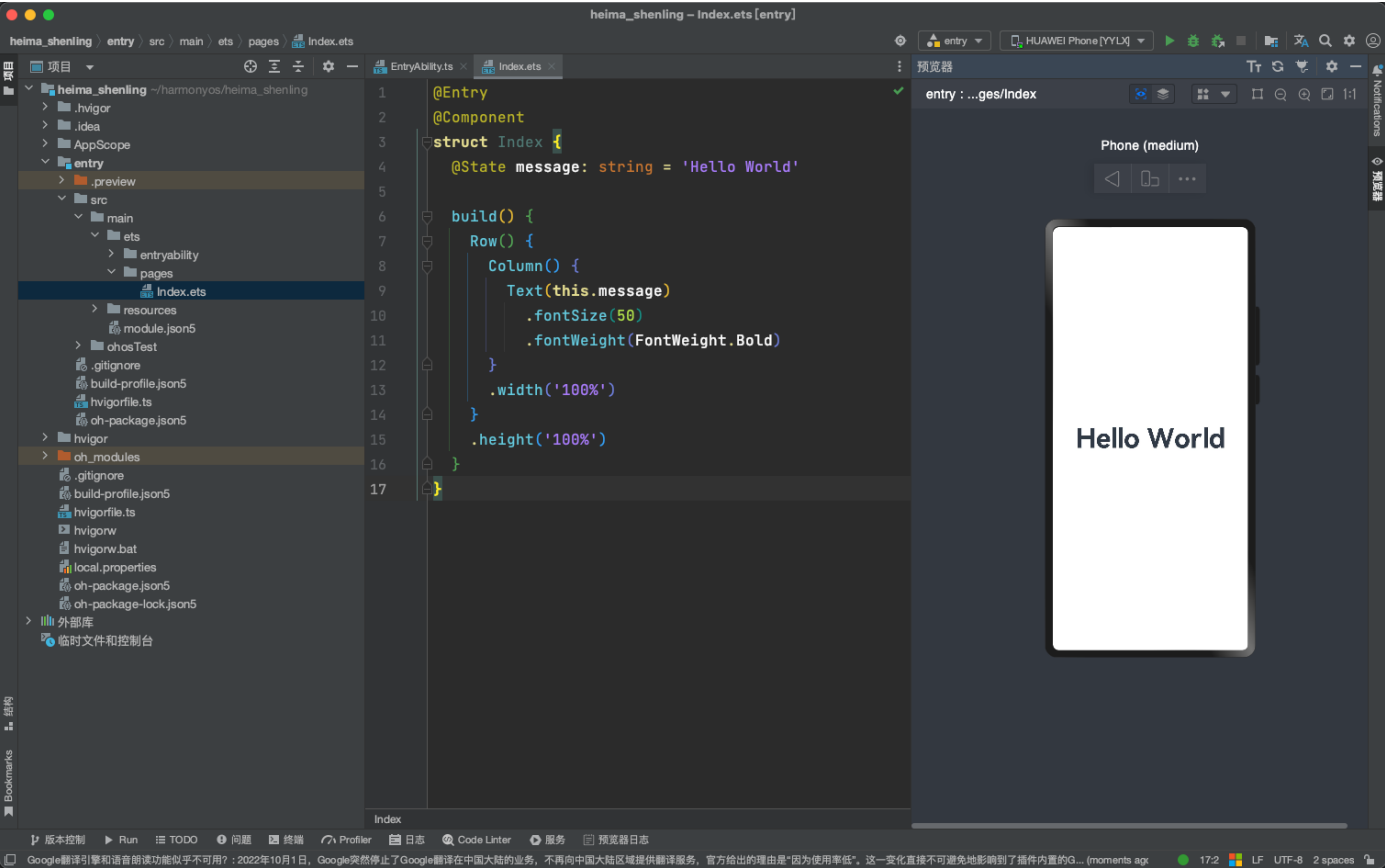
通过上述分层的架构设计，将不同业务拆分为HSP/HAR共享包进行模块化开发，可以更高效地管理工程、多人协作和特性集成。同时架构也具备更好的扩展性，当需要新增业务模块或设备类型时，新增features和products模块即可，并充分复用代码。

## 创建项目-使用git管理

使用DevEcoStudio创建一个空项目



直接finish



- 建立码云远程仓库

仓库名称 \* ✓

heima-shening

归属 路径 \* ✓

shuiruohanyu / heima-shening

仓库地址: <https://gitee.com/shuiruohanyu/heima-shening>

仓库介绍 0/100

用简短的语言来描述一下吧

☐ 开源 (所有人可见) ?

☒ 私有 (仅仓库成员可见)

☐ 初始化仓库 (设置语言、.gitignore、开源许可证)

☐ 设置模板 (添加 Readme、Issue、Pull Request 模板文件)

☐ 选择分支模型 (仓库创建后将根据所选模型创建分支)

创建

- 拷贝远程仓库链接

快速设置— 如果你知道该怎么操作, 直接使用下面的地址

HTTPS SSH <https://gitee.com/shuiruohanyu/heima-shening>

我们强烈建议所有的git仓库都有一个 README, LICENSE, .gitignore 文件

初始化 readme 文件

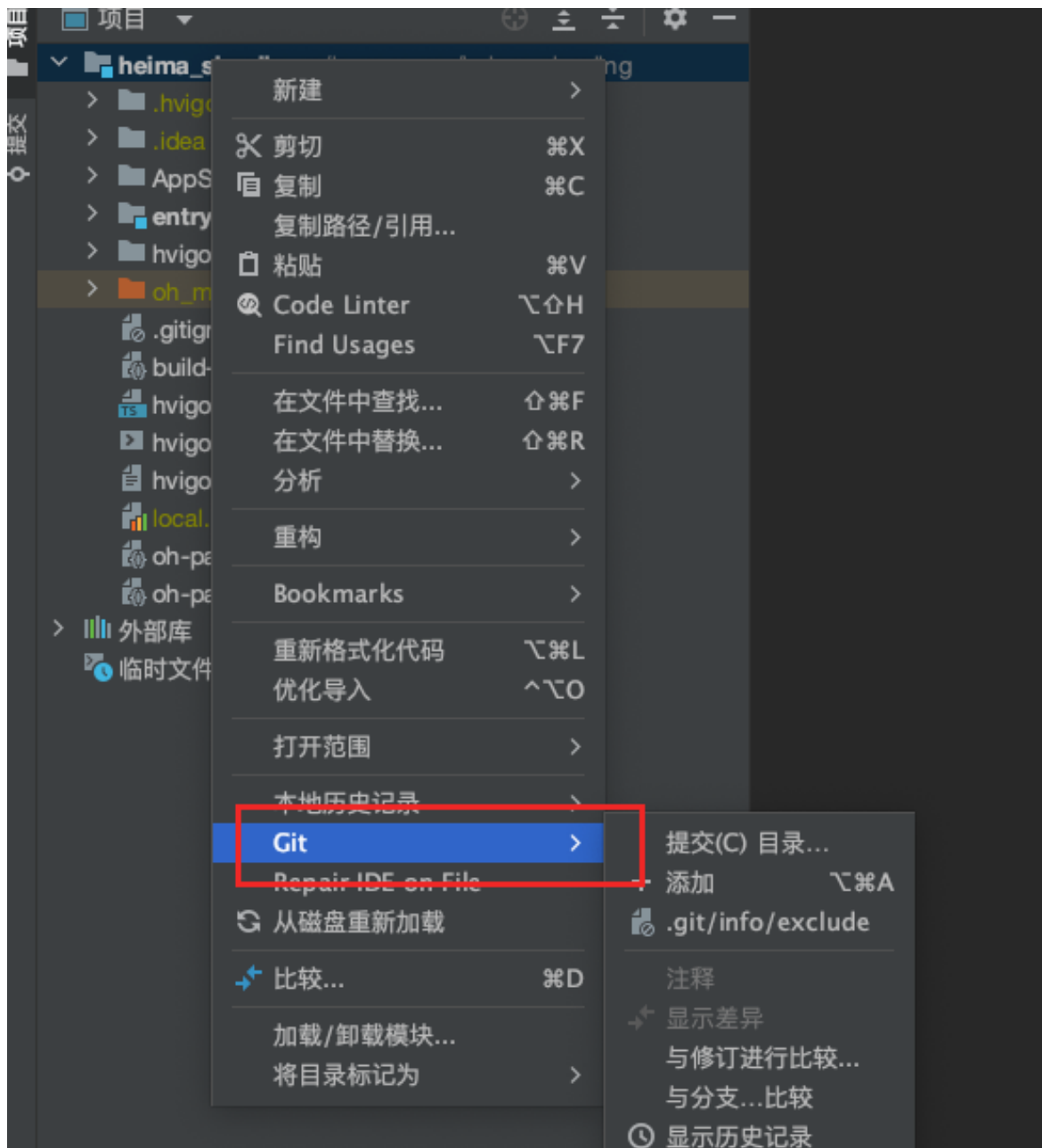
Git入门? 查看 [帮助](#), [Visual Studio](#) / [TortoiseGit](#) / [Eclipse](#) / [Xcode](#) 下如何连接本站, [如何导入仓库](#)

简易的命令行入门教程:

- 找到创建项目的目录下初始化仓库 并添加远程仓库 推送

```
$ git init
$ git add .
$ git commit -m "初始化神领物流项目"
$ git remote add origin <远程仓库地址>
$ git push -u origin master
```

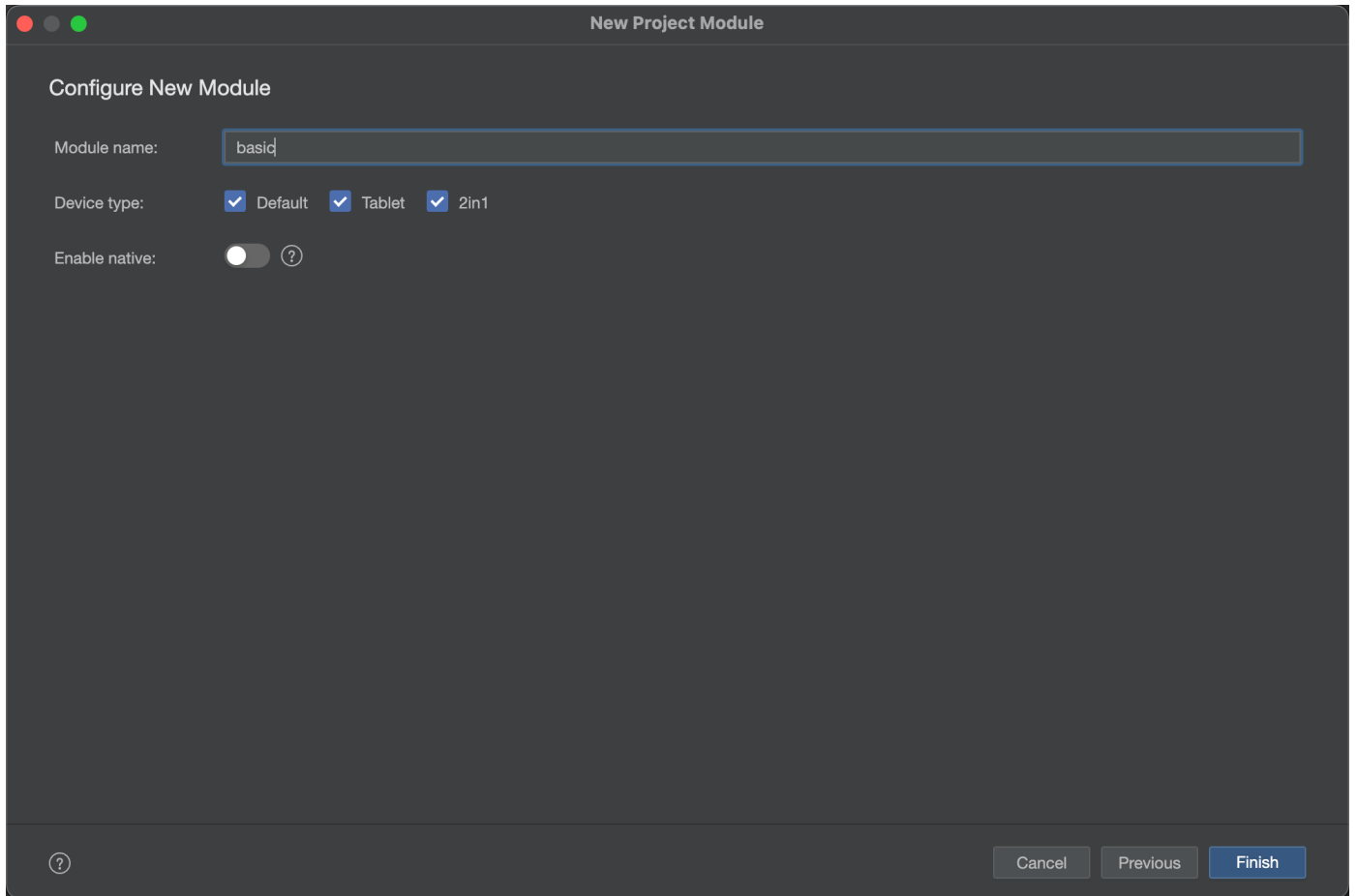
- 完成这一步之后, 我们就可以在IDE来操作git了



## 创建一个静态har包， 导入素材和资源

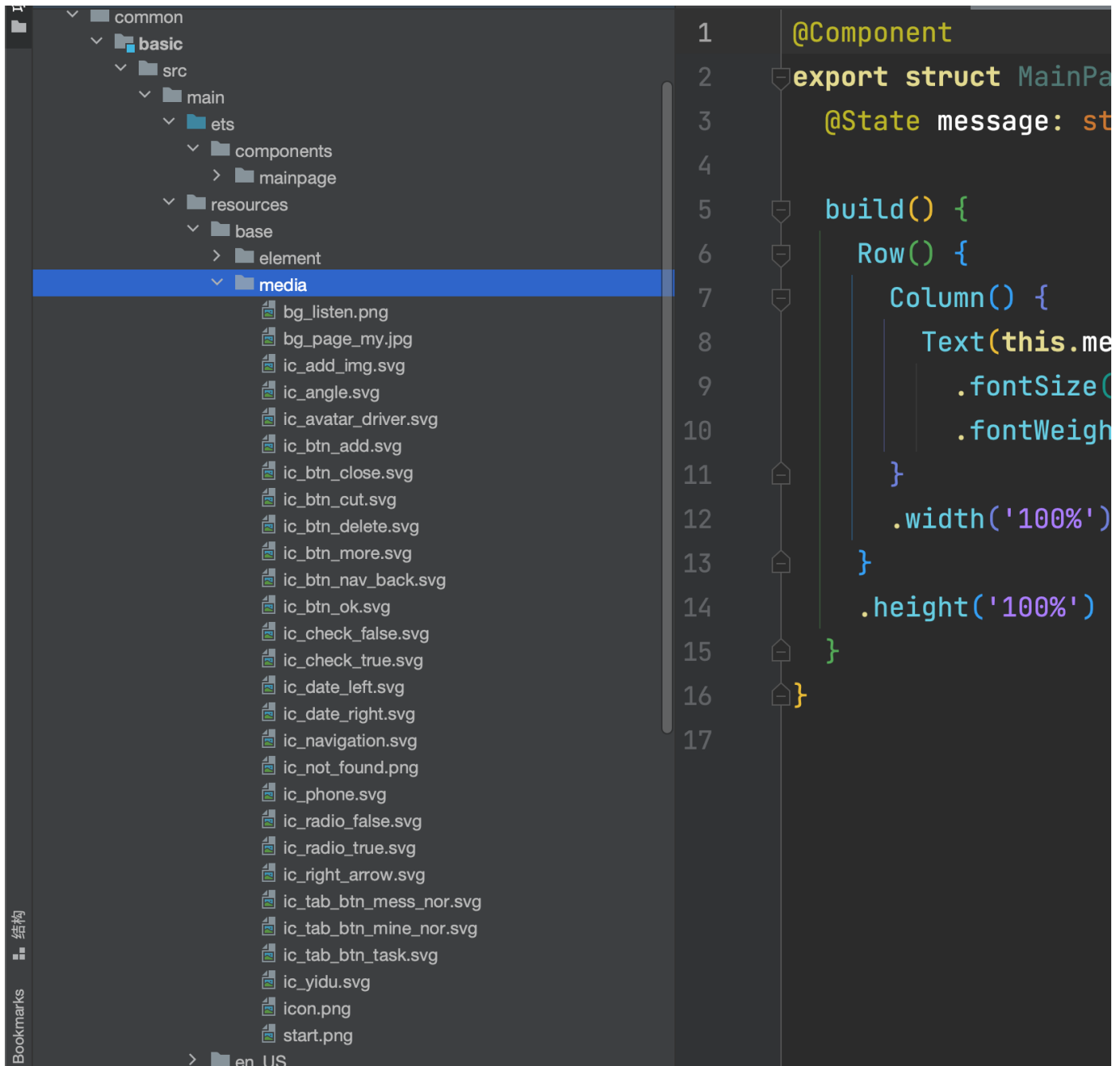
本次项目采用单层架构 + har公共包的模式进行开发

- 新建一个common目录, 在目录下新建一个static静态模块



[media.zip](#)

1. 下载该media文件夹，将所有的图标素材拖动到静态资源包下的 `src/main/resources/base/media` 下



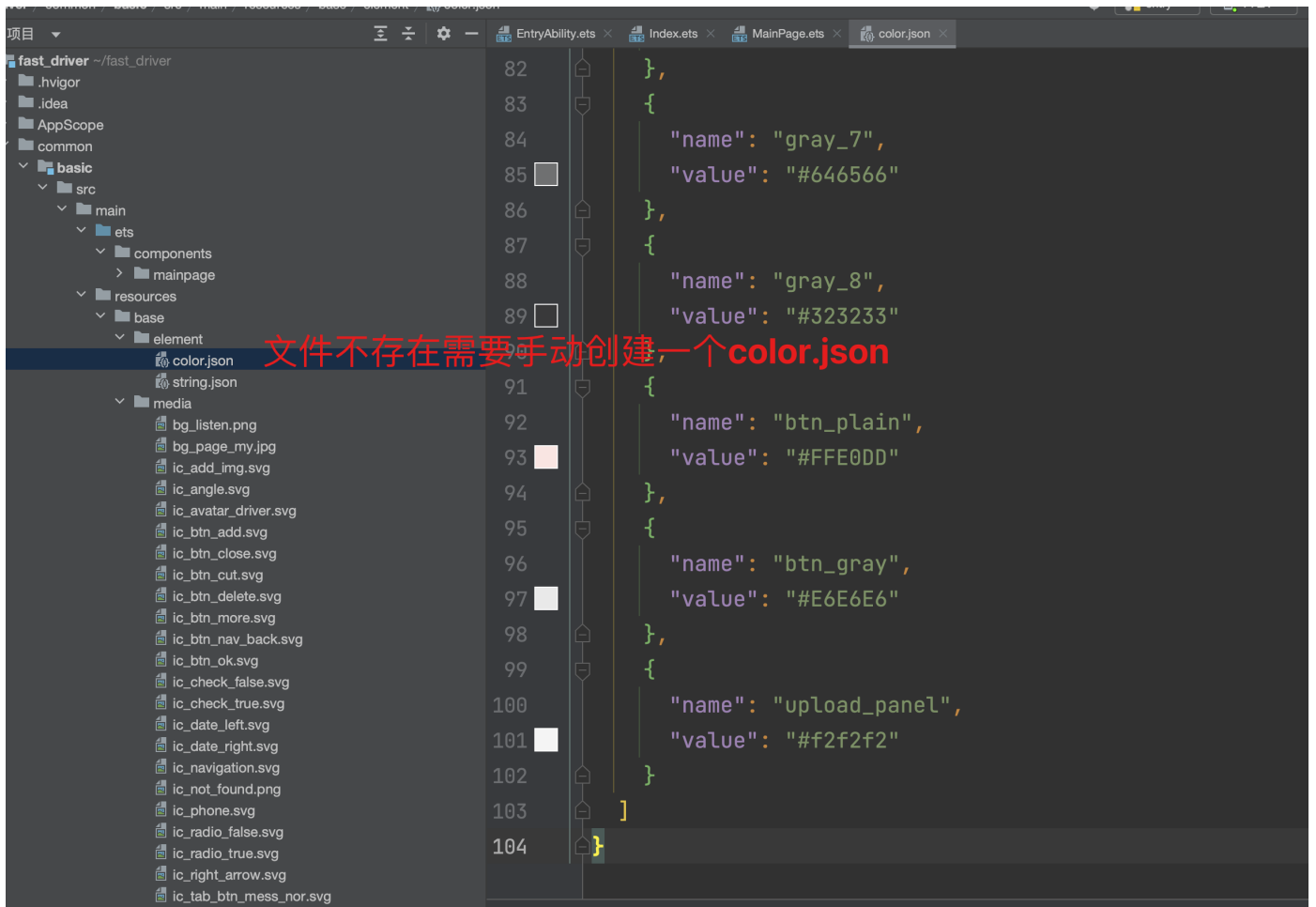
- 导入色值

```
{
  "color": [
    {
      "name": "start_window_background",
      "value": "#FFFFFF"
    },
    {
      "name": "primary",
      "value": "#FE6A3D"
    },
    {
      "name": "primary_disabled",
      "value": "#FADCD9"
    },
  ],
}
```



```
{
  "name": "success",
  "value": "#27BA9B"
},
{
  "name": "warning",
  "value": "#FFAB2E"
},
{
  "name": "danger",
  "value": "#FF4C4C"
},
{
  "name": "text_primary",
  "value": "#2A2929"
},
{
  "name": "text_secondary",
  "value": "#818181"
},
{
  "name": "text_placeholder",
  "value": "#C2C1C1"
},
{
  "name": "border",
  "value": "#D9D9D9"
},
{
  "name": "background_divider",
  "value": "#EEEEEE"
},
{
  "name": "background_page",
  "value": "#F4F4F4"
},
{
  "name": "black",
  "value": "#000000"
},
{
  "name": "white",
  "value": "#FFFFFF"
},
{
  "name": "gray_1",
  "value": "#F7F8FA"
},
{
  "name": "gray_2",
  "value": "#F2F3F5"
},
}
```

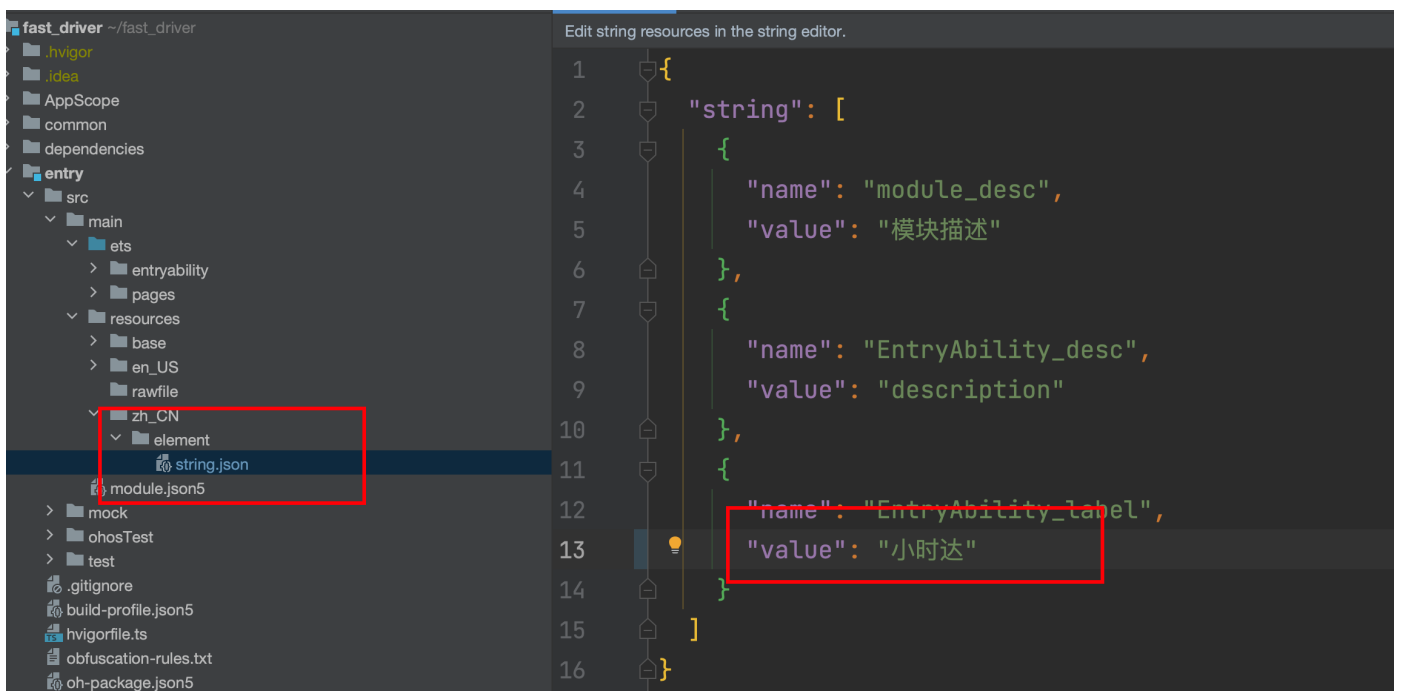
```
{
  "name": "gray_3",
  "value": "#EBEDF0"
},
{
  "name": "gray_4",
  "value": "#DEDEE0"
},
{
  "name": "gray_5",
  "value": "#C8C9CC"
},
{
  "name": "gray_6",
  "value": "#969799"
},
{
  "name": "gray_7",
  "value": "#646566"
},
{
  "name": "gray_8",
  "value": "#323233"
},
{
  "name": "btn_plain",
  "value": "#FFE0DD"
},
{
  "name": "btn_gray",
  "value": "#E6E6E6"
},
{
  "name": "upload_panel",
  "value": "#f2f2f2"
}
]
}
```



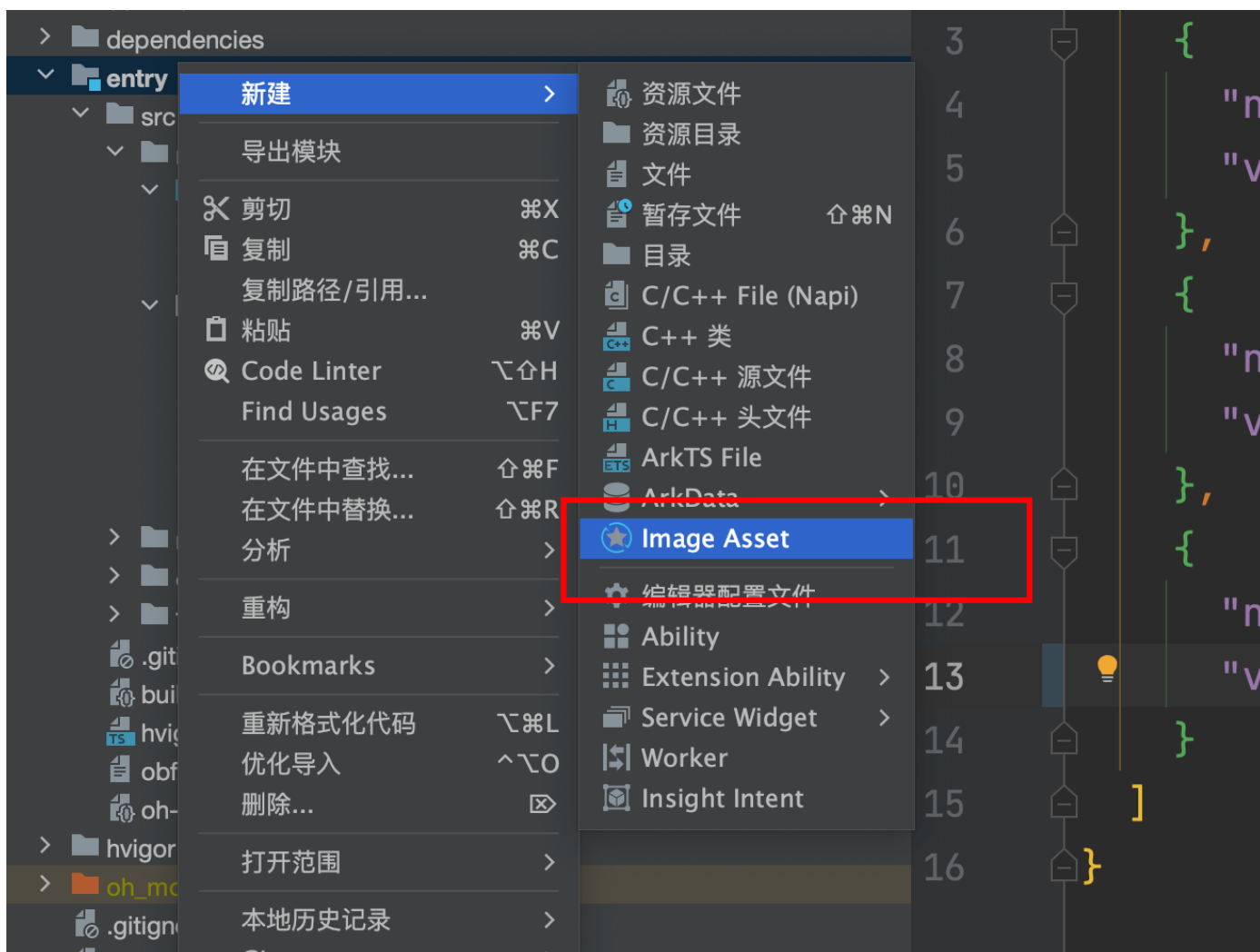
提交代码

## 修改项目名称和图标

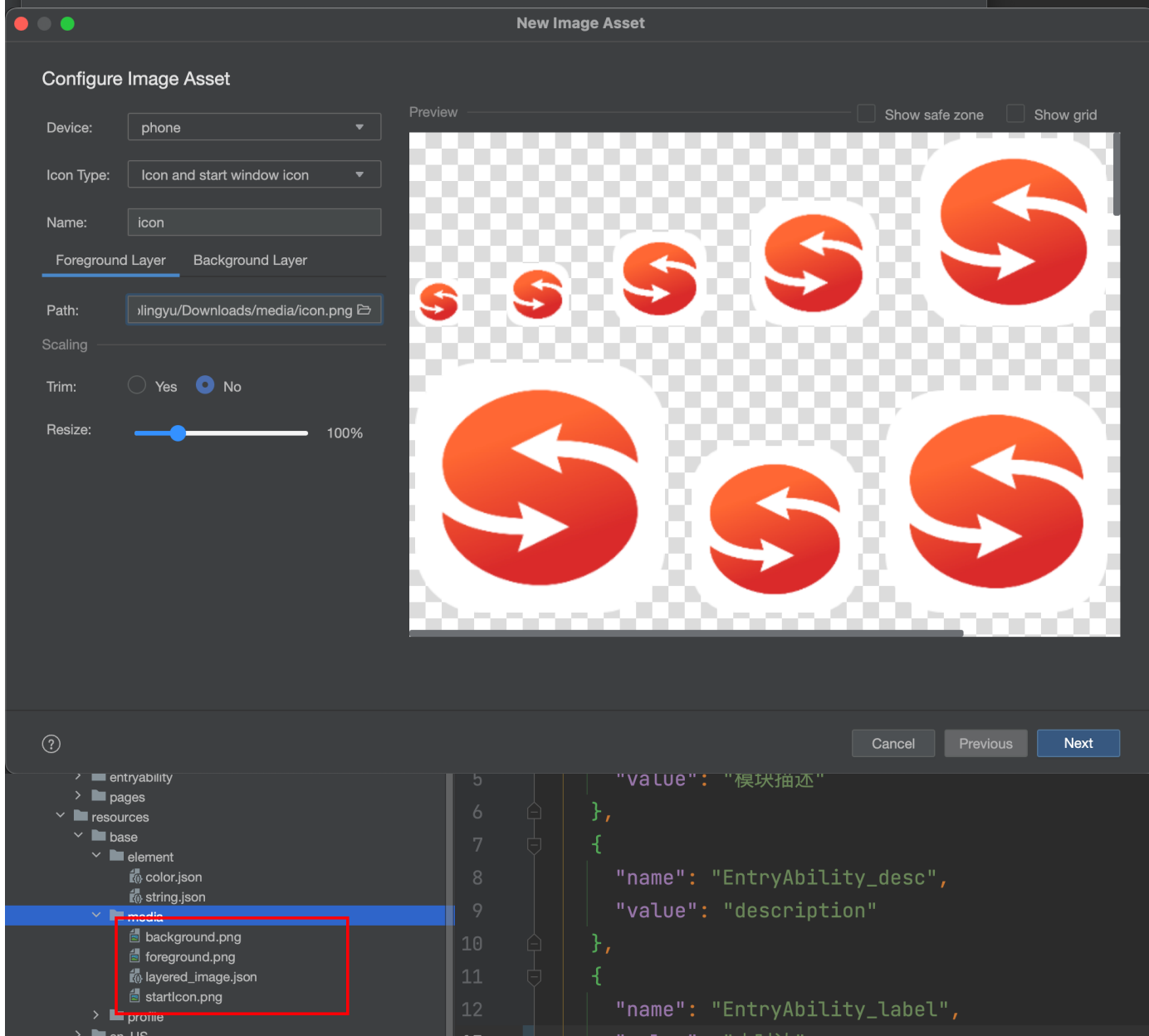
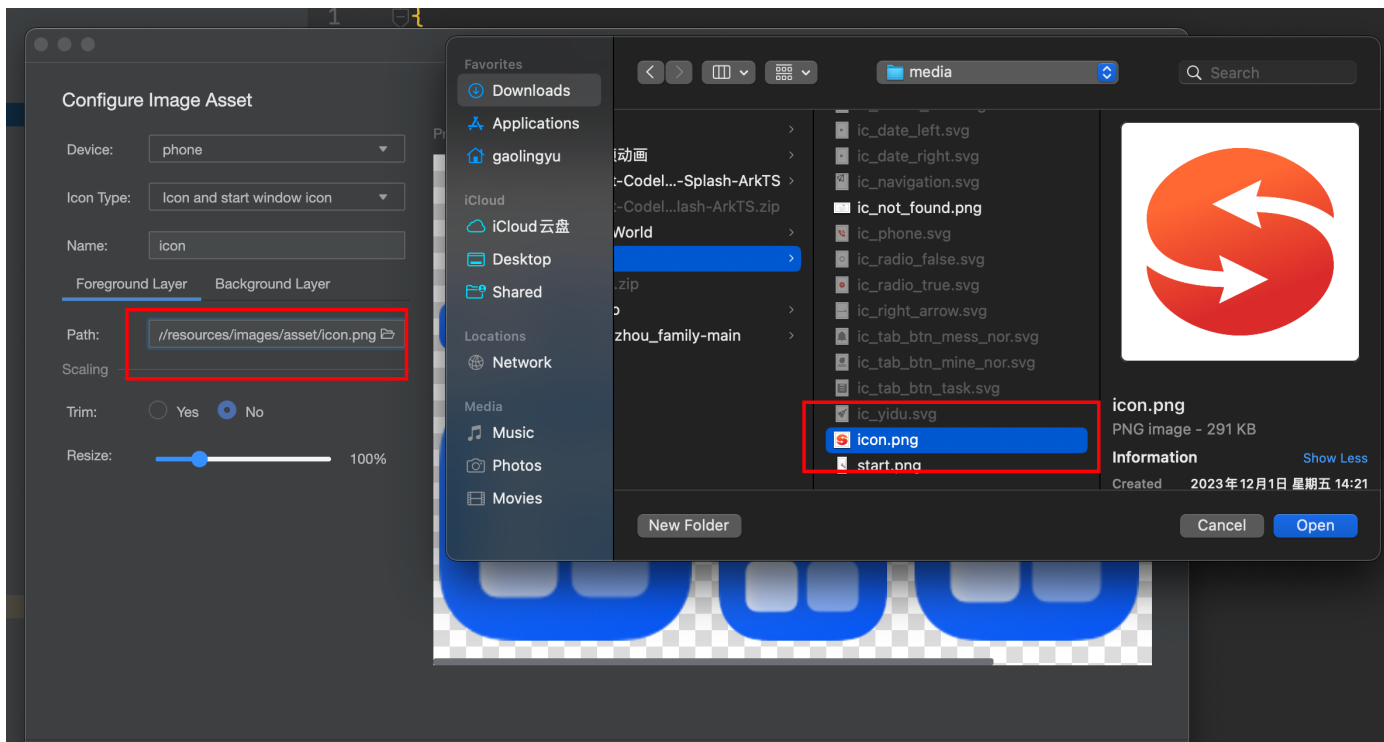
- 修改项目名称

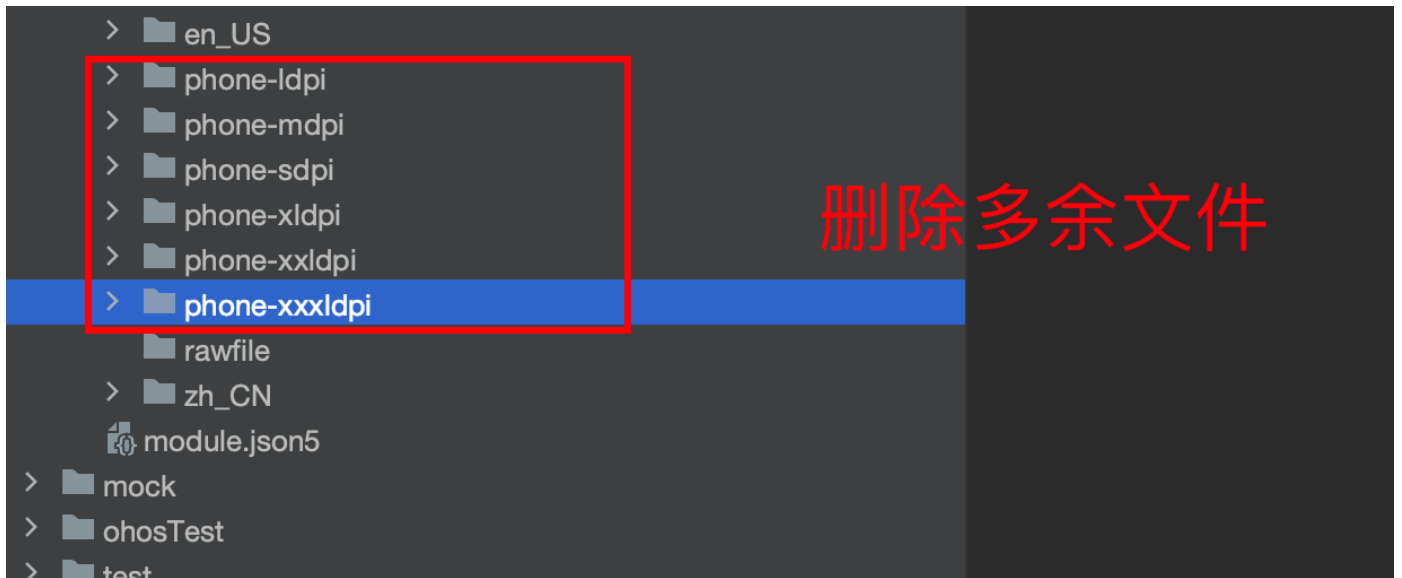
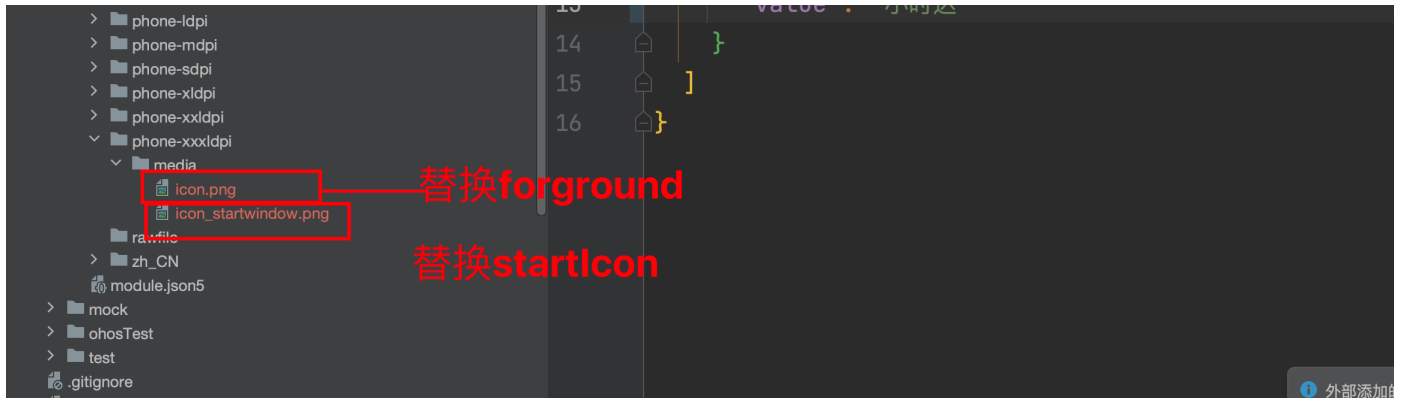


- 通过生成图标工具创建app图标



- 选择图标





提交代码

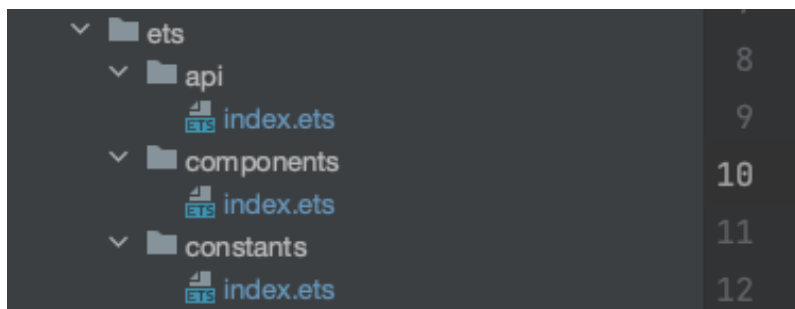
## 公共静态包(HAR)基础目录搭建

在har包中建立如下目录

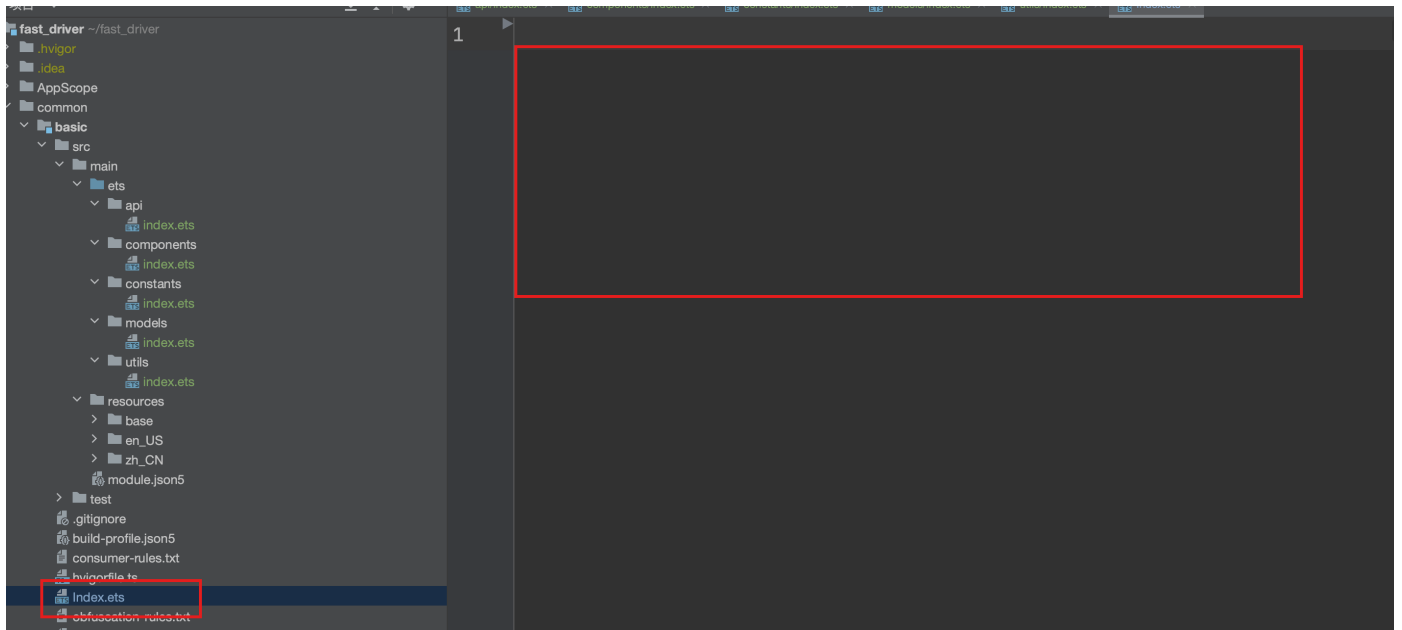
- 按照客户端开发的需要我们要在项目中建立如下结构

|             |               |
|-------------|---------------|
| -ets        |               |
| -api        | 负责存放所有的请求     |
| -components | 负责放置所有的公共组件   |
| -constants  | 负责存放所有的常量     |
| -models     | 负责存放所有的数据类型结构 |
| -utils      | 负责存放所有的工具     |

- 在项目中添加上述的目录，并统一新建一个index.ets文件



- 清空har包下index.ets文件内容



提交代码

## 搭建广告展示页Start

广告页的思路

-华为有广告业务，但是我们不用- ad模块

想自定义广告-

场景： app启动-有广告需求，就打开广告页，没有的话就去登录或者主页

腾讯体育的广告- 启动有广告页，退到后台的情况下，再次进入前台也会有广告



分析- 广告页作为一个app启动的首页，应该是在我们应用启动就进去的。

- 有的app有的需要广告页，有的不需要，搞个配置呗!!!
  - 1. 通过首选项配置存储我们的一些常用配置，比如要不要广告页，还有广告页的路由地址，点击广告页跳转的链接，广告页倒计时的秒数
  - 2. 在入口处进行判断是否需要广告页，需要的话，跳转广告页-广告页根据设置的参数进行渲染
  - 3. 有的同学可能会问，广告页能不能设置-因为运营人员肯定不能每次都去改我们底层的代码-这里我还可以设置成动态的-就是初始化的时候通过请求去读一下云端的请求，然后把我们的图片和一些参数配置下来，这样每次你启动app就是运营人员给你配置的广告和设置了
- 新建一个关于广告类的数据模型-basic- models/advert.ets



```
export class AdvertClass {
  showAd: boolean = false // 显示广告
  adTime: number = 5 // 广告时长
  adUrl?: string = '' // 广告链接
  adImg?: ResourceStr = '' // 广告图片
}
```

- 在model/index.ets中进行统一导出

```
export * from './advert'
```

- 在utils中新建一个关于读取首选项的类，用来读取和设置首选项的广告设置- utils/setting.ets

```
import { AdvertClass } from '../models'
import preferences from '@ohos.data.preferences'
import { USER_SETTING, USER_SETTING_AD } from '../constants'
// 默认广告选项
const defaultAd: AdvertClass = {
  showAd: true,
  adTime: 5,
  adImg: $r('app.media.start')
}
export class UserSettingClass {
  context: Context
  constructor(context: Context) {
    this.context = context
  }
  // 获取存储用户信息的首选项仓库
  getStore () {
    return preferences.getPreferences(this.context, USER_SETTING)
  }
  // 设置用户广告设置
  async setUserAd (ad: AdvertClass) {
    const adStore = await this.getStore()
    await adStore.put(USER_SETTING_AD, JSON.stringify(ad))
    await adStore.flush()
  }
  // 获取广告配置
  async getUserAd (): Promise<AdvertClass> {
    const adStore = await this.getStore()
    return JSON.parse(await adStore.get(USER_SETTING_AD, JSON.stringify(defaultAd)) as string) as AdvertClass
  }
}
```

在上面代码中，我们设计了读取和设置广告的两个方法，并且提供了一个默认广告的设置

- 在utils中统一导出

```
export * from './setting'
```

- 上面还用到了两个常量，我们同样需要在constants目录下定义一个文件专门用来记录-setting

```
export const USER_SETTING = 'fast_driver_setting' // 用来存储用户设置的首选项的key
export const USER_SETTING_AD = 'fast_driver_setting_ad' // 用来存储用户设置广告首选项的key
```

- 同样在constants/index.ets文件中导出

```
export * from './setting'
```

- 在basic/Index.ets统一导出

```
export * from './src/main/ets/models'
export * from './src/main/ets/constants'
export * from './src/main/ets/utils'
```

## entry模块-oh-package.json5

- 在ability中引入该har包依赖

```
{
  "name": "entry",
  "version": "1.0.0",
  "description": "Please describe the basic information.",
  "main": "",
  "author": "",
  "license": "",
  "dependencies": {
    "@hm/basic": "file:../common/basic"
  }
}
```

## ability中判断

- 导入包

```
import { AdvertClass, UserSettingClass } from '@hm/basic'
```

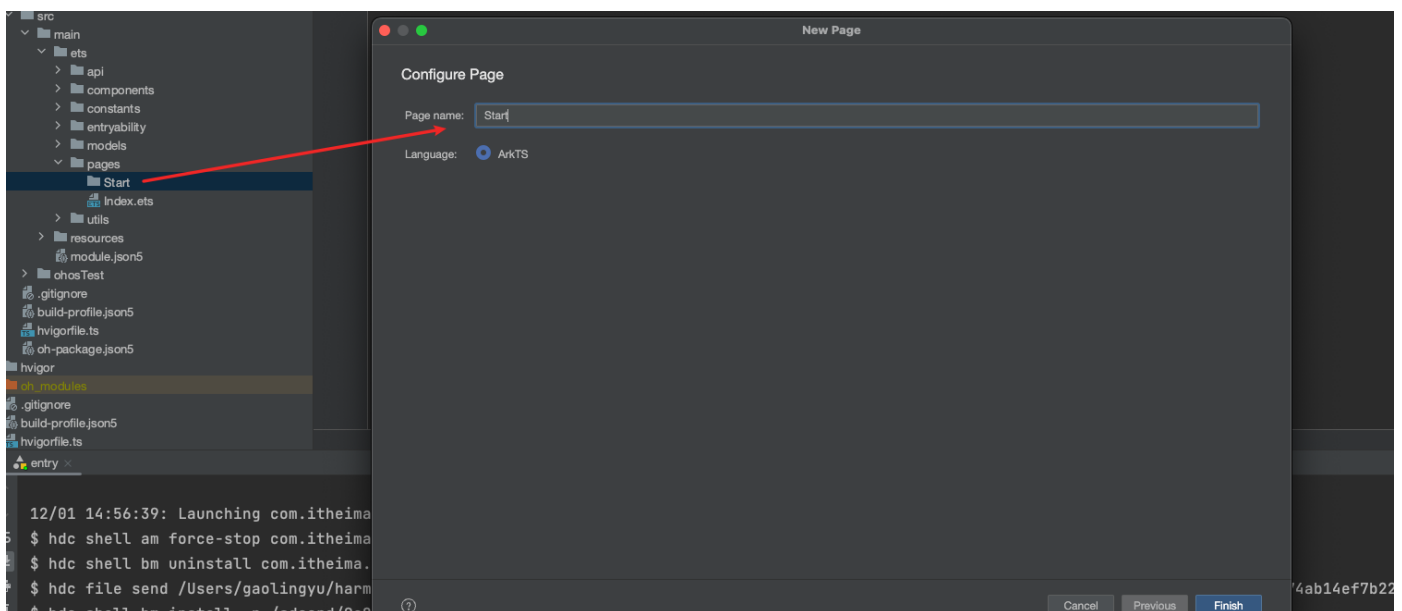
- 判断

```
async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
  // Main window is created, set main page for this ability
  // 检查是否有广告
  const setting = new UserSettingClass(this.context) // getContext拿到的是undefined
```

```
//
const ad = await new Promise<AdvertClass>((resolve, reject) => {
  setTimeout(() => {
    resolve(defaultAd) // 广告信息
  }, 500)
  // resolve()
})
// 通过模拟请求拿到广告信息
await setting.setUserAd(ad) // 写入首选项
if(ad.showAd) {
  // 展示广告的情况 展示广告页 有时限
  // 创建一个子窗口 子窗口加载广告 广告播完 窗口销毁
}
windowStage.loadContent('pages/Index'); // 正常加载页面
}
```

这里我们模拟了一个请求，给了一个默认广告， 写入首选项-正常加载主页

1. 在pages下新建Start目录，下面新建Start的page页面



- 实现Start页的页面结构及倒计时逻辑

```
import { UserSettingClass, AdvertClass } from '@hm/basic'

@Entry
@Component
struct Start {
  userSetting: UserSettingClass = new UserSettingClass(getContext(this))
  @State
  adObj: AdvertClass = {
    showAd: false,
    adTime: 0
  }
  timer: number = -1

  async aboutToAppear() {
```

```

    this.adObj = await this.userSetting.getUserAd()
    this.timer = setInterval(() => {
      if(this.adObj.adTime === 0) {
        clearInterval(this.timer)
        return
      }
      this.adObj.adTime--
    }, 1000)
  }
  build() {
    Stack({ alignContent: Alignment.TopEnd }) {
      Image(this.adObj.adImg).objectFit(ImageFit.Cover)
      Text(`${this.adObj.adTime}秒后跳过`)
        .padding({ left: 10, right: 10 })
        .margin({ right: 20, top: 20 })
        .height(30)
        .fontSize(14)
        .borderRadius(15)
        .backgroundColor($r("app.color.background_page"))
        .textAlign(TextAlign.Center)

      }.height('100%').width('100%')
    }
  }
}

```

- 使用子窗口模式加载广告

我们可以使用windowStage的createSubWindow来实现在当前页面上创建一个窗口

```

if (result.showAd) {
  const win = await windowStage.createSubWindow("ad_window")
  await win.showWindow()
  win.setUIContent("pages/Start/Start")
}

```

- 广告页在广告结束或者点击跳过广告时，关闭广告

Start/Start页面

```

closeWin () {
  window.findWindow("ad_window").destroyWindow()
}

```

- 完成Start页面代码

```

import { UserSettingClass, AdvertClass } from '@hm/basic'
import { window } from '@kit.ArkUI'

@Entry
@Component

```

```

struct Start {
  userSetting: UserSettingClass = new UserSettingClass(getContext(this))
  @State
  adObj: AdvertClass = {
    showAd: false,
    adTime: 0
  }
  timer: number = -1
  closeWin () {
    window.findWindow("ad_window").destroyWindow()
  }
  async aboutToAppear() {
    this.adObj = await this.userSetting.getUserAd()
    this.timer = setInterval(() => {
      if(this.adObj.adTime === 0) {
        clearInterval(this.timer)
        this.closeWin()
        return
      }
      this.adObj.adTime--
    }, 1000)
  }
  aboutToDisappear(): void {
    clearInterval(this.timer)
  }
  build() {
    Stack({ alignContent: Alignment.TopEnd }) {
      Image(this.adObj.adImg).objectFit(ImageFit.Cover)
      Text(`${this.adObj.adTime}秒后跳过`)
        .padding({ left: 10, right: 10 })
        .margin({ right: 20, top: 20 })
        .height(30)
        .fontSize(14)
        .borderRadius(15)
        .backgroundColor($r("app.color.background_page"))
        .textAlign(TextAlign.Center)
        .onClick(() => {
          this.closeWin()
        })

    }.height('100%').width('100%')
  }
}

```

这里请注意-如果需要使用线上的广告图片，需要开启网络权限

完整代码-EntryAbility

```

import { AbilityConstant, UIAbility, Want } from '@kit.AbilityKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { window } from '@kit.ArkUI';

```

```

import { AdvertClass, UserSettingClass } from '@hm/basic'

export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onCreate');
  }

  onDestroy(): void {
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onDestroy');
  }

  async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
    // Main window is created, set main page for this ability
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onWindowStageCreate');
    // 这里要尝试去读一下运营的配置-我们现在还没有实现接口, 直接模拟一下
    const userSetting = new UserSettingClass(this.context)
    const result = await new Promise<AdvertClass>((resolve, reject) => {
      setTimeout(() => {
        resolve({
          showAd: true,
          adTime: 5,
          adImg: $r("app.media.start")
        } as AdvertClass)
      }, 500)
    })
    await userSetting.setUserAd(result) // 写入首选项
    if (result.showAd) {
      const win = await windowStage.createSubWindow("ad_window")
      await win.showWindow()
      win.setUIContent("pages/Start/Start")
    }
    windowStage.loadContent('pages/Index');
  }

  onWindowStageDestroy(): void {
    // Main window is destroyed, release UI related resources
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onWindowStageDestroy');
  }

  onForeground(): void {
    // Ability has brought to foreground
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onForeground');
  }

  onBackground(): void {
    // Ability has back to background
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onBackground');
  }
}

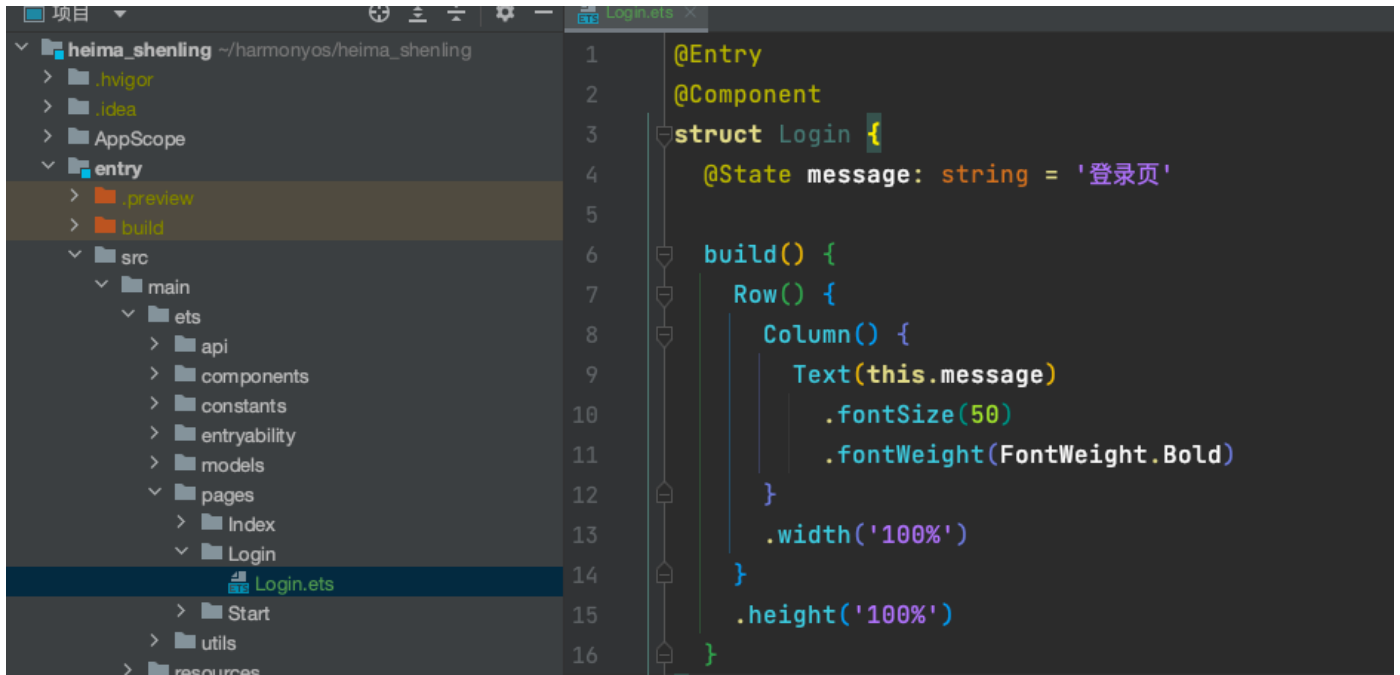
```

# 初始化App检查token

由于手机端是必须要求用户登录的，所以我们需要在跳转到主页前检查是否有token，有token的话跳转到主页，没有token的话跳转到登录

## Entry模块

### 1. 新建登录页- pages/Login/Login



## Har包basic

### 2. 在广告页跳转前检查token，有的话跳转到主页 Index，没有的话跳转到Login

- 声明一个token\_key的常量-constants/token.ets

```
export const TOKEN_KEY = 'user_token'
```

- 在constant/index.ts中导出

```
export * from './token'
```

这里需要在首选项中在声明两个方法

```

async setUserToken (token: string) {
  const adStore = await this.getStore()
  await adStore.put(TOKEN_KEY, token)
  // 写入磁盘
  await adStore.flush()
}

async getUserToken (): Promise<string> {
  const adStore = await this.getStore()
  return await adStore.get(TOKEN_KEY, "") as string
}

```

## Entry模块

```

async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
  // Main window is created, set main page for this ability
  hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onWindowStageCreate');
  // 这里要尝试去读一下运营的配置-我们现在还没有实现接口, 直接模拟一下
  const userSetting = new UserSettingClass(this.context)
  const result = await new Promise<AdvertClass>((resolve, reject) => {
    setTimeout(() => {
      resolve({
        showAd: true,
        adTime: 5,
        adImg: $r("app.media.start")
      } as AdvertClass)
    }, 500)
  })
  await userSetting.setUserAd(result) // 写入首选项
  if (result.showAd) {
    const win = await windowStage.createSubWindow("ad_window")
    await win.showWindow()
    win.setUIContent("pages/Start/Start")
  }
  const token = await userSetting.getUserToken()
  if(token) {
    AppStorage.setOrCreate(TOKEN_KEY, token)
    windowStage.loadContent('pages/Index');
  }else {
    windowStage.loadContent("pages/Login/Login")
  }
}

```

:::info

- 提交代码
- ...

## 登录页基本功能实现





Entry模块

- 拷贝结构

```
@Entry
@Component
struct Login {
  @Styles
  loginStyle() {
    .backgroundColor('#fff')
    .border({ color: $r('app.color.background_divider'), width: { bottom: 1 } })
```

```

        .width('100%')
        .height(58)
        .borderRadius(0)
    }

    build() {
        Column() {
            // 顶部标题
            Text("小时达").fontColor($r('app.color.text_primary')).fontSize(18).height(25)
            // 账号登录
            Row() {
                Text('账号登
录').fontColor($r('app.color.text_primary')).fontSize(24).fontWeight(FontWeight.Bold)
                Row() {
                    Text("账号登
录").fontColor($r('app.color.primary')).fontSize(16).fontWeight(FontWeight.Bold)
                    Image($r("app.media.ic_angle")).width(10).height(10).margin({ left: 5 })
                }
            }
            .width('100%')
            .justifyContent(FlexAlign.SpaceBetween)
            .margin({ top: 50, bottom: 50 })

            // 用户名输入框
            TextInput({ placeholder: '请输入账号' })
                .loginStyle()

            // 密码框
            TextInput({ placeholder: '请输入密码' })
                .loginStyle()
                .type(InputType.Password) // 密码框
                .showPasswordIcon(true) // 显示密码按钮

            // 登录按钮
            Button({ type: ButtonType.Capsule }) {
                Row() {
                    LoadingProgress().width(20).height(20).margin({ right: 12
                }).color($r('app.color.white'))
                    Text('登录').fontColor($r('app.color.white'))
                }
            }
            .backgroundColor($r('app.color.primary_disabled'))
            .width('100%')
            .height(50)
            .margin({ top: 50 })
        }
        .padding({ left: 32, right: 32 })
        .margin({ top: 40 })
    }
}

```

- 登录页中定义状态loading来控制按钮进度条的显示和隐藏

```

@State showLoading: boolean = false

....

if(this.showLoading) {
    // 显示进度条再显示
    LoadingProgress().width(20).height(20).margin({ right: 12
}).color($r('app.color.white'))
}
....

```

## Entry模块

\*\* 登录信息属于业务的数据，所以内容留存在当前的entry模块中\*\*

:::info

- 接下来我们来把表单数据进行一下双向绑定-和类型定义
- ...
- 首先在models下新建user.ts文件

这里我们拷贝接口文档中的interface接口声明-要去除默认自带的

```

export interface LoginForm {
    /**
     * 登录账号
     */
    account: string;
    /**
     * 登录密码
     */
    password: string;
}

```

- 使用的i2c插件，直接将interface转化成class
- 生成class

```
$ i2c ./user.ets
```

```

export interface LoginForm {
    /** 登录账号 */
    account: string;
    /** 登录密码 */
    password: string;
}

export class LoginFormModel implements LoginForm {
    account: string = ''
    password: string = ''

    constructor(model: LoginForm) {

```

```

    this.account = model.account
    this.password = model.password
  }
}

```

```

1  export interface LoginForm {
2    /**
3     * 登录账号
4     */
5    account: string;
6    /**
7     * 登录密码
8     */
9    password: string;
10 }
11 export class LoginFormModel implements LoginForm {
12   account: string = ''
13   password: string = ''
14
15   constructor(model: LoginForm) {
16     this.account = model.account
17     this.password = model.password
18   }
19 }
20

```

- 然后在models/index.ets中统一导出

```
export * from './user'
```

接下来就可以通过 引用models来使用所有模块的类型了

- 在Login页面中导入类型

```
import { LoginFormModel } from '../..models'
```

- 在Login组件中声明State数据

```
@State
accountForm: LoginFormModel = new LoginFormModel({
  account: '',
  password: ''
})
```

- 完成账户和密码的双向绑定

```
// 用户名输入框
TextInput({ placeholder: '请输入账号', text: this.accountForm.account })
  .loginStyle().onChange(value => {
    this.accountForm.account = value
  })

// 密码框
TextInput({ placeholder: '请输入密码', text: this.accountForm.password })
  .loginStyle()
  .type(InputType.Password) // 密码框
  .showPasswordIcon(true)
  .onChange((value) => {
    this.accountForm.password = value
  }) // 显示密码按钮
```

注意：这里不能使用\$\$ 因为我们\$\$只支持单层数据的绑定，出现嵌套的情况还得使用监听方法

- 如果账户名和密码有值，则使按钮可用

```
// 通过校验
getFormValidate() {
  if (this.accountForm.password && this.accountForm.account) {
    return true
  }
  return false
}
```

- 条件判断使用何种颜色和点击反馈

```
// 登录按钮
Button({ type: ButtonType.Capsule, stateEffect: this.getFormValidate() }) {
  Row() {
    if (this.showLoading) {
      // 显示进度条再显示
      LoadingProgress()
        .width(20)
        .height(20)
        .margin({ right: 12 })
        .color($r('app.color.white'))
    }
    Text('登录').fontColor($r('app.color.white'))
  }
}
```

```

    }
    .backgroundColor(this.getFormValidate() ? $r('app.color.primary') :
$r('app.color.primary_disabled'))
    .width('100%')
    .height(50)
    .margin({ top: 50 }).enabled(this.getFormValidate())

```

提交代码

## 封装统一请求工具request

- 申请基础网络权限-在module.json5中配置

```

"requestPermissions": [
  {
    "name": "ohos.permission.INTERNET",
  }
]

```

### Basic模块

- 在constants目录中设置baseUrl基础地址常量

```
export const BASE_URL = 'https://slwl-api.itheima.net/driver' // 请求基础地址
```

- 统一导出

```
export * from './url'
```

- 在utils下新建request.ets文件，导入请求包和baseUrl

```
import http from '@ohos.net.http'
import { BASE_URL } from '../constants'
```

- 封装泛型工具-models/index.ets

工具的目的- 让返回的结构完成统一

```

export class ResponseData<T> {
  code: number = 0
  msg: string = ""
  data: T | null = null // 这里应该必须只能这么声明!!!
}

```

- 封装一个公共的request方法来支持get/post/put/delete方法

[http官网链接](#)

```

import { BASE_URL, TOKEN_KEY } from "../constants"
import http from '@ohos.net.http'; // 原生的请求地址
import { promptAction, router } from '@kit.ArkUI';
import { UserSettingClass } from '.';
import { ResponseData } from '../models'

// 请求工具 get/put/delete/post/patch
// url 类型 参数
// 任何对象都可以as object
async function requestHttp<T>(url: string = "", method: http.RequestMethod =
http.RequestMethod.GET, data?: object): Promise<T> {
    // http需要创建一个请求对象
    const httpRequest = http.createHttp() // 创建一个请求对象- 如果你想用单例 那么请不要在请求时销毁
    let urlStr = BASE_URL + url // 基础地址拼接url地址
    // 进行处理
    // 官方文档上写的是ok的, 但是运行有问题, 虽然说了 如果get类型会进行拼接地址, 但是实际并没有
    // get/post区别是什么
    if (method === http.RequestMethod.GET) {
        // data参数都拼接到地址上 ?a=1&b=2&c=3
        if (data && Object.keys(data).length) {
            urlStr += "?" + Object.keys(data).map(key => {
                return `${key}=${data[key]}`
            }).join("&")
        }
    }
    // 组装参数
    const config: http.HttpRequestOptions = {
        extraData: method !== http.RequestMethod.GET ? data : "", // 当请求类型不等于get时 复制
        header: {
            "Content-Type": "application/json",
            Authorization: AppStorage.get(TOKEN_KEY) || "", // 设置请求头的token 每个接口都有
        },
        method,
        readTimeout: 10000 // 如果多少秒没响应就断开
    }
    try {
        // 发请求
        const result = await httpRequest.request(urlStr, config) // 发请求
        // 这里要处理一些异常
        // 判断状态码 http状态码 业务状态码
        if (result.responseCode === 401) {
            // 401 就表示token失效 超时 或者没传
            // 移动端产品应该去做无感刷新
            // token refresh_token => 换取新的token => 重新发请求
            // 提示错误 -删除目前的token 跳到登录页
            promptAction.showToast({ message: '登录失效' })
            // 删除token - 删两个地址 首选项- 全局状态
            new UserSettingClass(getContext()).setUserToken("") // 清空首选项token
            AppStorage.set(TOKEN_KEY, "") // 只能设置 不能删除
            router.replaceUrl({
                url: 'pages/Login/Login' // 跳转到登录页
            })
        }
    }
}

```

```

    // Hap har
    return Promise.reject(new Error("登录失效"))
  }
  else if (result.responseCode === 404) {
    promptAction.showToast({ message: '请求地址错误' })
    return Promise.reject(new Error("请求地址错误"))
  } else {
    // 假设已经成功 此时不一定成功哦 还有业务状态码呢 只有业务状态码 200的时候才成功
    const res = JSON.parse(result.result as string) as ResponseData<T> // { code, data,
msg }
    if (res.code === 200) {
      // 才认为这次请求真的成功了
      return res.data as T
    } else {
      // 业务错了
      promptAction.showToast({ message: "服务器异常" }) // res.msg没处理好 有可能出来一堆服务
器语言
      return Promise.reject(new Error(res.msg)) // 业务错误 请求终止
    }
  }
} catch (error) {
  promptAction.showToast({ message: error.message }) // res.msg没处理好 有可能出来一堆服务
器语言
  return Promise.reject(error) // 业务错误 请求终止
}
}

export class Request {
  static get<T>(url: string, data?: object): Promise<T> {
    return requestHttp<T>(url, http.RequestMethod.GET, data)
  }

  static post<T>(url: string, data?: object): Promise<T> {
    return requestHttp<T>(url, http.RequestMethod.POST, data)
  }

  static delete<T>(url: string, data?: object): Promise<T> {
    return requestHttp<T>(url, http.RequestMethod.DELETE, data)
  }

  static put<T>(url: string, data?: object): Promise<T> {
    return requestHttp<T>(url, http.RequestMethod.PUT, data)
  }
}

```

- 在utils中导出

```
export * from './request'
```



:::info

总结：上面请求工具总共处理了这么几件事

- 封装请求-拼接基础地址-传入参数
- 解构返回的数据-判断状态
- 200 认为成功直接返回data数据
- 200以外 401 认为是token超时
- 500认为是服务器任务
- 最后暴露一个类的四个静态方法 可以方便的调用 get/put/delete/post
- 提交代码
- ...

## 封装请求登录的api

### Entry模块

- 在api下新建user.ts，并在index.ts导出

```
import { Request } from '@hm/basic'
import { LoginFormModel } from '../models'
// 登录接口实现
export const login = (data: LoginFormModel) => {
  return Request.post<string>("/login/account", data)
}
```

- index.ets中导出

```
export * from './user'
```

:::info

封装api是为后续更灵活的使用

- Request.post这里面的string相当于传入的泛型数据，指代返回的data是个string
- 提交代码
- ...

## 登录存储token跳转主页

- 首先大家先通过这个网址

[神领TMS管理系统](#)

- 去注册一个属于自己的账号
- 在登录页引入登录api-存入token-跳转主页
- 导入api

```
import router from '@ohos.router'
import { login } from '.././api'
import { TOKEN_KEY, UserSettingClass } from '@hm/basic'
```

// 调用登录接口

```
async login() {
  try {
    this.showLoading = true
    const token = await login(this.accountForm)
    AppStorage.setOrCreate(TOKEN_KEY, token) // 写入全局状态
    new UserSettingClass(getContext(this)).setUserToken(token) // 存入首选项
    this.showLoading = false
    router.replaceUrl({
      url: 'pages/Index'
    })
  } catch (error) {

  } finally {
    this.showLoading = false
  }
}
```

- 注册事件位置

```
Button("登录")...
  .onClick(() => {
    this.login()
  })
```

- 当我们谈起键盘之后，账号和密码也可以直接提交

```
TextInput()
...
.onSubmit(() => {
  if(this.getFormValidate()) {
    this.login()
  }
})
```

:::info

总结

- 登录需要自己去网站上注册账号-移动端暂时没有注册接口
- 请求的时候需要准确的准备返回的响应体
- 登录成功之后-存入token- 跳转页面
- 这里登录失败为啥没处理，因为在请求模块的位置，我们判断了200，只要是200 我们就终止了请求，不会再往下走啦

- 最后不要忘记提交代码
- ...

## 布局首页结构

### basic模块

- 新建models/tab.ets

```
export class TabClass {  
    title: string = ''  
    name: string = ''  
    icon?: ResourceStr  
}
```

- 在models/index.ets中导出

```
export * from './tab'
```

### entry模块

- 将pages/Index调到pages/Index/Index
- 引入类型

```
import { TabClass } from '@hm/basic'
```

- 在页面中声明数据

```
@State tabsData: TabClass[] = [{  
    title: '任务',  
    name: 'task',  
    icon: $r("app.media.ic_tab_btn_task")  
},{  
    title: '消息',  
    name: 'message',  
    icon: $r("app.media.ic_tab_btn_mess_nor")  
},{  
    title: '我的',  
    name: 'my',  
    icon: $r("app.media.ic_tab_btn_mine_nor")  
}]
```



:::info

- 上图得知，首页底部分为三个区域，任务，消息，我的
- 任务里面有 待提货，在途，已完成
- ...
- 底部为第一级别，上部分内容为第二级别，我们先完成第一级别

首先把我们的首页设置成Index/Index，改成目录结构的嵌套

- 另外需要调整原来路由跳转的地方- Login/EntrtAbility

采用Tabs组件就可以轻松实现

```

import { TabClass } from '@hm/basic'
@Entry
@Component
struct Index {
  @State
  currentIndex: number = 0 // 当前激活项
  @State tabsData: TabClass[] = [{
    title: '任务',
    name: 'task',
    icon: $r("app.media.ic_tab_btn_task")
  }, {
    title: '消息',
    name: 'message',
    icon: $r("app.media.ic_tab_btn_mess_nor")
  }, {
    title: '我的',
    name: 'my',
    icon: $r("app.media.ic_tab_btn_mine_nor")
  }]
  @Builder
  getTabBar(item: TabClass) {
    Column() {
      Image(item.icon).width(22).height(22)
        .fillColor(item.name === this.tabsData[this.currentIndex].name ?
          $r('app.color.primary') : $r('app.color.text_secondary'))
      Text(item.title)
        .fontSize(12)
        .fontWeight(400)
        .margin({ top: 5 })
        .fontColor(item.name === this.tabsData[this.currentIndex].name ?
          $r('app.color.primary') : $r('app.color.text_secondary'))
    }.alignItems(HorizontalAlign.Center)
  }
  build() {
    Tabs({ barPosition: BarPosition.End, index: $$this.currentIndex }) {
      ForEach(this.tabsData, (item: TabClass) => {
        TabContent(){
          if(item.name === 'task') {
            Text("任务组件")
          }
          else if(item.name === 'message') {
            Text("消息组件")
          }
          else {
            Text("我的组件")
          }
        }.tabBar(this.getTabBar(item))
      })
    }
  }
}

```

## 我的组件基本布局

### Entry模块



- 页面也可以被当成组件来使用- 既可以被引用-也可以被跳转
- 组件不能当成页面来使用

:::info

注意:

我的是组件而不是页面，因为它属于Tabs中的子组件，所以我们在pages/Index 目录下新建一个My文件夹里面新建一个My.ets(组件可以是Page,也可以不是，但是如果是Page的话，需要进行导出)

:::

- 新建pages/Index/My/My.ets文件-结构从静态模板中拷贝

```
@Preview
@Component
struct My {
  build() {
    Column(){
      // 基本信息
      Column() {
        Image($r("app.media.ic_avatar_driver"))
          .width(67)
          .height(67)
          .borderRadius(34.5)
          .backgroundColor($r('app.color.white'))
        Text("司机")
          .fontSize(18)
          .fontWeight(600)
          .lineHeight(25)
          .margin({
            top: 9,
            bottom: 9
          })
          .fontColor($r('app.color.white'))
        Text(`司机编号:
66666666`) .fontSize(14).fontWeight(400).lineHeight(20).fontColor($r('app.color.white'))
        Text(`手机号: 66666666`)
          .fontSize(14)
          .fontWeight(400)
          .lineHeight(20)
          .margin({
            top: 10
          })
          .fontColor($r('app.color.white'))
      }
      .backgroundImage($r("app.media.bg_page_my"))
      .backgroundImageSize(ImageSize.Cover)
      .width('100%')
      .alignItems(HorizontalAlign.Center)
      .justifyContent(FlexAlign.Center)
      .height(292)
      .margin({
        top: -2
      })

      // 本月任务
```

```

Column() {
  Text("- 本月任务 -")
    .fontSize(14).fontColor($r('app.color.text_secondary')).lineHeight(20)
  Row() {
    Column() {

      Text("1000").fontSize(18).fontColor($r('app.color.text_primary')).lineHeight(25).margin({
        bottom: 17
      })
      Text("任务总量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
        .justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

      Column() {
        Text("1000")
          .fontSize(18)
          .fontColor($r('app.color.text_primary'))
          .lineHeight(25)
          .margin({ bottom: 17 })
        Text("完成任务量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
          .justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

        Column() {
          Text("1000")
            .fontSize(18)
            .fontColor($r('app.color.text_primary'))
            .lineHeight(25)
            .margin({ bottom: 17 })
          Text("运输里程(km)").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
            .justifyContent(FlexAlign.SpaceAround).layoutWeight(1)
        }.justifyContent(FlexAlign.SpaceBetween).width('100%').layoutWeight(1)
      }
    }.backgroundColor($r('app.color.white'))
    .borderRadius(8)
    .margin({ left: 14.5, right: 14.5, top: -55, bottom: 15 })
    .height(134)
    .padding({ top: 13.5, bottom: 13.5 })
    .justifyContent(FlexAlign.SpaceBetween)

    // 信息列表
    Column() {
      Row() {
        Text("车辆信息").fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
          Image($r('app.media.ic_btn_more')).width(24).height(24)
      }
      .justifyContent(FlexAlign.SpaceBetween)
      .alignItems(VerticalAlign.Center)
      .border({ width: { bottom: 1 }, color: $r('app.color.background_page') })
      .width('100%')
    }
  }
}

```



```

        .height(60)

        Row() {
            Text("任务数
据").fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
            Image($r("app.media.ic_btn_more")).width(24).height(24)
        }
        .justifyContent(FlexAlign.SpaceBetween)
        .alignItems(VerticalAlign.Center)
        .border({ width: { bottom: 1 }, color: $r('app.color.background_page') })
        .width('100%')
        .height(60)

        Row() {
            Text("系统设
置").fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
            Image($r("app.media.ic_btn_more")).width(24).height(24)
        }
        .justifyContent(FlexAlign.SpaceBetween)
        .alignItems(VerticalAlign.Center)
        .width('100%')
        .height(60)
    }
    .padding({ left: 17.5, right: 17.5 })
    .backgroundColor($r('app.color.white'))
    .margin({ left: 14.5, right: 14.5, bottom: 15 })
    .borderRadius(8)

}.width('100%').height('100%').backgroundColor($r('app.color.background_page')).borderRadi
us(8)
}
}

export default My

```

这里用了默认导出 `export default My` 为什么不用按需导出啦，按需导出针对的是我们的组件库及一些公用的方法和组件，因为不确定一个组件可能会导出哪些内容，可能一个可能多个，但是这里My属于业务组件，只用于完成这个业务，所以这里用了默认导出

- 放置在Index组件中的**我的位置**

```

ForEach(this.tabsData, (item: TabClass) => {
  TabContent(){
    if(item.name === 'task') {
      Text("任务组件")
    }
    else if(item.name === 'message') {
      Text("消息组件")
    }
    else {
      My()
    }
  }.tabBar(this.getTabBar(item))
})

```

车辆信息 >

任务数据 >

系统设置 >

好像长得一样

:::info

- 大家拷贝过程中发现了什么，就是我们车辆信息，任务数据，系统设置好像结构基本都一样，但是这样代码可读性确实有点低，怎么办？
  - 封装组件库是一个好的办法
  - 大家先把代码提交，接下来，我们来抽提两个组件 HmCard和HmCardItem
- ...

## 抽提HmCard和HmCardItem组件

Basic模块

:::info

- 抽提组件的原因是为了复用，小时达中有很多类似的需求设计如下图

9:41

<

## 回车登记

出车时间 2022.05.04 13:00

---

回车时间 请选择 >

原定时间 2022.05.04 13:00

延迟时间 不能超过2个小时 >

请输入延迟提货原因

0/50

提交

车辆信息 >

任务数据 >

系统设置 >

综上，我们需要一个卡片的容器，和一个左右显示内容的组件，并且容器中还可以放置其他组件

- 所以我们首先封装一个卡片HmCard- components/HmCard.ets

```
@Component
struct HmCard {
    @BuilderParam
```

```

HmCardFn: () => void

build() {
  Column() {
    Column() {
      if(this.HmCardFn) {
        this.HmCardFn()
      }
    }.backgroundColor($r('app.color.white')).borderRadius(10).width('100%').padding({
      left: 15,
      right: 15
    }).backgroundColor($r('app.color.white'))
  }.width('100%').padding(15)
}
}

export { HmCard }

```

- 在components/index.ets中导出

```
export * from './HmCard'
```

:::info

- HmCard组件中，我们定义了一个BuilderParam的函数（插槽），通过传入的方式传入内容  
...
- 接下来我们封装HmCardItem.ets组件，在HmCard同级新建HmCardItem组件

需求

:::info

1. 能够两头对齐
  2. 右侧能够显示向右的图标，也可以不显示
  3. 右侧还可以显示内容-并且应该是可变得
  4. 右侧的内容可以触发点击事件，并且在使用它时可以监听到
  5. 可以控制是否显示下边框
- ...

HmCardItem.ets代码实现

```

@Component
struct HmCardItem {
  onRightClick: () => void = () => {}
  leftTitle: string = ''
  showBottomBorder: boolean = true
  showRightIcon: boolean = true
  @Prop rightText: string

  build() {
    Row() {

```

```

Text(this.leftTitle).fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
    Row() {
        if (this.rightText) {

Text(this.rightText).fontColor($r("app.color.text_secondary")).fontWeight(400).fontSize(14)
        }
        if (this.showRightIcon) {
            Image($r("app.media.ic_btn_more")).width(24).height(24)
        }
    }.onClick(() => {
        this.onRightClick()
    })
}
.justifyContent(FlexAlign.SpaceBetween)
.alignItems(VerticalAlign.Center)
.border({
    width: {
        bottom: this.showBottomBorder ? 1 : 0
    },
    color: $r('app.color.background_divider')
})
.width('100%')
.height(60)
}
}

export { HmCardItem }

```

- 在components/index.ets导出

```

export * from './HmCard'
export * from './HmCardItem'

```

在basic/Index.ets中导出

```

export * from './src/main/ets/components'

```

## Entry模块

- 引入组件

```

import { HmCard, HmCardItem } from '@hm/basic'

```

- 接下来使用HmCard和HmCardItem去替换我的页面的三个内容吧

```
HmCard() {  
  HmCardItem({ leftTitle: '车辆信息', rightText: '' })  
  HmCardItem({ leftTitle: '任务设置', rightText: '' })  
  HmCardItem({ leftTitle: '系统设置', rightText: '', showBottomBorder: false })  
}
```

一模一样的，Nice！



:::info

总结

- 封装了HmCard和HmCardItem组件，利用插槽技术把CardItem放入Card中
- CardItem设置了几个属性 如 显示图标 右侧文本(Prop) 显示底边框
- 提交代码

...

## 获取我的个人信息

之前的遗留问题

```
}
const token = await setting.getUserToken()
// AppStorage如果当前key里面不存在 调用set的话 是设置不进去的
AppStorage.setOrCreate(TOKEN_KEY, token) | 写到全局内存中
if(token) {
  // token
  windowStage.loadContent('pages/Index/Index'); // 正常加载页面
}else {
  windowStage.loadContent('pages/Login/Login'); // 正常加载页面
}
}
```

在总入口的位置创建一个key 后面就不需要创建了

---info

标准流程

- 定义接口数据结构
- 封装api
- 组件中定义响应式数据
- 封装方法调用api获取数据赋值给响应式数据
- 在aboutToAppear中调用方法
- 将响应式数据更换到视图上换成真是的
- ...

1. 找到[获取个人信息](#)的接口文档



成功(200)

HTTP 状态码: 200      内容格式: JSON

| 数据结构                |                         |                          |    | </> 生成代码 | 示例                                                                             |
|---------------------|-------------------------|--------------------------|----|----------|--------------------------------------------------------------------------------|
| <code>code</code>   | <code>integer</code>    | 状态编码: 200-成功, 非200 -> 失败 | 必需 |          | {<br>"code"<br>"data"<br>"ava"<br>"nam"<br>"num"<br>"pho"<br>},<br>"msg":<br>} |
| ▼ <code>data</code> | <code>object {4}</code> | 响应数据                     | 必需 |          |                                                                                |
| <code>avatar</code> | <code>string</code>     | 头像                       | 必需 |          |                                                                                |
| <code>name</code>   | <code>string</code>     | 姓名                       | 必需 |          |                                                                                |
| <code>number</code> | <code>string</code>     | 司机编号                     | 必需 |          |                                                                                |
| <code>phone</code>  | <code>string</code>     | 手机号                      | 必需 |          |                                                                                |
| <code>msg</code>    | <code>string</code>     | 提示信息                     | 必需 |          |                                                                                |

修改时间 6 个月前

- 定义个人信息接口

```
export interface UserInfo {  
  /**  
   * 头像  
   */  
  avatar: string;  
  /**  
   * 姓名  
   */  
  name: string;  
  /**  
   * 司机编号  
   */  
  number: string;  
  /**  
   * 手机号  
   */  
  phone: string;  
}  
  
export class UserInfoModel implements UserInfo {  
  avatar: string = ''  
  name: string = ''  
  number: string = ''  
  phone: string = ''  
  
  constructor(model: UserInfo) {
```

```

    this.avatar = model.avatar
    this.name = model.name
    this.number = model.number
    this.phone = model.phone
  }
}

```

#### 4. 在api/user.ts中封装获取用户信息的接口

```

import { UserInfoModel } from '../models'

// 获取用户信息
export const getUserInfo = () => {
  return Request.get<UserInfoModel>("/users")
}

```

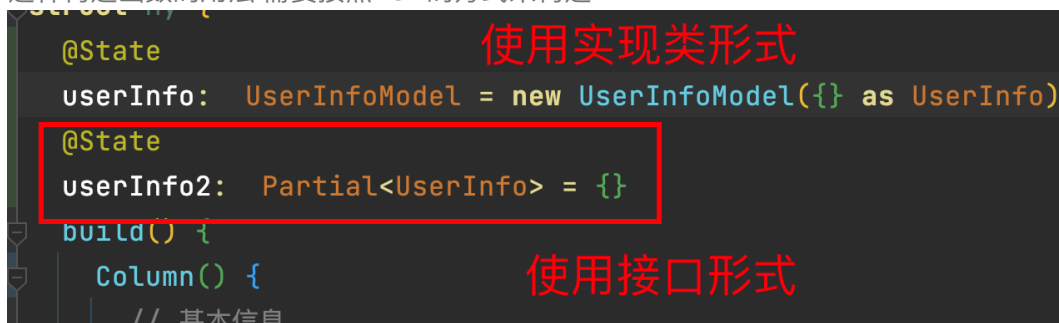
#### 5. 在my页面中导入类型并声明State数据

```

import { UserInfo, UserInfoModel } from '../../models'
struct My {
  @State
  userInfo: UserInfoModel = new UserInfoModel({} as UserInfo)
  ...
}

```

这里我们用了一个这样的用法 `new UserInfoModel({} as UserInfo)` 其实和我们之前使用 `Partial = {}` 的意思是一样的，但是由于这里是class，class附带着constructor构造函数，所以如果想要使用这种构造函数的用法 需要按照new的方式来构造



```

// 使用实现类形式
@State
userInfo: UserInfoModel = new UserInfoModel({} as UserInfo)

// 使用接口形式
@State
userInfo2: Partial<UserInfo> = {}

```

#### 6. 封装一个方法

```

async getUserInfo() {
  this.userInfo = await getUserInfo()
}

```

#### 7. 在父组件通过Provide传入一个当前name

```
@State
@Watch("updateName")
currentIndex: number = 0 // 当前激活项
@Provide
currentName: string = ""
updateName () {
  this.currentName = this.tabsData[this.currentIndex].name
}
```

8. 在子组件监听数据变化，如果是当前的tab则获取个人信息

```
@Consume
@Watch("getUserInfo")
currentName: string
async getUserInfo() {
  if(this.currentName === 'my') {
    this.userInfo = await getUserInfo()
  }
}
```

9. 更新数据

```

Column() {
    Image(this.userInfo.avatar || $r("app.media.ic_avatar_driver"))
        .width(67)
        .height(67)
        .borderRadius(34.5)
        .backgroundColor($r(#FFFFFF))
    Text(this.userInfo.name)
        .fontSize(18)
        .fontWeight(600)
        .lineHeight(25)
        .margin({
            top: 9,
            bottom: 9
        })
        .fontColor($r(#FFFFFF))
    Text(`司机编号: ${this.userInfo.number}`)
        .fontSize(14)
        .fontWeight(400)
        .lineHeight(20)
        .margin({
            top: 10
        })
        .fontColor($r(#FFFFFF))
}

```

...info

遵循我们一开始的标准流程后续所有的请求都按照此方式来进行

- 提交代码

...

## 任务数据获取

上一节如果你能够顺利完成的话，那么接下来，我们来做一个带参数的查询

### 获取任务数据接口地址

1. 导入数据类型- entry/models/user\_task.ets

导入的数据包括

- 请求参数UserTaskInfoParams
- 响应数据UserTaskInfo

```

/** 响应数据 */
export interface UserTaskInfo {

```

```

    /** 完成任务数量,基于实际完成时间统计 */
    completedAmounts: number;
    /** 每日里程,基于实际完成时间统计 */
    dailyMileage: DailyMileage[];
    /** 任务数量,基于计划完成时间统计 */
    taskAmounts: number;
    /** 运输里程, 单位: 公里, 基于实际完成时间统计 */
    transportMileage: number;
}

export interface DailyMileage {
    /** 日期,格式: 2022-07-16 00:00:00 */
    dateTime: string | null;
    /** 里程, 单位: 公里;计算公式: 原始数据 (单位米) /1000 四舍五入取整 */
    mileage: number | null;
}

export interface UserTaskInfoParams {
    /** 月 */
    month: string;
    /** 年 */
    year: string;
}

export class UserTaskInfoModel implements UserTaskInfo {
    completedAmounts: number = 0
    dailyMileage: DailyMileage[] = []
    taskAmounts: number = 0
    transportMileage: number = 0

    constructor(model: UserTaskInfo) {
        this.completedAmounts = model.completedAmounts
        this.dailyMileage = model.dailyMileage
        this.taskAmounts = model.taskAmounts
        this.transportMileage = model.transportMileage
    }
}

export class DailyMileageModel implements DailyMileage {
    dateTime: string | null = null
    mileage: number | null = null

    constructor(model: DailyMileage) {
        this.dateTime = model.dateTime
        this.mileage = model.mileage
    }
}

export class UserTaskInfoParamsModel implements UserTaskInfoParams {
    month: string = ''
    year: string = ''

    constructor(model: UserTaskInfoParams) {
        this.month = model.month
        this.year = model.year
    }
}

```

```
}  
}
```

使用i2c插件自动生成实现类class-具体方式参照上一小节

## 2. 封装api-user.ets

```
import { UserTaskInfoModel, UserTaskInfoParamsModel } from '../models'  
  
// 获取用户任务数据  
export const getUserTaskInfo = (data: UserTaskInfoParamsModel) => {  
    return Request.get<UserTaskInfoModel>("/users/taskReport", data)  
}
```

## 3. 在my组件中定义数据，封装方法调用

```
@State  
userTaskInfo: UserTaskInfoModel = new UserTaskInfoModel({} as UserTaskInfo)  
queryTaskParams: UserTaskInfoParamsModel = new UserTaskInfoParamsModel({  
    year: new Date().getFullYear().toString(),  
    month: (new Date().getMonth() + 1).toString()  
})  
async getUserInfo() {  
    if(this.currentName === 'my') {  
        this.userInfo = await getUserInfo()  
        this.userTaskInfo = await getUserTaskInfo(this.queryTaskParams)  
    }  
}  
async getUserTaskInfo() {  
    this.userTaskInfo = await getUserTaskInfo(this.queryTaskParams)  
}
```

```
{"taskAmounts":0,"completedAmounts":0,  
  "transportMileage":"0","dailyMileage":[]}
```

:::info

注意：由于接口的问题，任务数据中的年和月是必填的，而且必须是字符串，否则会报错，请注意

:::

## 4. 最后一步替换数据

```

Column() {
  Text("- 本月任务 -").fontSize(14).fontColor($r(#818181)).lineHeight(20)
  Row() {
    Column() {
      Text(this.userTaskInfo.taskAmounts?.toString() || '').fontSize(18).fontColor($r(#2A2929)).lineHeight(25).margin({ bottom: 17 })
    }
    Text("任务总量").fontSize(12).fontColor($r(#2A2929)).lineHeight(17)
  }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

  Column() {
    Text(this.userTaskInfo.completedAmounts?.toString() || '').fontSize(18).fontColor($r(#2A2929)).lineHeight(25).margin({ bottom: 17 })
    Text("完成任务量").fontSize(12).fontColor($r(#2A2929)).lineHeight(17)
  }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

  Column() {
    Text(this.userTaskInfo.transportMileage?.toString() || '').fontSize(18).fontColor($r(#2A2929)).lineHeight(25).margin({ bottom: 17 })
    Text("运输里程(km)").fontSize(12).fontColor($r(#2A2929)).lineHeight(17)
  }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)
}.justifyContent(FlexAlign.SpaceBetween).width(100%).layoutWeight(1)

```

```

// 本月任务
Column() {
  Text("- 本月任务 -")
    .fontSize(14).fontColor($r('app.color.text_secondary')).lineHeight(20)
  Row() {
    Column() {
      Text(this.userTaskInfo.taskAmounts?.toString() || '')
        .fontSize(18).fontColor($r('app.color.text_primary')).lineHeight(25).margin({
          bottom: 17
        })
    }
    Text("任务总量")
      .fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
  }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

  Column() {
    Text(this.userTaskInfo.completedAmounts?.toString() || '')
      .fontSize(18)
      .fontColor($r('app.color.text_primary'))
      .lineHeight(25)
      .margin({ bottom: 17 })
    Text("完成任务量")
      .fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
  }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

  Column() {
    Text(this.userTaskInfo.transportMileage?.toString() || '')

```

```

        .fontSize(18)
        .fontColor($r('app.color.text_primary'))
        .lineHeight(25)
        .margin({ bottom: 17 })
        Text("运输里程
(km)").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
        }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)
        }.justifyContent(FlexAlign.SpaceBetween).width('100%').layoutWeight(1)
    }
    .backgroundColor($r('app.color.white'))
    .borderRadius(8)
    .margin({ left: 14.5, right: 14.5, top: -55, bottom: 15 })
    .height(134)
    .padding({ top: 13.5, bottom: 13.5 })
    .justifyContent(FlexAlign.SpaceBetween)

```

分两块

- 用户信息不用每次都获取
- 用户任务数据-其实在变化，需要每次进入就查询

解释：

接口返回的是number类型，而只有字符串才能显示到文本上，我们使用了toString()，只不过你需要this.TaskInfo.transportMileage?.toString() 并且加上短路表达式来 如果前者为空 则处理成空字符 why?

因为TaskInfo我们定义的是空对象，直接对undefind的数据进行toString，会直接空指针异常的

- 最后提交代码

:::info

- 提交代码

...

## 系统设置页面





## 系统设置

换绑手机



修改密码



消息通知设置



修改密码



清理缓存

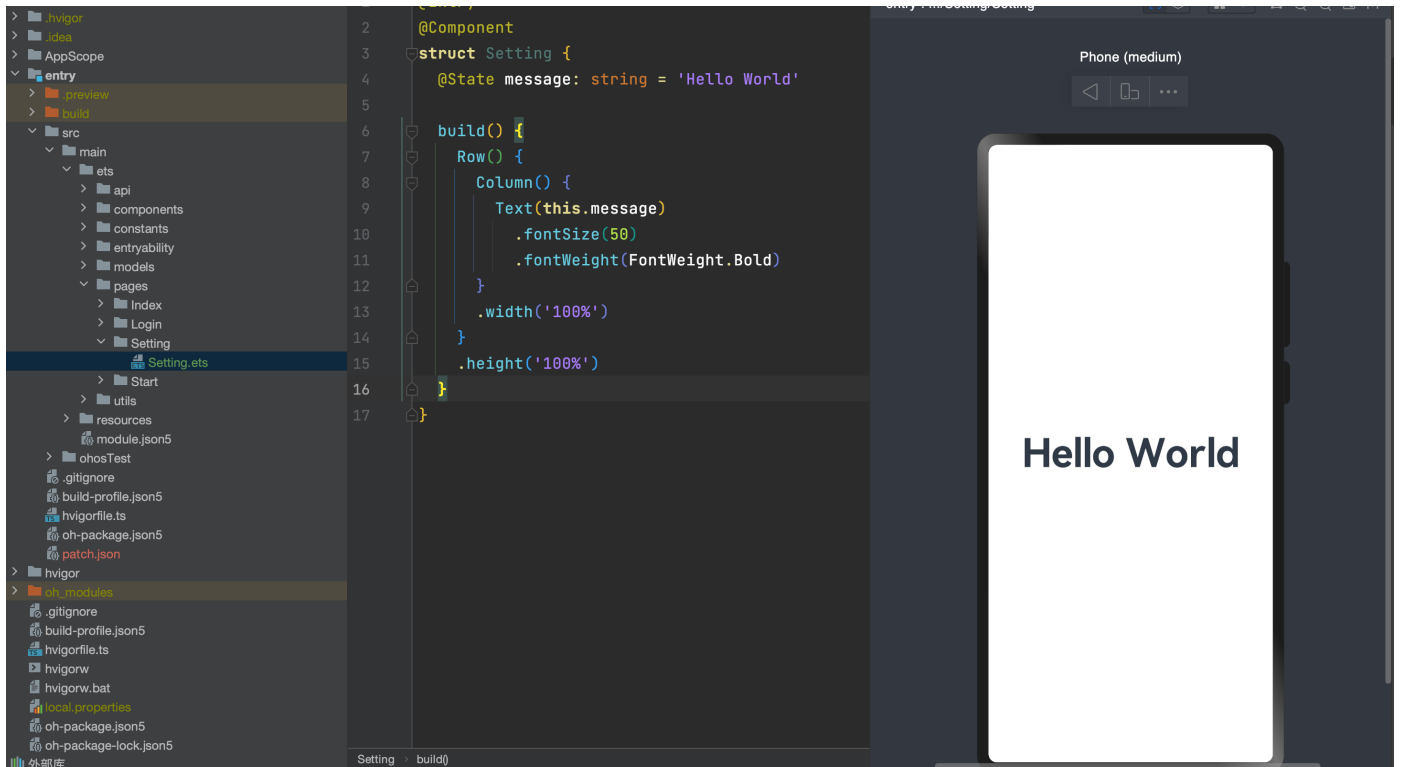


退出

上面这个页面我们完全可以复用之前的HmCard和HmCardItem，但是顶部的内容还带标题比较通用，所以我们来封装一个新的组件

1. 新建一个Page，因为车辆设置是点击系统设置跳转的页面，所以它是一个页面，不是一个组件

- 在pages下新建Setting/Setting.ets组件



- 我们先给主页布局，使用Column纵向布局，设置默认背景色

```
@Entry
@Component
struct Setting {

    build() {
        Column() {

        }.width('100%')
        .height('100%').backgroundColor($r('app.color.background_page'))
    }
}
```

- 使用之前封装好的组件HmCard HmCardItem 构建主内容区，并放置退出按钮

换绑手机



修改密码



消息通知设置



修改密码



清理缓存



退出

```
import { HmCard, HmCardItem } from '@hm/basic'

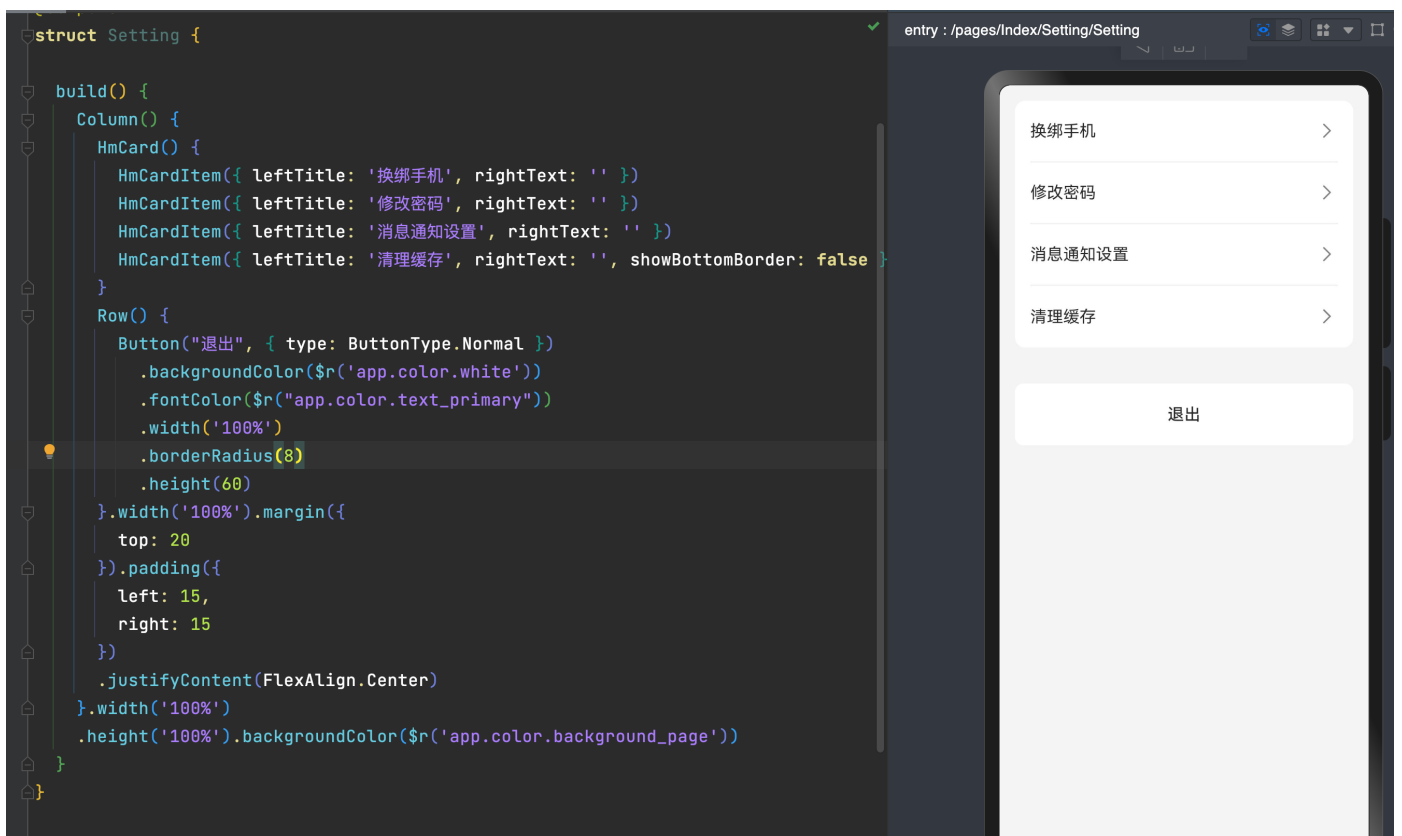
@Entry
@Component
struct Setting {

  build() {
    Column() {
      HmCard() {
        HmCardItem({ leftTitle: '换绑手机', rightText: '' })
      }
    }
  }
}
```

```

        HmCardItem({ leftTitle: '修改密码', rightText: '' })
        HmCardItem({ leftTitle: '消息通知设置', rightText: '' })
        HmCardItem({ leftTitle: '清理缓存', rightText: '', showBottomBorder: false })
    }
    Row() {
        Button("退出", { type: ButtonType.Normal })
            .backgroundColor($r('app.color.white'))
            .fontColor($r("app.color.text_primary"))
            .width('100%')
            .borderRadius(8)
            .height(60)
    }.width('100%').margin({
        top: 20
    }).padding({
        left: 15,
        right: 15
    })
        .justifyContent(FlexAlign.Center)
    }.width('100%')
    .height('100%').backgroundColor($r('app.color.background_page'))
}
}

```



4. 在点击系统设置时，跳转到系统设置页-pages/Index/My/My.ets

```
HmCardItem({ leftTitle: '系统设置', rightText: '', onRightClick: () => {
  router.pushUrl({
    url: 'pages/Setting/Setting'
  })
})
})
```

```
// 信息列表
HmCard() {
  HmCardItem({ leftTitle: '车辆信息', rightText: '' })
  HmCardItem({ leftTitle: '任务设置', rightText: '' })
  HmCardItem({ leftTitle: '系统设置', rightText: '', onRightClick: () => {
    router.pushUrl({
      url: 'pages/Setting/Setting'
    })
  })
}
```

:::info

总结:

- 封装好的组件用起来很爽
- 按照业务的需求和UI的设计抽提组件时要考虑复用性，这样就会积累起自己的一套组件库，甚至为开源做贡献
- 提交代码
- ...

## 系统设置页面-封装HmNavBar

basic包



系统设置

:::info

分析需求

- 能够设置中间的标题部分

- 点击左侧按钮能够返回上一层页面

...



## 1. 新建components/HmNavBar组件

```
import router from '@ohos.router';
@Preview
@Component
struct HmNavBar {
  title: string = "测试个人中心"
  showBackIcon: boolean = true

  build() {
    Stack({ alignContent: Alignment.TopStart }) {
      Row() {
        if (this.title) {
          Text(this.title).fontColor($r('app.color.text_primary')).fontSize(18).fontWeight(600)
        }
      }.justifyContent(FlexAlign.Center).alignItems(VerticalAlign.Center).width('100%').height('100%')

      if (this.showBackIcon) {
        Row() {
          Image($r("app.media.ic_btn_nav_back")).width(44).height(44).onClick(() => {
            router.back() // 回上一页
          })
        }.alignItems(VerticalAlign.Center).width(44)
      }
    }.backgroundColor($r('app.color.white')).height(50).width('100%').padding(10)
  }
}

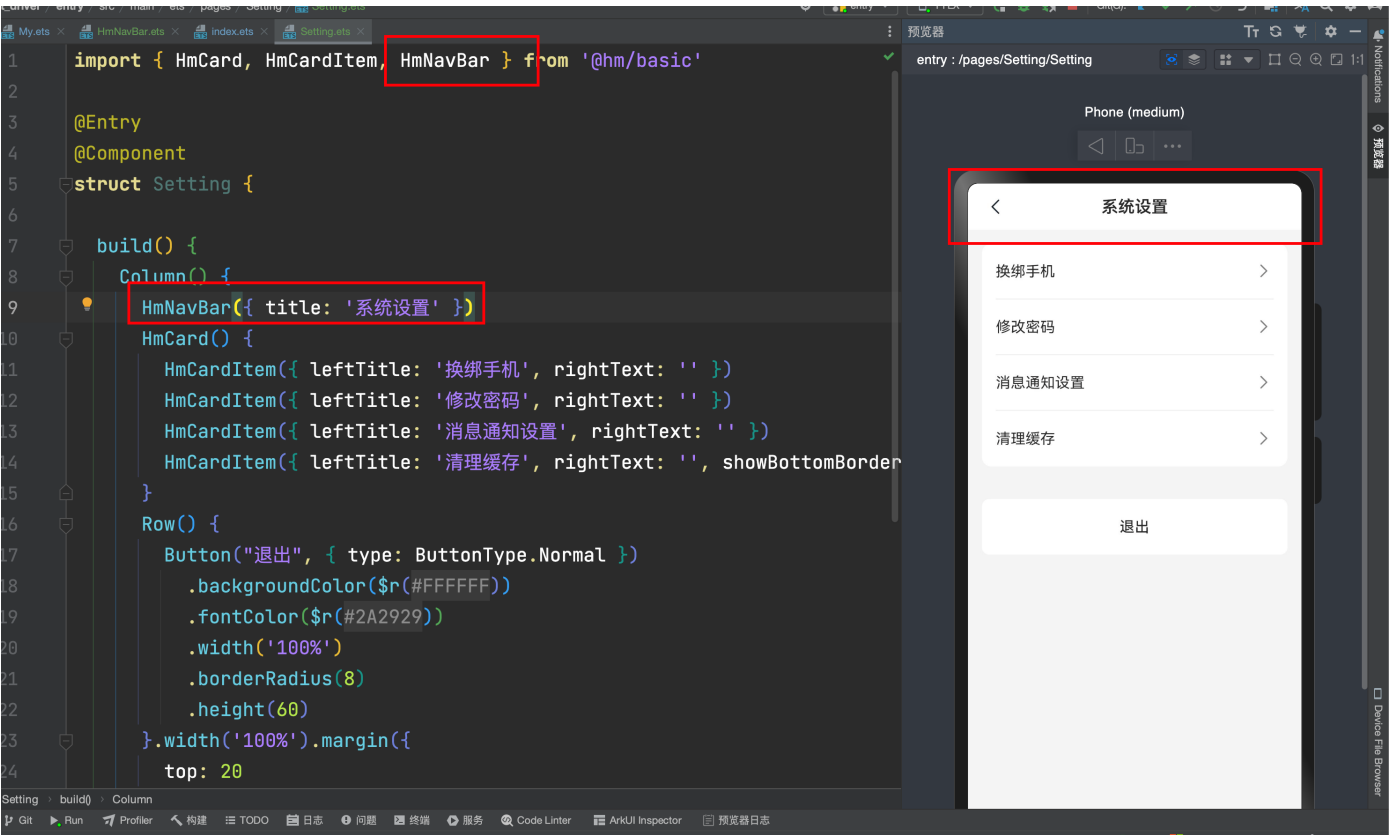
export { HmNavBar }
```

## 2. 在components/index.ets中导出

```
export * from './HmNavBar'
```

3. 直接在系统设置中引用，放置到最上方即可

```
HmNavBar({ title: '系统设置' })
```



:::info

总结

- 这里我们的HmNavBar的组件title和showBackIcon属性均没有采用@Link和@Prop，因为它并不需要向响应式更新，这里请知晓，需要的话再去封装对应的需求即可
  - 提交代码
- ...

# 自定义弹窗-实现退出登录功能

## Basic



...danger

按道理说-这是个很简单的需求，但是！！！目前鸿蒙提供的默认弹窗有点不忍直视，它是长这个样子的



- 差别有点大，为了避免以后我们和UI或者产品开撕的风险，我们还是自己来吧，系统默认的弹窗不好看，不符合要求，我们自己整一个呗，怎么整呢？我们需要使用鸿蒙提供的[自定义弹窗功能](#)  
...
- 现在来浅析一下自定义弹窗的使用方式

1. 使用@CustomDialog装饰器装饰自定义弹窗。
2. @CustomDialog装饰器用于装饰自定义弹框，此装饰器内进行自定义内容（也就是弹框内容）。



3. 创建构造器，与装饰器呼应相连。
4. 点击与onClick事件绑定的组件使弹窗弹出

- 封装一个自定义弹窗组件

```
@Preview
@CustomDialog
@Component
struct HmConfirm {
    // 必不可少的属性 用来控制这个结构的显示和隐藏的
    controller: CustomDialogController
    message: string = "确定要退出登录吗? "
    buttonList: HmConfirmButton[] = [{
        text: '确定',
        fontSize: 14,
        fontColor: $r('app.color.text_secondary')
    }]
    build() {
        Row() {
            Column() {
                Row() {
                    Text(this.message)
                        .fontSize(16)
                        .fontColor($r('app.color.text_primary'))
                }
                .width('100%')
                .height(88)
                .justifyContent(FlexAlign.Center)
                .border({
                    color: $r('app.color.background_divider'),
                    width: {
                        bottom: 0.5
                    }
                })
            })
            Row() {
                ForEach(this.buttonList, (item: HmConfirmButton, index: number) => {
                    Text(item.text)
                        .fontSize(item.fontSize || 16)
                        .textAlign(TextAlign.Center)
                        .fontColor(item.fontColor || $r('app.color.text_secondary'))
                        .layoutWeight(1)
                        .border({
                            color: $r('app.color.background_divider'),
                            width: {
                                right: this.buttonList.length > 1 && index != this.buttonList.length - 1
                                    ? 0.5: 0
                            }
                        })
                })
                .height('100%')
                .onClick(async () => {
                    if(item.action) {

```

```

        await item.action()
    }
    this.controller.close()
  })
})
}
.height(49)
.width('100%')
}
}
.width(278)
.borderRadius(12)
.backgroundColor($r('app.color.white'))
}
}

export { HmConfirm }

class HmConfirmButton {
  fontColor?: ResourceStr = ''
  fontSize?: number = 16
  text: string = ""
  action?: () => void = () => {}
}

```

- 统一导出

```
export * from './HmConfirm'
```

## Entry模块

- 在Setting中声明CustomDialogController

```

confirm: CustomDialogController = new CustomDialogController({
  builder: HmConfirm({
    message: '确定要退出登录吗?',
    buttonList: [{
      text: '取消'
    }, {
      text: '确定',
      fontColor: $r('app.color.primary'),
      action: () => {
        this.logout()
      }
    }]
  }),
  customStyle: true,
  alignment: DialogAlignment.Center,
  autoCancel: false
})

```

- 给退出登录按钮注册点击事件

```
.onClick(() => {  
    this.confirm.open()  
})
```

- 实现logout方法

```
logout () {  
    // 删除token  
    AppStorage.set<string>(TOKEN_KEY, "")  
    new UserSettingClass(getContext(this)).setUserToken("")  
    router.replaceUrl({  
        url: 'pages/Login/Login'  
    })  
}
```

确定要退出登录吗？

取消

确定

提交代码

## 封装骨架屏组件

basic模块



Placeholder text for the first item in the first group.

Placeholder text for the second item in the first group.

Placeholder text for the third item in the first group.

Placeholder text for the fourth item in the first group.



Placeholder text for the first item in the second group.

Placeholder text for the second item in the second group.

Placeholder text for the third item in the second group.

Placeholder text for the fourth item in the second group.



Placeholder text for the first item in the third group.

Placeholder text for the second item in the third group.

Placeholder text for the third item in the third group.

Placeholder text for the fourth item in the third group.



我们发现获取用户资料时速度较慢，此时页面会出现undefined的情况，我们可以判断下数据是否存在，如果不存在，我们直接显示一个骨架屏，如果数据出来，再把骨架屏撤

- 在basic中封装HmSkeleton组件

```
@Preview
@Component
struct HmSkeleton {
  @Prop
  showAvatar: boolean = true
  @Prop
  count: number = 3
  timer: number = -1
  @State
  currentColor: string = "#f3f4f5"

  aboutToAppear(): void {
    this.timer = setInterval(() => {
      if (this.currentColor === "#f3f4f5") {
        this.currentColor = "#f7f8f9"
      } else {
        this.currentColor = "#f3f4f5"
      }
    }, 100)
  }

  aboutToDisappear(): void {
    clearInterval(this.timer)
  }

  @Builder
  getSingleItem() {
    Column() {
      Row({ space: 10 }) {
        if (this.showAvatar) {
          Row()
```

```

        .width(40)
        .height(40)
        .borderRadius(20)
        .backgroundColor(this.currentColor)
    }
    Column({ space: 10 }) {
        Row()
            .width("30%")
            .height(26)
            .backgroundColor(this.currentColor)
        Row()
            .width("100%")
            .height(26)
            .backgroundColor(this.currentColor)
        Row()
            .width("100%")
            .height(26)
            .backgroundColor(this.currentColor)
        Row()
            .width("50%")
            .height(26)
            .backgroundColor(this.currentColor)
    }
    .layoutWeight(1)
    .alignItems(HorizontalAlign.Start)

}

.alignItems(VerticalAlign.Top)
.width('100%')
}
.width('100%')

}

build() {
    Column({ space: 30 }) {
        ForEach(Array.from(Array(this.count)), () => {
            this.getSingleItem()
        })
    }
    .alignItems(HorizontalAlign.Start)
    .padding(20)
    .width('100%')
    .height('100%')
    .backgroundColor($r("app.color.white"))
}
}

export default HmSkeleton

```

- 在components/index.ets中导出

```
export * from './HmSkeleton'
```

## entry模块

- 定义状态判断

```
@State
loading: boolean = true
async getUserInfo() {
  if(this.currentName === 'my') {
    this.loading = true
    this.userInfo = await getUserInfo()
    this.userTaskInfo = await getUserTaskInfo(this.queryTaskParams)
    this.loading = false
  }
}
```

- 在builde中判断

```
build() {
  Column(){
    if(this.loading) {
      HmSkeleton({ count: 1 })
    }else {
      // 基本信息
      Column() {...}
      .backgroundImage($r("app.media.bg_page_my"))
      .backgroundImageSize(ImageSize.Cover)
      .width('100%')
      .alignItems(HorizontalAlign.Center)
      .justifyContent(FlexAlign.Center)
      .height(292)
      .margin({
        top: -2
      })

      // 本月任务
    }
  }
}
```

提交代码

# 设计任务模块tabs

## entry模块

任务组件



Unexpected Text in JSON



任务



消息



我的

9:41



待提货

在途

已完成

任务编号: XAHH1234567

已延迟

起

北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00

提货

任务编号: XAHH1234568

**起** 北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

**止** 河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00-14:00

提货

任务编号: XAHH1234569

**起** 北京市昌平区回龙观街道西三旗桥东金燕龙写  
字楼8877号

**止** 河南省郑州市路北区北清路99号



任务



消息



我的

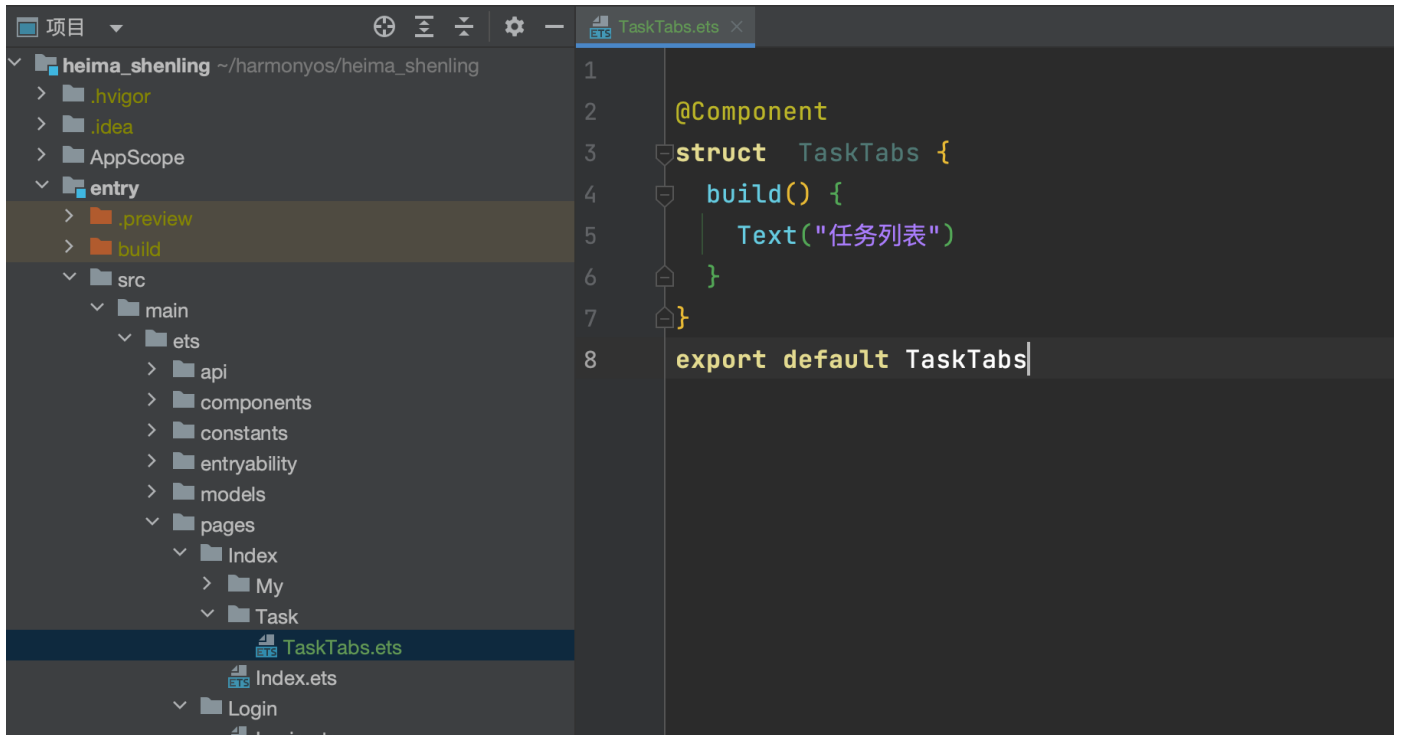
...info

任务模块分析

- 任务模块又是一个tabs，位置居于上方
- 里面三个待提货 在途 已完成
- 先要完成任务的基本布局，才可以往下推进

...

1. 新建一个TaskTabs组件-位置 pages/Index/Task/TaskTabs.ets(组件-非Page)



2. 在Index中引入该组件，并放入任务的tab内

```
import ...  
import TaskTabs from './Task/TaskTab'
```

```
@Entry
```

```
@Component
```

```
struct Index {
```

```
    @State currentName: string = 'task'
```

```
    build() {
```

```
        Tabs({ barPosition: BarPosition.End }){
```

```
            ForEach(this.tabsData, (item: TabClass) => {
```

```
                TabContent(){
```

```
                    if(item.name === 'task') {
```

```
                        TaskTabs()
```

```
                    }
```

```
                    else if(item.name === 'message') {
```

```
                        Text("消息组件")
```

```
                    }
```

```
                    else {
```

```
                        My()
```

```
                    }
```

```
                }.tabBar(this.getTabBar(item))
```

```
            })
```

```
        }
```

```
        .onChange(index => {
```

```
            this.currentName = this.tabsData[index].name
```

```
        })
```

```
        .animationDuration(300)
```

```
@Component
```

```
struct TaskTabs {
```

```
    build() {
```

```
        Text("任务列表")
```

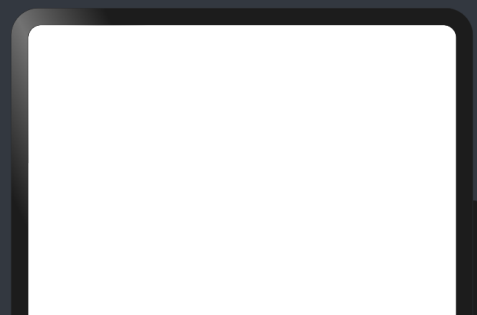
```
    }
```

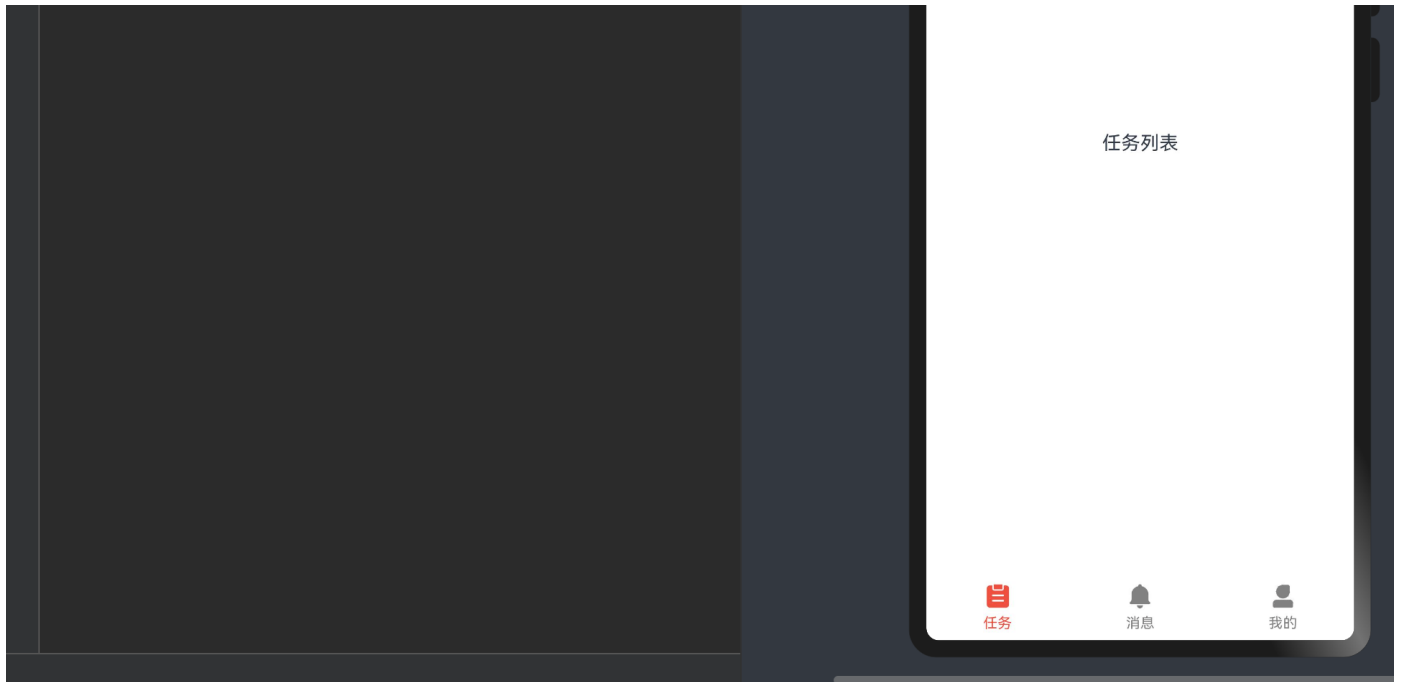
```
}
```

```
export default TaskTabs
```

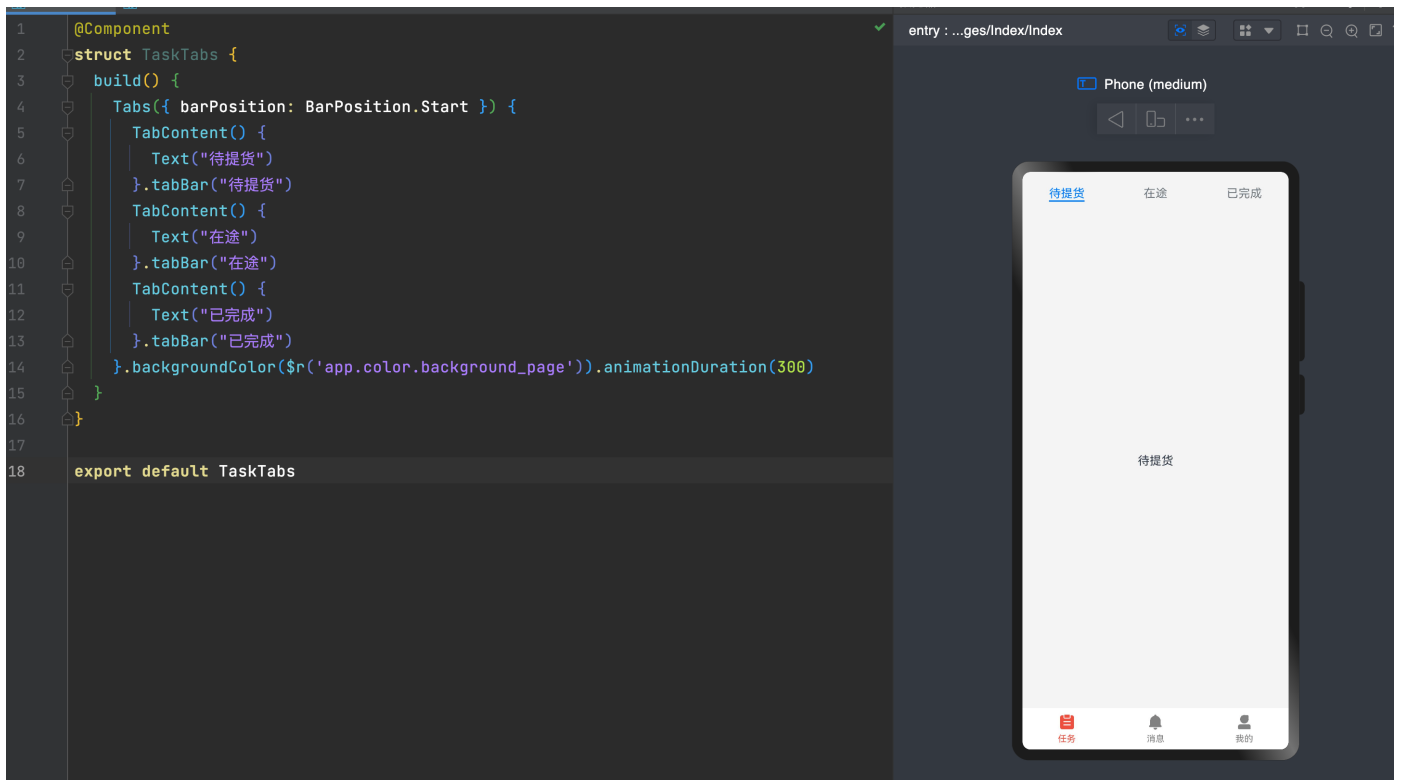
entry > images/index/index

Phone (medium)





### 3. 使用tabs组件构建任务组件的布局和内容



```
@Component
struct TaskTabs {
  build() {
    Tabs({ barPosition: BarPosition.Start }) {
      TabContent() {
        Text("待提货")
      }.tabBar("待提货")
      TabContent() {
        Text("在途")
      }.tabBar("在途")
      TabContent() {
        Text("已完成")
      }.tabBar("已完成")
    }.backgroundColor($r('app.color.background_page')).animationDuration(300)
  }
}

export default TaskTabs
```

```

    TabContent() {
      Text("已完成")
    }.tabBar("已完成")
  }.backgroundColor($r('app.color.background_page')).animationDuration(300)
}
}

export default TaskTabs

```

4. 似曾相识，和首页的Index不是一样吗，我们之前不是设置了一个TabClass吗，正好用上

- 定义state数据

```

@State
tabsData: TabClass[] = [{
  name: 'waiting',
  title: '待提货'
},{
  name: 'line',
  title: '在途'
},{
  name: 'finish',
  title: '已完成'
}]
@State
currentIndex: number = 0

```

- 完成循环渲染

```

build() {
  Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex }) {
    ForEach(this.tabsData, (item: TabClass) => {
      TabContent(){
        Text(item.title)
      }.tabBar(item.title)
    })
  }.backgroundColor($r('app.color.background_page')).animationDuration(300)
}

```

由于目前Tabs组件不支持设置上面页签的位置和方式，所以我们要采用自定义的方式来实现tabbar的设置

- 用Stack来包裹一下

```

build() {
  Stack({ alignContent: Alignment.Top }) {
    Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex }) {
      ForEach(this.tabsData, (item: TabClass) => {
        TabContent(){
          Text(item.title)
        }.tabBar(item.title)
      })
    }
  }
}

```

```

        }.backgroundColor($r('app.color.background_page')).animationDuration(300)
        Row ({ space: 30 }) {

        }
        .padding({
            left: 40,
            right: 40
        })
        .width('100%')
        .height(50)
        .backgroundColor($r("app.color.white"))

    }
}

```

- 自定义tabbar

```

@Builder
getTabBar(item: TabClass) {
    Column() {
        Text(item.title)
            .fontSize(16)
            .fontColor(this.tabsData[this.currentIndex].name === item.name ?
$r('app.color.text_primary') : $r('app.color.text_secondary'))
            .fontWeight(600)
            .animation({
                duration: 300
            })
            .margin({
                bottom: 10
            })
        Divider()
            .strokeWidth(4)
            .color($r('app.color.primary'))
            .lineCap(LineCapStyle.Round)
            .width(this.tabsData[this.currentIndex].name === item.name ? 23 : 0)
            .animation({
                duration: 300
            })
    }
}

```

- 定义tabController， 绑定Tabs组件

```

tabController: TabsController = new TabsController()

Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex, controller:
this.tabController }) {

```

- 点击时每个tab项时， 切换tab

```
.onClick(() => {  
    const index = this.tabsData.findIndex(i => i.name === item.name)  
    this.tabController.changeIndex(index)  
})
```

- 实际效果



完整代码

```
import { TabClass } from '@hm/basic'
```



```

@Preview
@Component
struct TaskTabs {
  tabController: TabsController = new TabsController()
  @State
  tabsData: TabClass[] = [{
    name: 'waiting',
    title: '待提货'
  }, {
    name: 'line',
    title: '在途'
  }, {
    name: 'finish',
    title: '已完成'
  }]
  @State
  currentIndex: number = 0

  @Builder
  getTabBar(item: TabClass) {
    Column() {
      Text(item.title)
        .fontSize(16)
        .fontColor(this.tabsData[this.currentIndex].name === item.name ?
$r('app.color.text_primary') : $r('app.color.text_secondary'))
        .fontWeight(600)
        .animation({
          duration: 300
        })
        .margin({
          bottom: 10
        })
      Divider()
        .strokeWidth(4)
        .color($r('app.color.primary'))
        .lineCap(LineCapStyle.Round)
        .width(this.tabsData[this.currentIndex].name === item.name ? 23 : 0)
        .animation({
          duration: 300
        })
    }
    .onClick(() => {
      const index = this.tabsData.findIndex(i => i.name === item.name)
      this.tabController.changeIndex(index)
    })
  }
  build() {
    Stack({ alignContent: Alignment.Top }) {
      Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex, controller:
this.tabController }) {
        ForEach(this.tabsData, (item: TabClass) => {
          TabContent(){

```

```

        Text(item.title)
    }.tabBar(item.title)
})
}.backgroundColor($r('app.color.background_page')).animationDuration(300)
Row ({ space: 30 }) {
    ForEach(this.tabsData, (item: TabClass) => {
        this.getTabBar(item)
    })
}
}.padding({
    left: 40,
    right: 40
})
}.width('100%')
}.height(50)
}.backgroundColor($r("app.color.white"))

}
}
}
export default TaskTabs

```

:::info

当无法调整tabbar位置的时候，换个思路，采用自定义的方式来实现

- 提交代码

...

## 待提货功能分析及数据加载

entry模块

待提货      在途      已完成

任务编号：XAHH1234567

已延迟

起

北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00

提货

任务编号：XAHH1234568

起

北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00-14:00

提货

任务编号：XAHH1234569

起

北京市昌平区回龙观街道西三旗桥东金燕龙写  
字楼8877号

止

河南省郑州市路北区北清路99号



任务



消息



我的

:::info  
分析

- 待提货是一个列表
- 每一项都是单独的一个任务

- 每一项的结构较为个性-而且在其他位置有可能用到-要考虑封装组件
- 有下拉刷新和上拉加载的需求- 难点

该如何下手

- 无外乎就是数据视图的展示
- 先把提货的组件建好,
- 把数据加载出来
- 再去实现后面的内容
- ...

1. 在pages/Index/Task下新建TaskList.ets组件

```
@Preview
@Component
struct TaskList {
    build() {
        Text("待提货列表")
    }
}
export default TaskList
```

2. 在TaskTabs中导入并使用

```
tasklabs.ets x tasklabs.ets x
import TaskList from './TaskList'

}

build() {
  Tabs({ barPosition: BarPosition.Start }) {
    ForEach(this.tabsData, (item: TabClass) => {
      TabContent(){
        if(item.name === 'waiting') {
          TaskList()
        }
      }.tabBar(this.getTabBar(item))
    })
  }
  .backgroundColor($r(#F4F4F4))
  .animationDuration(300)
  .onChange((index: number) => {
    this.currentTabName = this.tabsData[index].name
  })
  .barWidth(200)
}
}
```

### 3. 定义任务数据的类型-[接口文档](#)

新建一个models/task.ts文件，负责管理所有任务数据的类型

#### 步骤

- 拷贝请求参数类型和响应数据
- 这里将响应数据的Data改成TaskListData，将其Item[]改成TaskListItem名称-为了后续更好的辨别
- 使用i2c插件实现类的自动生成-参照前面的i2c用法

```
export interface TaskListParams {
  /** 结束时间 */
  endTime: string | null;
  /** 页码 */
  page: number;
  /** 页面大小 */
  pageSize: number;
  /** 开始时间 */
}
```

```

    startTime: string | null;
    /** 作业状态, 1为待提货)、2为在途(在途和已交付)、3为改派、5为已作废、6为已完成(已回车登记) */
    status: number;
    /** 运输任务id */
    transportTaskId: string | null;
}
/** 响应数据, 分页数据统一对象 */
export interface TaskListData {
    /** 总条目数 */
    counts: number;
    /** 数据列表 */
    items: TaskInfoItem[];
    /** 页码 */
    page: number;
    /** 总页数 */
    pages: number;
    /** 页尺寸 */
    pageSize: number;
}

export interface TaskInfoItem {
    /** 实际到达时间 */
    actualArrivalTime: string;
    /** 实际发车时间 */
    actualDepartureTime: string;
    /** 创建时间 */
    created: string;
    /** 司机id */
    driverId: string;
    /** 是否可提货 */
    enablePickUp: boolean;
    /** 目的机构地址 */
    endAddress: string;
    /** 目的机构id */
    endAgencyId: number;
    /** 交付对接人 */
    finishHandover: string;
    /** 司机作业单id */
    id: string;
    /** 计划到达时间 */
    planArrivalTime: string;
    /** 计划发车时间 */
    planDepartureTime: string;
    /** 起始机构地址 */
    startAddress: string;
    /** 起始机构id */
    startAgencyId: number;
    /** 提货对接人 */
    startHandover: string;
    /** 作业状态, 作业状态, 1为待提货)、2为在途)、3为改派)、4为已交付)、5为已作废 */
    status: string;
    /** 运输任务id */

```

```

    transportTaskId: string;
}
export class TaskListParamsModel implements TaskListParams {
    endTime: string | null = null
    page: number = 0
    pageSize: number = 0
    startTime: string | null = null
    status: number = 0
    transportTaskId: string | null = null

    constructor(model: TaskListParams) {
        this.endTime = model.endTime
        this.page = model.page
        this.pageSize = model.pageSize
        this.startTime = model.startTime
        this.status = model.status
        this.transportTaskId = model.transportTaskId
    }
}
export class TaskListDataModel implements TaskListData {
    counts: number = 0
    items: TaskInfoItem[] = []
    page: number = 0
    pages: number = 0
    pageSize: number = 0

    constructor(model: TaskListData) {
        this.counts = model.counts
        this.items = model.items
        this.page = model.page
        this.pages = model.pages
        this.pageSize = model.pageSize
    }
}
export class TaskInfoItemModel implements TaskInfoItem {
    actualArrivalTime: string = ''
    actualDepartureTime: string = ''
    created: string = ''
    driverId: string = ''
    enablePickUp: boolean = false
    endAddress: string = ''
    endAgencyId: number = 0
    finishHandover: string = ''
    id: string = ''
    planArrivalTime: string = ''
    planDepartureTime: string = ''
    startAddress: string = ''
    startAgencyId: number = 0
    startHandover: string = ''
    status: string = ''
    transportTaskId: string = ''
}

```

```

constructor(model: TaskInfoItem) {
    this.actualArrivalTime = model.actualArrivalTime
    this.actualDepartureTime = model.actualDepartureTime
    this.created = model.created
    this.driverId = model.driverId
    this.enablePickUp = model.enablePickUp
    this.endAddress = model.endAddress
    this.endAgencyId = model.endAgencyId
    this.finishHandover = model.finishHandover
    this.id = model.id
    this.planArrivalTime = model.planArrivalTime
    this.planDepartureTime = model.planDepartureTime
    this.startAddress = model.startAddress
    this.startAgencyId = model.startAgencyId
    this.startHandover = model.startHandover
    this.status = model.status
    this.transportTaskId = model.transportTaskId
}
}

```

- 声明status类型的枚举

请求类型采用的是一个枚举，需要使用Enum来声明

```

// 查询状态的枚举
export enum TaskTypeEnum {
    Waiting = 1,
    Line = 2,
    Finish = 6
}

```

- 在models/index.ets中导出

```
export * from './task'
```

- 封装请求列表api - 新建api/task.ets

```

import { TaskListData, TaskListParams } from '../models'
import { Request } from '@hm/basic'

// 获取任务列表

export const getTaskList = (params: TaskListParams) => {
    return Request.get<TaskListData>("/tasks/list", params)
}

```

- 在api/index.ets中导出



```
export * from './task'
```

- 在TaskList中声明数据，封装方法，获取数据，显示获取的数据

```
@State
queryParams: TaskListParams = new TaskListParamsModel({
  status: TaskTypeEnum.Waiting,
  page: 1,
  pageSize: 5,
}) as TaskListParams
@State
taskListData: TaskInfoItem[] = []
```

- 封装方法调用-显示

```
import { getTaskList } from '../../../api/task'
import { TaskInfoItem, TaskListParams, TaskListParamsModel, TaskTypeEnum } from
'../../../models'

@Preview
@Component
struct TaskList {
  @State
  queryParams: TaskListParams = new TaskListParamsModel({
    status: TaskTypeEnum.Waiting,
    page: 1,
    pageSize: 5,
  }) as TaskListParams
  @State
  taskListData: TaskInfoItem[] = []
  aboutToAppear() {
    this.getTaskList()
  }
  async getTaskList() {
    const result = await getTaskList(this.queryParams)
    this.taskListData = result.items
  }
  build() {
    Text(JSON.stringify(this.taskListData))
  }
}
export default TaskList
```

```

if (method === http.RequestMethod.GET) {
  // data参数都拼接到地址上 ?a=1&b=2&c=3
  if (data && Object.keys(data).length) {
    urlStr += "?" + Object.keys(data).filter(key => !!data[key]).map(key => {
      return `${key}=${data[key]}`
    }).join("&")
  }
}

```

过滤掉为空的参数 避免后端解析失败

如图



:::info  
总结

- 标准的流程
- 定义类型
- 封装api
- 调用接口
- 获取数据
- 显示数据-待实现
- 提交代码
- ...

## 使用列表循环渲染数据

- 首先需要有一个统一的卡片



- 静态结构-新建一个组件pages/Index/Task/TaskItemCard.ets

```
@Preview
@Component
struct TaskItemCard {
  build() {
    Column() {
      Row() {
```

```

Text(`任务编号: 213893928399283924`)
  .fontSize(16)
  .fontColor($r("app.color.text_primary"))
  .fontWeight(500)
  .lineHeight(22)
}.justifyContent(FlexAlign.SpaceBetween).width('100%')

Row() {
  Text("起")
    .fontSize(12)
    .fontColor($r("app.color.white"))
    .backgroundColor($r("app.color.text_primary"))
    .width(22)
    .height(22)
    .borderRadius(11)
    .textAlign(TextAlign.Center)
  Text("北京市昌平区回龙观街道西三旗桥东金燕龙写字楼8877号")
    .margin({ left: 11.5 })
    .fontColor($r('app.color.text_secondary'))
    .fontSize(14)
}.margin({ top: 21 }).width('100%')

Row() {
  Text("止")
    .fontSize(12)
    .fontColor($r("app.color.white"))
    .backgroundColor($r('app.color.primary'))
    .width(22)
    .height(22)
    .borderRadius(11)
    .textAlign(TextAlign.Center)
  Text("河南省郑州市路北区北清路99号")
    .margin({ left: 11.5 })
    .fontColor($r('app.color.text_secondary'))
    .fontSize(14)
}.margin({ top: 14.5 }).width('100%')

Divider()
  .vertical(true)
  .height(2)
  .color($r('app.color.background_divider'))
  .opacity(0.6)
  .margin({ left: 8, right: 8, top: 21 })
Row() {
  Column() {
    Text('提货时间').fontSize(14).fontColor($r('app.color.text_secondary'))
    Text("2022.05.04
13:00").fontSize(14).fontColor($r('app.color.text_secondary')).margin({ top: 4 })
  }.alignItems(HorizontalAlign.Start)

  Button("提货", { type: ButtonType.Capsule })
    .backgroundColor($r('app.color.primary'))

```

```

        .fontColor($r("app.color.white"))
        .fontSize(14)
        .height(32)

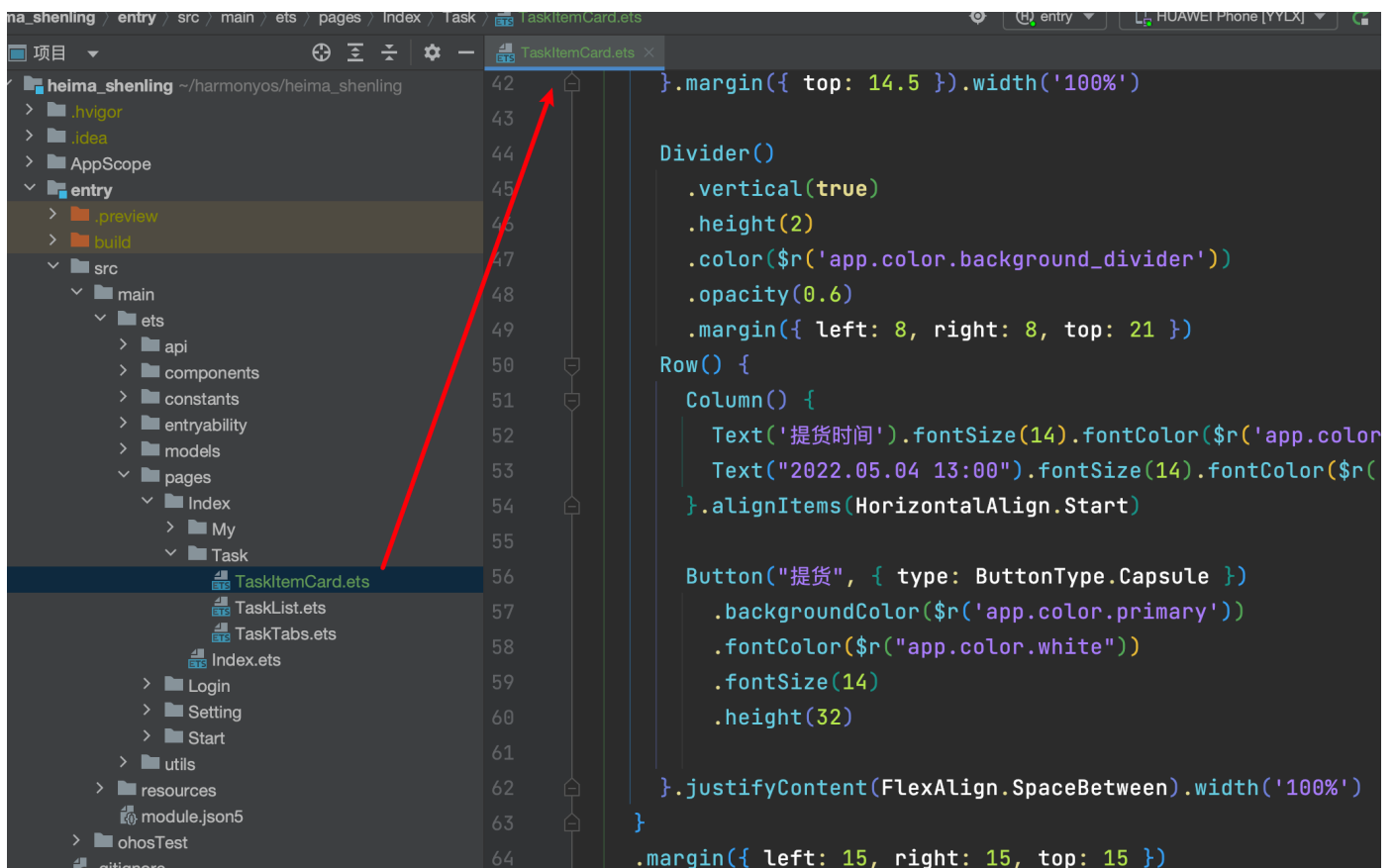
    }.justifyContent(FlexAlign.SpaceBetween).width('100%')
}
.margin({ left: 15, right: 15, top: 15 })
.padding({ left: 19.5, right: 19.5, bottom: 18.5, top: 18.5 })
.borderRadius(10)
.backgroundColor($r('app.color.white'))

}
}

export default TaskItemCard

```

- 封装TaskItemCard组件-位置-pages/Index/Task/TaskItemCard.ets



- 使用List和ListItem配合生成N个TaskItemCard



```
build() {  
  List () {  
    ForEach(this.taskListData, (item: TaskInfoItemModel) => {  
      ListItem(){  
        TaskItemCard()  
      }  
    })  
  }  
}
```

```

build() {
  List () {
    ForEach(this.taskListData, (item: TaskInfoItemModel) => {
      ListItem(){
        TaskItemCard()
      }
    })
  }
}
}

```

:::info

- 目前实现了基本的首页的显示
- 利用List和ListItem进行循环渲染-数据是假的-下一步换成真实的
- 提交代码
- ...

## 任务卡片数据显示

- 把任务卡片的数据变成真实的

之前在获取任务列表数据时，已经声明了TaskListItem的类型可以直接使用，可以用interface也可以用class

1. 在TaskItemCard中声明一个属性，接收单个任务的数据

- TaskItemCard.ets

```

import { TaskInfoItem, TaskInfoItemModel } from '../../models'

@Component
struct TaskItemCard {
  taskItemData: TaskInfoItemModel = new TaskInfoItemModel({} as TaskInfoItem)
  ...
}

```

- 替换静态数据

```

Text(`任务编号: ${this.taskItemData.transportTaskId}`)
    .fontSize(14)

Row() {
    Text("止")
        .fontSize(12)
        .fontColor($r(#FFFFFF))
        .backgroundColor($r(#EF4F3F))
        .width(22)
        .height(22)
        .borderRadius(11)
        .textAlign(TextAlign.Center)
    Text(this.taskItemData.endAddress)
        .margin({ left: 11.5 })
        .fontColor($r(#818181))
        .fontSize(14)
}.margin({ top: 14.5 }).width('100%')
.margin({ left: 8, right: 8, top: 21 })

Row() {
    Column() {
        Text('提货时间').fontSize(14).fontColor($r(#818181))
        Text(this.taskItemData.planDepartureTime).fontSize(14)
        .alignItems(HorizontalAlign.Start)

        Button("提货", { type: ButtonType.Capsule })
            .backgroundColor($r(#EF4F3F))
    }
}

```

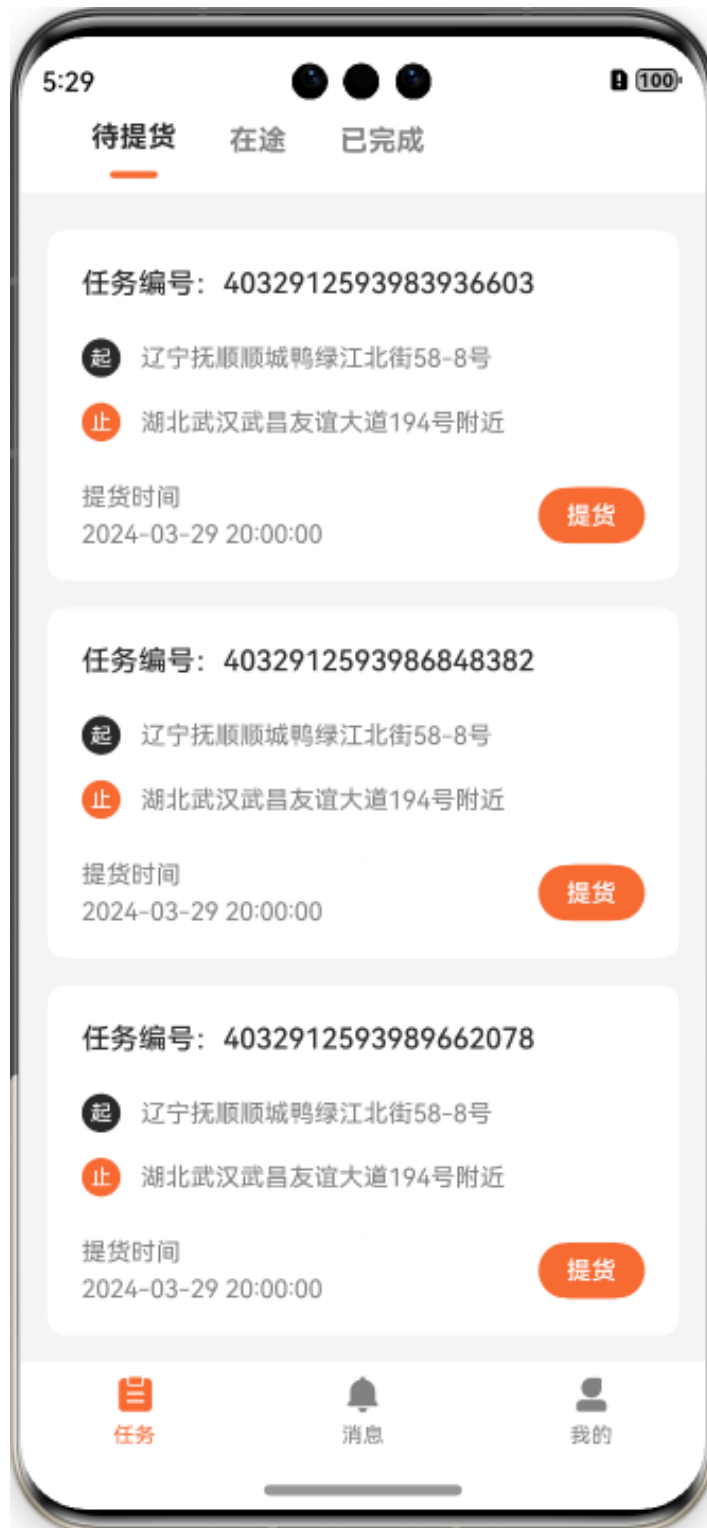
- 父组件TaskList传入item数据给TaskItemCard

```

build() {
    List () {
        ForEach(this.taskListData, (item: TaskInfoItemModel) => {
            ListItem(){
                TaskItemCard({ taskItemData: item })
            }
        })
    }
}

```





:::info

总结

子组件TaskItemCard中声明数据，接受数据，并没有使用修饰符，因为此数据是只读的，父级数据只会渲染，不会操作发生删除和修改

- 提交代码

...

## 根据状态控制提货按钮

任务编号: 3111716124860458266

**起** 辽宁抚顺顺城鸭绿江北街58-8号

**止** 湖北武汉武昌友谊大道194号附近

提货时间

2023-11-17 20:00:00

提货

任务编号: 3111716124861814870

**起** 辽宁抚顺顺城鸭绿江北街58-8号

**止** 湖北武汉武昌友谊大道194号附近

提货时间

2023-11-17 20:00:00

提货

:::info

分析

当前司机只能有一个任务，必须完成提货，交货，回车登记才可以继续下一个提货

:::

我们的接口中有一个字段可以帮助我们识别该任务用户可以提取

`driverId` string 司机id

`enablePickUp` boolean 是否可提货

`endAddress` string 目的机构地址

`endAgencyId` integer 目的机构id

`finishHandover` string 交付对接人

`id` string 司机作业单id

我们根据这个字段true/false来控制一下提货按钮的显示颜色

```
// 获取按钮是否可提货
getBtnEnable() {
  if (this.taskItemData.enablePickUp) {
    return true
  }
  return false
}

...

Button("提货", { type: ButtonType.Capsule })
  .backgroundColor(this.getBtnEnable() ? $r('app.color.primary') :
$r('app.color.primary_disabled'))
  .fontColor($r("app.color.white"))
  .fontSize(14)
  .height(32)
  .enabled(this.getBtnEnable())
```



...info

总结

根据状态控制按钮颜色

提交代码

...

## 封装抽提HmList组件实现上拉加载

basic模块

:::info

HmList组件要完成的功能

- 支持下拉刷新
- 支持上拉加载
- 支持自定义结构
- 支持传入数据渲染内容
- 支持设置加载提示文本
- 支持设置加载完成文本
- 支持控制是否显示加载的进度条
- ...

结构依然采用之前可以使用的Refresh包裹List的基本架构

- 只不过我们需要向外暴露更多的属性
- finished 是否加载结束（布尔值）
- dataSource 数据源（Link修饰符）
- renderItem 渲染单个Item的(BuilderParam)
- onLoad 执行上拉加载的逻辑(函数)
- onRefresh 执行下拉刷新的逻辑（函数）
- loadingText 加载时的文本（文本）
- finishText 结束时的文本（文本）
- showLoadingIcon 显示加载进度（布尔值）
- refreshIng 控制下拉刷新（State修饰 布尔值）
- loading 控制上拉加载（State布尔值）

在components下新建HmList.ets组件

```
import { promptAction } from '@kit.ArkUI'

@Component
struct HmList {
  @State
  refreshIng: boolean = false // 控制下拉刷新的变量
  // 传入数据的数组 根据数组进行渲染
  @Prop
  dataSource: object[] = [] // 数据源

  // 上拉加载的方法
  onLoad: () => void = () => {} // 由调用者传入

  // 下拉刷新方法
  onRefresh: () => void = () => {} // 下拉刷新的方法

  // 需要一个标记 还没有数据的标记
```

```

@Prop
finished: boolean = false // 是否还有下一页数据

@State
loading: boolean = false // 是否正在加载中 1.显示加载中文本 2. 用来做阀门 当前这次请求没结束之前
下次请求滚远点

loadingText: string = "加载中.." // 加载中的文本

finishText: string = "没有数据啦" // 所有数据加载完成的文本

@BuilderParam
renderItem: (item: object) => void // 由调用者传入 由HmList调用 传入每一个项的数据
@Builder
getBottomDisplay () {
    // 获取底部的展示内容
    Row({ space: 10 }) {
        if(this.finished) {
            // 此时应该没有动画的loading
            Text(this.finishText)
                .fontSize(14)
                .fontColor($r("app.color.text_secondary"))
        }else {
            Text(this.loadingText)
                .fontSize(14)
                .fontColor($r("app.color.text_secondary"))
            LoadingProgress()
                .width(20)
                .aspectRatio(1)
                .color($r("app.color.text_secondary"))
        }
    }
    .width('100%')
    .height(50)
    .justifyContent(FlexAlign.Center)
}
build() {
    Refresh({ refreshing: $$this.refreshing }) {
        List() {
            ForEach(this.dataSource, (item: object) => {
                // 每一项的结构的UI内容不是由列表决定 而由使用者决定
                // 传入builderParam
                if(this.renderItem) {
                    this.renderItem(item)
                }
            })
            // 最后放置提示文本的地方
            ListItem () {
                this.getBottomDisplay()
            }
        }
        .onReachEnd(async () => {

```

```

        // 实现上拉加载
        // 需要一个标记 是否已经加载完所有数据
        // 在没有加载完所有页数据的情况 且没有请求在进程中的情况下
        if(!this.finished && !this.loading) {
            this.loading = true // 关闭阀门
            await this.onLoad() // 实现上拉加载
            this.loading = false // 打开阀门
        }
    })

}

.onStateChange(async (state) => {
    // 实现下拉刷新
    if(state === RefreshStatus.Refresh) {
        // 松手加载
        await this.onRefresh() // 调用刷新方法
        this.refreshIng = false // 关闭下拉的动画效果
        this.loading = false // 关闭上拉刷新的loading
        // 下拉刷新意味着所有数据全都不要 重新来过

    }
})

}
}
export { HmList }

```

- 在components/index.ets中导出

```
export * from './HmList'
```

## Entry模块

- 在TaskList中应用组件， 实现加载下一页

```

import { HmList } from '@hm/basic/Index'
import { getTaskList } from '../../../api'
import { TaskInfoItem, TaskInfoItemModel, TaskListParams, TaskListParamsModel,
TaskTypeEnum } from '../../../models'
import TaskItemCard from './TaskItemCard'

// 待提货
@Component
struct TaskList {
    @State
    queryParams: TaskListParamsModel = new TaskListParamsModel(
    {
        status: TaskTypeEnum.Waiting, // 待提货的类型
        page: 1, // 第几页
        pageSize: 5 // 每页几条数据
    } as TaskListParams

```

```

)
@State
taskListData: TaskInfoItem[] = []
@State
allPage: number = 1 // 默认只有一页

async getTaskList() {
  const result = await getTaskList(this.queryParams)
  // 追加数据
  // this.taskListData = this.taskListData.concat(result.items) // 拿到返回的数组
  this.taskListData.push(...result.items) // 延展运算符的写法
  this.allPage = result.pages // 总页数
  this.queryParams.page++ // 下次请求的页码
}
@Builder
renderItem (item: object) {
  TaskItemCard({ taskItemData: item as TaskInfoItemModel })
}
build() {
  HmList({
    dataSource: this.taskListData, // 数据源
    finished: this.allPage < this.queryParams.page , // 是否还有下一页
    // 上拉加载的函数
    onLoad: async () => {
      // 上拉加载
      await this.getTaskList()
    },
    renderItem: (item: object) => {
      // 如果需要的是builderParams的参数 可以用普通函数包裹一个Builder的函数
      this.renderItem(item)
    },
    loadingText: '拼命加载中',
    finishText: '没啦没啦'
  })
}
}

export default TaskList

```

提交代码

## 实现下拉刷新

- 给HmList传入onRefresh函数



```
HmList({
  onLoad: async () => {
    await this.getTaskList(true)
  },
  onRefresh: async () => {
    await this.onRefresh()
  },
  dataSource: $taskListData,
  renderItem: this.renderItem,
  finished: this.allPage < this.queryParams.page,
  finishText: '没啦没啦',
  loadingText: '拼命加载中'
})
```

- 实现onRefresh函数

```
// 下拉刷新函数
async onRefresh() {
  // 重新请求第一页数据
  this.queryParams.page = 1 // 重置第一页
  await this.getTaskList(false) // 直接赋值
}
```

- 改造getTaskList方法

```
async getTaskList(append: boolean) {
  const result = await getTaskList(this.queryParams)
  if(append) {
    this.taskListData.push(...result.items)
  }else {
    this.taskListData = result.items
  }

  this.allPage = result.pages
  this.queryParams.page++
}
```

- 完整代码

```
import { HmList } from '@hm/basic/Index'
import { getTaskList } from '../../api'
import { TaskInfoItem, TaskInfoItemModel, TaskListParams, TaskListParamsModel,
TaskTypeEnum } from '../../models'
import TaskItemCard from './TaskItemCard'
import { promptAction } from '@kit.ArkUI'

// 待提货
@Component
struct TaskList {
  @State
```

```

queryParams: TaskListParamsModel = new TaskListParamsModel(
    {
        status: TaskTypeEnum.Waiting, // 待提货的类型
        page: 1, // 第几页
        pageSize: 5 // 每页几条数据
    } as TaskListParams
)
@State
taskListData: TaskInfoItem[] = []
@State
allPage: number = 1 // 默认只有一页

async getTaskList(append: boolean) {
    const result = await getTaskList(this.queryParams)
    // 追加数据
    // this.taskListData = this.taskListData.concat(result.items) // 拿到返回的数组
    if (append) {
        this.taskListData.push(...result.items) // 延展运算符的写法
    } else {
        this.taskListData = result.items // 直接赋值
    }

    this.allPage = result.pages // 总页数
    this.queryParams.page++ // 下次请求的页码
}

@Builder
renderItem(item: object) {
    TaskItemCard({ taskItemData: item as TaskInfoItemModel })
}

// 下拉刷新函数
async onRefresh() {
    // 重新请求第一页数据
    this.queryParams.page = 1 // 重置第一页
    await this.getTaskList(false) // 直接赋值
}

build() {
    HmList({
        dataSource: this.taskListData, // 数据源
        finished: this.allPage < this.queryParams.page, // 是否还有下一页
        // 上拉加载的函数
        onLoad: async () => {
            // 上拉加载
            await this.getTaskList(true) // 追加逻辑
        },
        onRefresh: async () => {
            // 下拉刷新
            await this.onRefresh()
        },
        renderItem: (item: object) => {

```

```
        // 如果需要的是builderParams的参数 可以用普通函数包裹一个Builder的函数
        this.renderItem(item)
    },
    loadingText: '拼命加载中',
    finishText: '没啦没啦'
  })
}
}

export default TaskList
```

:::info

总结

- 下拉刷新要保证能够发出请求，所以设置allPage为1，查询页码为1
  - 下拉刷新时覆盖数据，上拉加载是追加数据
- 提交代码

...

# 改造下拉刷新的样式

basic模块

|                        |               |   |                                                                                           |
|------------------------|---------------|---|-------------------------------------------------------------------------------------------|
| builder <sup>10+</sup> | CustomBuilder | 否 | 下拉时，自定义刷新样式的组件。<br>说明：<br>API version 10及之前版本，自定义组件的高度限制在64vp之内。API version 11及以后版本没有此限制。 |
|------------------------|---------------|---|-------------------------------------------------------------------------------------------|

Refresh组件支持自定义构建样式

- 声明变量记录当前下拉状态

```
@State
refreshStatus: RefreshStatus = RefreshStatus.Inactive
```

- 下拉状态发生变化时赋值状态

```

    .onStateChange(async value => {
        this.refreshStatus = value
        if (value === RefreshStatus.Refresh) {
            await this.onRefresh()
            this.refreshIng = false
            this.loading = false
        }
    })
}

```

- 实现Refresh的builder的函数

```

// 动态生成文本
getStatusText() {
    switch (this.refreshStatus) {
        case RefreshStatus.Inactive:
            return ""
        case RefreshStatus.Drag:
            return "继续下拉"
        case RefreshStatus.OverDrag:
            return "松手加载"
        case RefreshStatus.Refresh:
            return "加载中"
    }
    return ""
}

@Builder
getRefreshDisplay() {
    Row({ space: 10 }) {
        LoadingProgress()
            .color($r('app.color.primary'))
            .width(40)
            .height(40)
        Text(this.getStatusText())
            .fontColor($r('app.color.text_secondary'))
            .fontSize(14)
    }
    .justifyContent(FlexAlign.Center)
    .height(50)
    .width('100%')
}

```

- 绑定builder

```
build() {  
  Refresh({ refreshing: $$this.refreshing, builder: this.getRefreshDisplay }) {  
    List() {  
      ForEach(this.dataSource, (item: object) => {  
        ListItem() {  
          if (this.renderItem) {  
            this.renderItem(item)  
          }  
        }.width('100%')  
      })  
      ListItem() {  
        this.getBottomDisplay()  
      }  
    }  
  }  
}
```



提交代码

## 任务列表跳转到任务详情

entry模块

待提货

在途

已完成

任务编号: XAHH1234567

已延迟

起

北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00

提货

任务编号: XAHH1234568

起

北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00 - 14:00

提货

2022.05.04 13:00-14:00

任务编号: XAHH1234569

**起** 北京市昌平区回龙观街道西三旗桥东金燕龙写字楼8877号

**止** 河南省郑州市路北区北清路99号



任务



消息



我的

9:41



任务详情

取消任务

基本信息



**起** 北京市昌平区回龙观街道西三旗桥东金燕龙写字楼8877号

**止** 河南省郑州市路北区北清路99号

任务编号

1557211886558850

联系人

张三



联系电话

13512345678



提货时间

2022.05.04 13:00

预计送达时间

2022.05.05 10:00

车辆司机信息



运输路线



物品信息



提货信息



延迟提货

提货

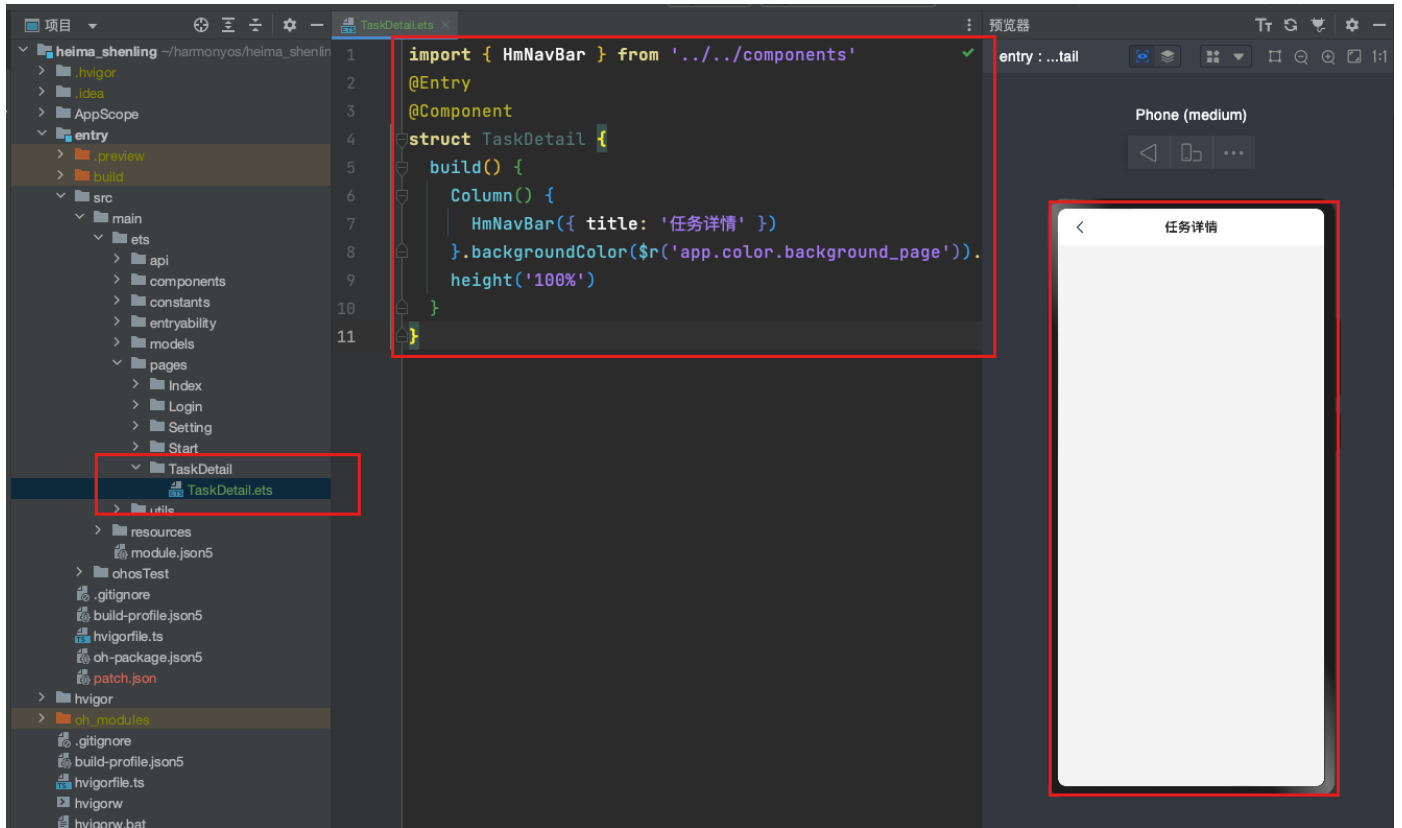
:::info  
分析

1. 在提货按钮可用情况下，点击提货，跳转到任务详情
2. 通过路由传入任务详情的id

3. 在任务详情接受任务id

...

4. 新建一个TaskDetail的页面，因为是列表 => 详情，所以TaskDetail应该是个页面，在pages/TaskDetail/中新建一个TaskDetail.ets



代码

```
import { HmNavBar } from '@hm/basic'
@Entry
@Component
struct TaskDetail {
    build() {
        Column() {
            HmNavBar({ title: '任务详情' })
        }.backgroundColor($r('app.color.background_page')).
        height('100%')
    }
}
```

2. 在TaskItemCard组件中，监听onClick事件中，通过路由跳转传入任务id到任务详情页

```
// 去提货
toPickUp () {
  if(this.getBtnEnable()) {
    router.pushUrl({
      url: 'pages/TaskDetail/TaskDetail',
      params: {
        id: this.taskItemData.id
      }
    })
  }
}
```

```
Button("提货", { type: ButtonType.Capsule })
  .backgroundColor(this.getBtnEnable() ? $r(#EF4F3F) : $r(#FADCD9))
  .fontColor($r("app.color.white"))
  .fontSize(14)
  .height(32)
  .enabled(this.getBtnEnable())
  .onClick(() => {
    this.toPickUp()
  })
}
justifyContent(FlexAlign.SpaceBetween) width('100%')
```

3. 在TaskDetail中接收id参数，并显示

#### basic模块

- 声明一个统一的路由class来接收Params参数
- models/router.ets

```
export class CommonRouterParams {
  id?: string = ''
}
```

- 在models/index.ets 统一导出

```
export * from './router'
```

#### entry模块

```
aboutToAppear() {  
    const params = router.getParams() as CommonRouterParams  
    if(params && params.id) {  
        AlertDialog.show({  
            message: params.id  
        })  
    }  
}
```

:::info

- 这里特别注意下，大家需要用一個剛註冊的司機賬號，才可以擁有一個待提貨權限，不要和老師同學使用一個賬號，這樣狀態會極其混亂，每個同學請注意自己的司機賬號-注册地址-<https://fe-slwl-manager.itheima.net/#/driver-register>

...

6:06

100

待提货

在途

已完成

任务编号: 4032912593983936603

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2024-03-29 20:00:00

提货

任务编号: 4032912593986848382

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2024-03-29 20:00:00

提货

任务编号: 4032912593989662078

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2024-03-29 20:00:00

提货



任务



消息



我的



:::info

总结

通过路由跳转传递id，就可以进行数据查询了  
提交代码

...

## 根据id获取任务详情数据

---

:::info

标准流程

- 定义数据类型
- 封装api
- 获取数据
- 赋值数据
- 显示数据
- ...

basic模块

models/common

- 定义一个公共的图片类型

```
// 后续上传 basic模块也需要类型
export interface ImageList {
  /** 图片url */
  url: string;
}
export class ImageListModel implements ImageList {
  url: string = ''

  constructor(model: ImageList) {
    this.url = model.url
  }
}
```

- 在modes/index.ets导出

```
export * from './common'
```

## entry模块

### 1. 拷贝接口返回的详情类型 [接口文档](#)

apifox生成的接口类型太多重复，直接拷贝下面的接口用i2c生成即可-新建一个task\_detail.ets, 字段太多，单独分开下

```
import { ImageList } from '@hm/basic'
/** 响应数据，响应数据 */
export interface TaskDetailInfo {
  /** 实际到达时间 */
  actualArrivalTime: string;
  /** 实际发车时间 */
  actualDepartureTime: string;
  /** 提货凭证 */
  cargoPickUpPictureList: ImageList[];
  /** 提货图片 */
  cargoPictureList: ImageList[];
  /** 回单凭证 */
  certificatePictureList: ImageList[];
```

```

/** 回单图片 */
deliverPictureList: ImageList[];
/** 司机id */
driverId: string;
/** 司机姓名 */
driverName: string;
/** 目的市 */
endAddress: string;
/** 目的机构id */
endAgencyId: string;
/** 目的机构详细地址 */
endCity: string;
/** 目的省份 */
endProvince: string;
exceptionList: ExceptionList[];
/** 交付对接人 */
finishHandoverName: string;
/** 交付对接人电话 */
finishHandoverPhone: string;
/** 司机作业单id */
id: string;
/** 车牌号码 */
licensePlate: string;
/** 计划到达时间 */
planArrivalTime: string;
/** 计划发车时间 */
planDepartureTime: string;
/** 起始机构详细地址 */
startAddress: string;
/** 起始机构id */
startAgencyId: string;
/** 起始市 */
startCity: string;
/** 提货对接人 */
startHandoverName: string;
/** 提货对接人电话 */
startHandoverPhone: string;
/** 起始省份 */
startProvince: string;
/** 作业状态, 1为待提货)、2为在途)、3为改派)、4为已交付)、5为已作废、6为已完成(已回车登记) */
status: number;
/** 运输任务id */
transportTaskId: string;
}

export interface ExceptionList {
/** 异常描述 */
exceptionDescribe: string;
/** 异常图片 */
exceptionImagesList: ImageList[];
/** 上报的位置 */

```



```

exceptionPlace: string;
/** 异常时间 */
exceptionTime: string;
/** 异常类型(中文) */
exceptionType: string;
/** 处理结果 */
handleResult: string;
}

```

```

export class TaskDetailInfoModel implements TaskDetailInfo {
    actualArrivalTime: string = ''
    actualDepartureTime: string = ''
    cargoPickUpPictureList: ImageList[] = []
    cargoPictureList: ImageList[] = []
    certificatePictureList: ImageList[] = []
    deliverPictureList: ImageList[] = []
    driverId: string = ''
    driverName: string = ''
    endAddress: string = ''
    endAgencyId: string = ''
    endCity: string = ''
    endProvince: string = ''
    exceptionList: ExceptionList[] = []
    finishHandoverName: string = ''
    finishHandoverPhone: string = ''
    id: string = ''
    licensePlate: string = ''
    planArrivalTime: string = ''
    planDepartureTime: string = ''
    startAddress: string = ''
    startAgencyId: string = ''
    startCity: string = ''
    startHandoverName: string = ''
    startHandoverPhone: string = ''
    startProvince: string = ''
    status: number = 0
    transportTaskId: string = ''

    constructor(model: TaskDetailInfo) {
        this.actualArrivalTime = model.actualArrivalTime
        this.actualDepartureTime = model.actualDepartureTime
        this.cargoPickUpPictureList = model.cargoPickUpPictureList
        this.cargoPictureList = model.cargoPictureList
        this.certificatePictureList = model.certificatePictureList
        this.deliverPictureList = model.deliverPictureList
        this.driverId = model.driverId
        this.driverName = model.driverName
        this.endAddress = model.endAddress
        this.endAgencyId = model.endAgencyId
        this.endCity = model.endCity
        this.endProvince = model.endProvince
    }
}

```

```

    this.exceptionList = model.exceptionList
    this.finishHandoverName = model.finishHandoverName
    this.finishHandoverPhone = model.finishHandoverPhone
    this.id = model.id
    this.licensePlate = model.licensePlate
    this.planArrivalTime = model.planArrivalTime
    this.planDepartureTime = model.planDepartureTime
    this.startAddress = model.startAddress
    this.startAgencyId = model.startAgencyId
    this.startCity = model.startCity
    this.startHandoverName = model.startHandoverName
    this.startHandoverPhone = model.startHandoverPhone
    this.startProvince = model.startProvince
    this.status = model.status
    this.transportTaskId = model.transportTaskId
  }
}

export class ExceptionListModel implements ExceptionList {
  exceptionDescribe: string = ''
  exceptionImagesList: ImageList[] = []
  exceptionPlace: string = ''
  exceptionTime: string = ''
  exceptionType: string = ''
  handleResult: string = ''

  constructor(model: ExceptionList) {
    this.exceptionDescribe = model.exceptionDescribe
    this.exceptionImagesList = model.exceptionImagesList
    this.exceptionPlace = model.exceptionPlace
    this.exceptionTime = model.exceptionTime
    this.exceptionType = model.exceptionType
    this.handleResult = model.handleResult
  }
}

```

使用i2c生成对应实现类

- 统一导出

```
export * from './task_detail'
```

## 2. 封装请求任务详情api- task.ets

```

import { TaskDetailInfoModel } from '../models'

export const getTaskDetail = (id: string) => {
  return Request.get<TaskDetailInfoModel>(`/tasks/details/${id}`)
}

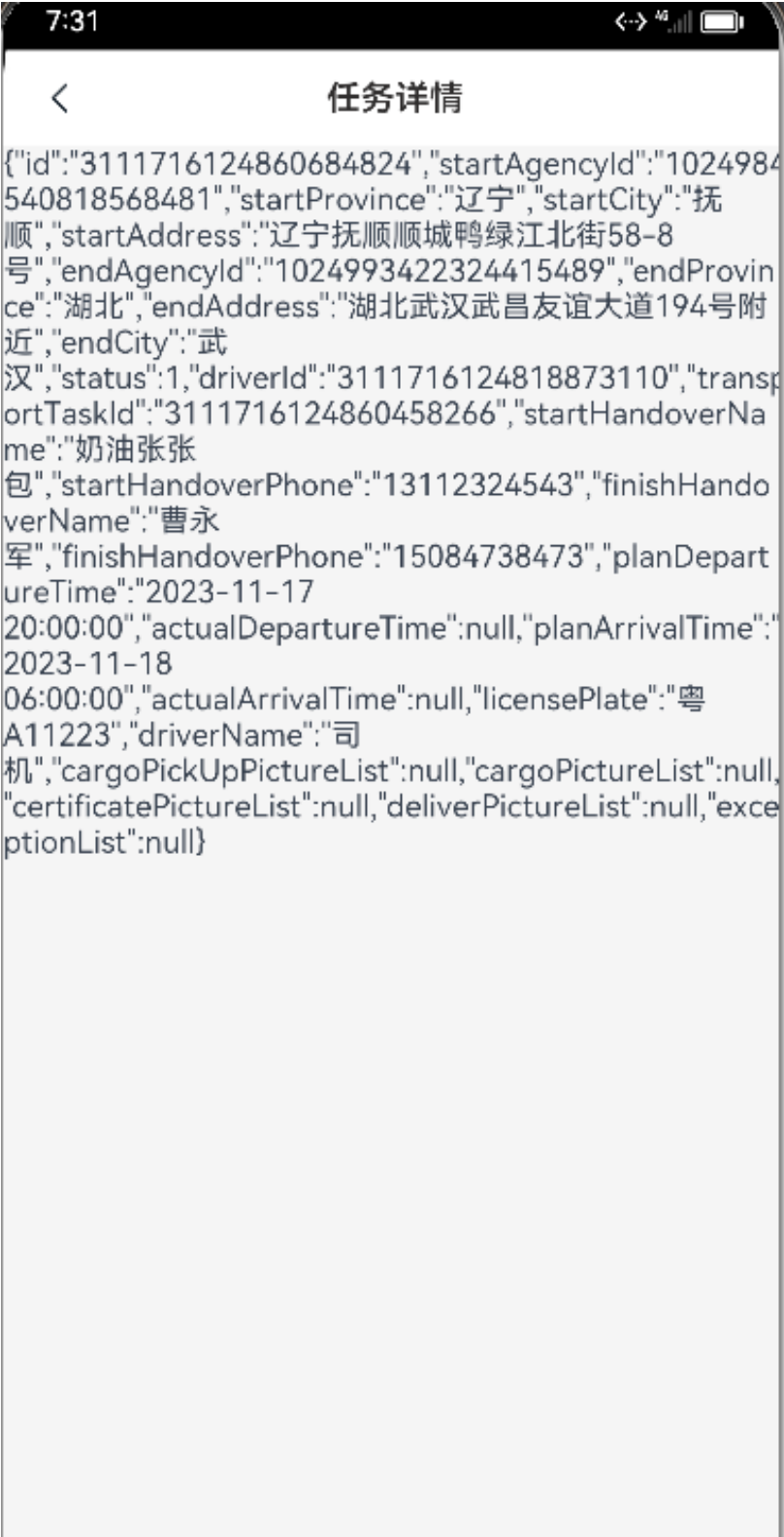
```

### 3. 在任务详情封装方法，调用，赋值状态显示数据

```
import { HmNavBar, CommonRouterParams } from '@hm/basic'
import { router } from '@kit.ArkUI'
import { TaskDetailInfo, TaskDetailInfoModel } from '../../models'
import { getTaskDetail } from '../../api'

@Entry
@Component
struct TaskDetail {
  @State
  taskDetailData: TaskDetailInfoModel = new TaskDetailInfoModel({} as TaskDetailInfo)
  aboutToAppear() {
    const params = router.getParams() as CommonRouterParams
    if(params && params.id) {
      this.getTaskDetail(params.id)
    }
  }
  async getTaskDetail(id: string) {
    this.taskDetailData = await getTaskDetail(id)
  }
  build() {
    Column() {
      HmNavBar({ title: '任务详情' })
      Text(JSON.stringify(this.taskDetailData))
    }.backgroundColor($r('app.color.background_page')).
    height('100%')
  }
}
```

在页面中显示数据， 如图



:::info  
提交代码  
:::

# 任务详情结构-封装折叠容器-HmToggleCard



## 基本信息



**起** 北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

**止** 河南省郑州市路北区北清路99号

任务编号 1557211886558850

联系人 张三

联系电话 13512345678



提货时间 2022.05.04 13:00

预计送达时间 2022.05.05 10:00

## 车辆司机信息



## 运输路线



## 物品信息



坦华信自



延迟提货

提货

### basic模块

:::info

- 容器可以展开-可以折叠,
- 可以存放子组件内容
- 可以显示标题

...

在components下新建 HmToggleCard.ets

```
@Component
struct HmToggleCard {
    title: string = "基本信息"
    @State
    toggleCard: boolean = true // 是否展示
    // 尾随闭包
    @BuilderParam
    CardContent: () => void
    build() {
        Column() {
            Column() {
                Row () {
                    Text(this.title)
                        .fontSize(16)
                        .fontColor($r("app.color.text_primary"))
                        .fontWeight(500)
                    Image(this.toggleCard ? $r("app.media.ic_btn_cut") :
$r("app.media.ic_btn_add"))
                        .width(24)
                        .aspectRatio(1)
                        .onClick(() => {
                            animateTo({ duration: 300 }, () => {
                                this.toggleCard = !this.toggleCard
                            })
                        })
                })
            }
        }
        .width("100%")
        .height(50)
    }
}
```

```

        .justifyContent(FlexAlign.SpaceBetween)
        if(this.toggleCard && this.CardContent) {
            this.CardContent()
        }
    }
    .backgroundColor($r("app.color.white"))
    .borderRadius(10)
    .padding({
        left: 18,
        right: 18
    })

}
.padding(10)

}
}
export { HmToggleCard }

```

在index.ts中导出

```
export * from './HmToggleCard'
```

### entry模块

- 在任务详情中导入测试

```
import { HmToggleCard } from '@hm/basic'
```

```

build() {
    Column() {
        HmNavBar({ title: '任务详情' })
        HmToggleCard({ title: '基本信息' }) {
            Text(JSON.stringify(this.TaskDetailData))
        }
    }
    .backgroundColor($r("#F4F4F4"))
    height('100%')
}

```



:::info  
提交代码  
:::

## 使用Scroll组件实现滚动结构

:::info  
注意：  
Scroll有且只能有一个子组件，不要设置Scroll的高度，让内容去自动撑满  
:::

```
scroller: Scroller = new Scroller() // 滚动

build() {
```



```
Column () {
  HmNavBar({ title: '任务详情' })
  Scroll(this.scroller) {
    Column() {
      HmToggleCard({ title: '基本信息' }) {
        Text(JSON.stringify(this.taskDetailData))
      }
      HmToggleCard({ title: '基本信息' }) {
        Text(JSON.stringify(this.taskDetailData))
      }
      HmToggleCard({ title: '基本信息' }) {
        Text(JSON.stringify(this.taskDetailData))
      }
    }.padding({
      bottom: 20
    })
  }
}.backgroundColor($r('app.color.background_page')).height('100%')
}
```



...info  
提交代码  
...

## 任务详情静态-基本信息展示

PushKit  
mpass



## 基本信息



**起** 北京市昌平区回龙观街道西三旗桥东金燕龙  
写字楼8877号

**止** 河南省郑州市路北区北清路99号

任务编号 1557211886558850

联系人 张三

联系电话 13512345678 

提货时间 2022.05.04 13:00

预计送达时间 2022.05.05 10:00

## 车辆司机信息



车牌号 京A123456

司机姓名 张三

运输路线



北京市  
北京市



河南省  
郑州市

## 提货信息



请拍照上传回单凭证



请拍照上传货品照片



设计图中的货物详情-因接口没有实际数据，所以不在实现。

...

1. 实现基本信息数据展示-拷贝静态结构

```

@Extend(Text) function baseTextOneStyle() {
    .fontSize(12)
    .fontColor($r('app.color.white'))
    .backgroundColor($r('app.color.text_primary'))
    .width(22)
    .height(22)
    .borderRadius(11)
    .textAlign(TextAlign.Center)
}

@Extend(Text) function baseTextTwoStyle() {
    .margin({ left: 11.5
})}.fontColor($r('app.color.text_secondary')).fontSize(14).lineHeight(20)
}

```

- 准备一个class提供给builder函数使用

builder里面的数据 如果用传参数的形式- 基础数据不响应 引用数据类型是响应式的  
`builder($$: { title: string, name: string }).Next`版本不允许这种声明类型的方式

```

class BaseBuilderClass {
    title: string = ""
    value: string = ""
    icon?: ResourceStr = ""
}

```

同学们问：为什么这里不在models中声明了，因为这里只是在我们业务组件内容进行一个简单的使用，不涉及以后的数据通用型，所以在这里使用是可以的

## 自定义builder

```

@Builder
getBaseContentItem(item: BaseBuilderClass) {
    Row() {
        Text(item.title).fontSize(14).fontColor($r('app.color.text_secondary'))
            .lineHeight(20)
        Row() {
            Text(item.value).fontSize(14).fontColor($r('app.color.text_secondary'))
            if (item.icon) {
                Image(item.icon).width(24).height(24)
            }
        }
    }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
        top: 14
    })
}

// 获取基础信息
@Builder
getBaseContent() {
    Row() {
        Column() {

```

```

        Row() {
            Text("起").baseTextOneStyle()
            Text("北京市昌平区回龙观街道西三旗桥东金燕龙写字楼8877号").baseTextTwoStyle()
        }.margin({ top: 21 })
        Row() {
            Text("止").baseTextOneStyle().backgroundColor($r('app.color.primary'))
            Text("河南省郑州市路北区北清路99号").baseTextTwoStyle()
        }.margin({ top: 14.5 })
    }.alignItems(HorizontalAlign.Start)
    .layoutWeight(1)
    .margin({
        right: 20
    })

    Column() {
        Image($r("app.media.ic_navigation")).width(22).height(22)
        Text("开始导航").fontSize(14).margin({ top: 10, bottom: 10 })
    }.justifyContent(FlexAlign.SpaceBetween)
    .margin({
        top: 20
    })

}.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).width('100%')

    Divider().vertical(false).height(2).color($r('app.color.background_divider')).margin({
left: 8, right: 8, top: 21 })
    this.getBaseContentItem({
        title: '任务编号',
        value: '2132324324343434'
    })
    this.getBaseContentItem({
        title: '联系人',
        value: '2132324324343434'
    })
    this.getBaseContentItem({
        title: '联系电话',
        value: '2132324324343434',
        icon: $r('app.media.ic_phone')
    })
    this.getBaseContentItem({
        title: '提货时间',
        value: '2132324324343434'
    })
    this.getBaseContentItem({
        title: '预计送达时间',
        value: '2132324324343434'
    })
}

```



### 基本信息

**起** 北京市昌平区回龙观街道西三旗桥东  
金燕龙写字楼8877号



**止** 河南省郑州市路北区北清路99号

开始导航

|        |                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------|
| 任务编号   | 2132324324343434                                                                                     |
| 联系人    | 2132324324343434                                                                                     |
| 联系电话   | 2132324324343434  |
| 提货时间   | 2132324324343434                                                                                     |
| 预计发货时间 | 2132324324343434                                                                                     |
| 预计送达时间 | 2132324324343434                                                                                     |

替换为真实数据

### 基本信息

**起** 辽宁抚顺顺城鸭绿江北街58-8号



**止** 湖北武汉武昌友谊大道194号附近

开始导航

|        |                                                                                                 |
|--------|-------------------------------------------------------------------------------------------------|
| 任务编号   | 3111716124860458266                                                                             |
| 联系人    | 奶油张张包                                                                                           |
| 联系电话   | 13112324543  |
| 提货时间   | 2023-11-17 20:00:00                                                                             |
| 预计送达时间 | 2023-11-18 06:00:00                                                                             |

```
this.getBaseContentItem({
  title: '任务编号',
  value: this.taskDetailData.transportTaskId
})
this.getBaseContentItem({
  title: '联系人',
```

```

        value: this.taskDetailData.startHandoverName
    })
    this.getBaseContentItem({
        title: '联系电话',
        value: this.taskDetailData.startHandoverPhone,
        icon: $r('app.media.ic_phone')
    })
    this.getBaseContentItem({
        title: '提货时间',
        value: this.taskDetailData.planDepartureTime
    })
    this.getBaseContentItem({
        title: '预计送达时间',
        value: this.taskDetailData.planArrivalTime
    })

```

- 准备builder函数

```

// 司机信息
@Builder
getDriverContent() {
    Row() {
        Text(`车牌号`).fontSize(14).fontColor($r('app.color.text_secondary'))
            .lineHeight(20)
        Text("京A123456").fontSize(14).fontColor($r('app.color.text_secondary'))
    }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
        top: 14
    })

    Row() {
        Text(`司机姓名`).fontSize(14).fontColor($r('app.color.text_secondary'))
            .lineHeight(20)
        Text("张三").fontSize(14).fontColor($r('app.color.text_secondary'))
    }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
        top: 14
    })
}

// 运输路线
@Builder
getTransLineContent() {
    Row() {
        Column() {
            Text("北京
市").fontSize(16).fontColor($r('app.color.text_primary')).lineHeight(22).fontWeight(600)
            Text("北京市").fontSize(14).lineHeight(22)
        }.width(50)

        Image($r("app.media.ic_right_arrow")).width(36).height(16)
        Column() {
            Text("河南
省").fontSize(16).fontColor($r('app.color.text_primary')).lineHeight(22).fontWeight(600)

```

```

        Text("郑州市").fontSize(14).lineHeight(22)
    }.width(50)

}).justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).width('100%').padding({
    left: 60,
    right: 60
})
}

```

- 放置车辆司机信息和运输路线插槽

```

build() {
    Column () {
        HmNavBar({ title: '任务详情' })
        Scroll(this.scroller) {
            Column() {
                HmToggleCard({ title: '基本信息' }) {
                    this.getBaseContent()
                }
                HmToggleCard({ title: '车辆司机信息' }) {
                    this.getDriverContent()
                }
                HmToggleCard({ title: '运输路线' }) {
                    this.getTransLineContent()
                }
            }.padding({
                bottom: 20
            })
            .layoutWeight(1)
        }
    }.backgroundColor($r('app.color.background_page')).height('100%')
}

```

替换真实的数据

```
getDriverContent() {
  Row() {
    Text("车牌号").fontSize(14).fontColor($r('app.color.text_secondary'))
    .lineHeight(20)
    Text(this.TaskDetailData.licensePlate).fontSize(14).fontColor($r('app.color.text_secondary'))
  }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
    top: 14
  })

  Row() {
    Text('司机姓名').fontSize(14).fontColor($r('app.color.text_secondary'))
    .lineHeight(20)
    Text(this.TaskDetailData.driverName).fontSize(14).fontColor($r('app.color.text_secondary'))
  }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
    top: 14
  })
}

getTransLineContent() {
  Row() {
    Column() {
      Text(this.TaskDetailData.startProvince).fontSize(16).fontColor($r('app.color.text_secondary'))
      Text(this.TaskDetailData.startCity).fontSize(14).lineHeight(22)
    }.width(50)

    Image($r("app.media.ic_right_arrow")).width(36).height(16)

    Column() {
      Text(this.TaskDetailData.endProvince).fontSize(16).fontColor($r('app.color.text_secondary'))
      Text(this.TaskDetailData.endCity).fontSize(14).lineHeight(22)
    }.width(50)
  }.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center)
  left: 60,
  right: 60
}
```

<

任务详情

基本信息

起

辽宁抚顺顺城鸣绿江北街58-8号

止

湖北武汉武昌友谊大道194号附近

开始导航

任务编号

3111716124860458266

联系人

奶油张张包

联系电话

13112324543

提货时间

2023-11-17 20:00:00

预计送达时间

2023-11-18 06:00:00



:::info

总结

使用静态模版，利用插槽技术，替换真实数据  
提交代码

:::

# 提货信息-上传组件HmUpload基本封装

basic模块

# 提货信息



请拍照上传回单凭证



请拍照上传货品照片



:::info

分析

需要上传组件进行图片的上传，我们需要封装一个公共的组件HmUpload  
先完成基本的布局

:::

1. 新建components下的HmUpload.ets文件， 完成最基本布局

请拍照上传回单凭证



```
@Component
struct HmUpload {
  title: string = ""
  build() {
    Column() {
      Text(this.title).fontSize(14).fontColor($r("app.color.text_secondary")).margin({
        top: 16,
        bottom: 16
      })
      Row() {
        Image($r("app.media.ic_add_img")).width(30).height(30)
      }
      .width(95)
      .height(95)
      .backgroundColor('#F2F2F2')
      .alignItems(VerticalAlign.Center)
      .justifyContent(FlexAlign.Center)
    }.alignItems(HorizontalAlign.Start).width('100%')
  }
}
export { HmUpload }
```

2. 在components/index.ets导出

```
export * from './HmUpload'
```

## Entry模块

3. 在TaskDetail中使用

```
// 提货信息内容
@Builder
getPickUpContent() {
```

```

    HmUpload({ title: '请拍照上传回单凭证' })
    HmUpload({ title: '请拍照上传货品照片' })
  }
  build() {
    Column() {
      HmNavBar({ title: '任务详情' })
      Scroll(this.scroller) {
        Column() {
          HmToggleCard({ title: '基本信息' }) {
            this.getBaseContent()
          }

          HmToggleCard({ title: '车辆司机信息' }) {
            this.getDriverContent()
          }

          HmToggleCard({ title: '运输路线' }) {
            this.getTransLineContent()
          }

          HmToggleCard({ title: '提货信息' }) {
            this.getPickUpContent()
          }
        }.padding({
          bottom: 120
        })
      }
    }.backgroundColor($r('app.color.background_page')).height('100%')
  }
}

```



:::info

总结

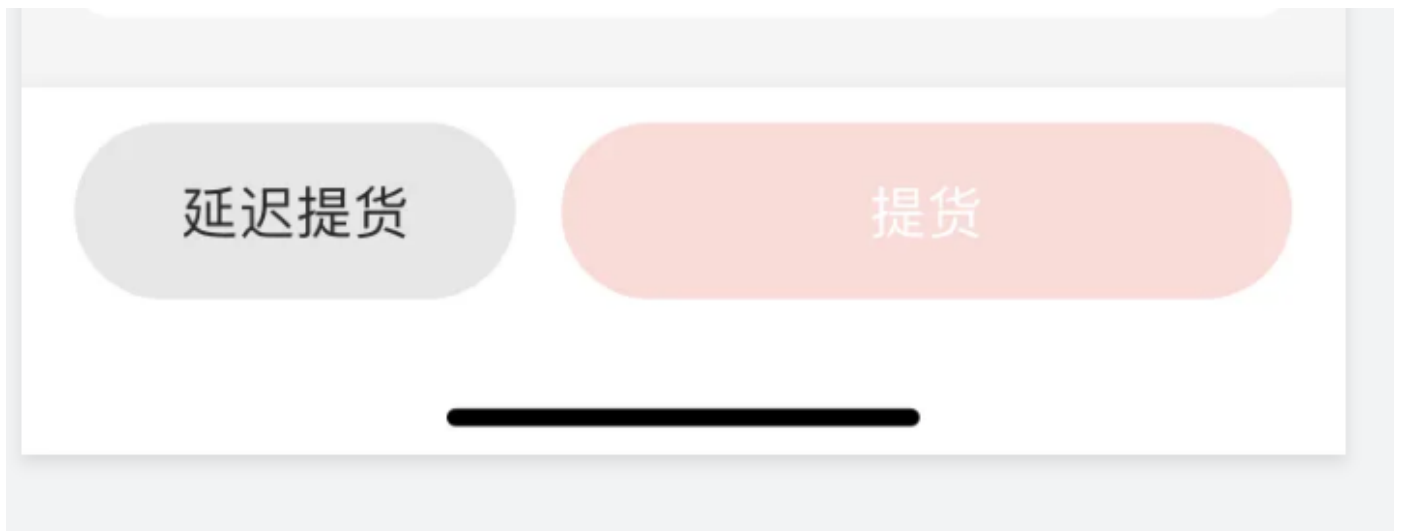
先封装HmUpload的基本结构，后续再去实现上传



提交代码

...

## 任务详情-底部提货按钮结构



:::info

实现上传组件前，先把底部的提交结构实现

...

### 1. 从静态页面中拷贝结构

```
// 底部按钮结构
@Builder
getBottomBtn() {
  //已完成不显示任何按钮
  Row() {
    Button("延迟收货", { type: ButtonType.Capsule })
      .backgroundColor($r('app.color.btn_gray'))
      .fontColor($r('app.color.text_primary'))
      .fontSize(16)
      .height(50)
      .width(125)
    Button("提货", { type: ButtonType.Capsule })
      .backgroundColor($r('app.color.primary_disabled'))
      .fontColor($r('app.color.white'))
      .height(50)
      .flexGrow(1)
      .margin({ left: 13 })
  }
  .width('100%')
  .padding({ left: 15, right: 15 })
  .height(70)
  .justifyContent(FlexAlign.SpaceBetween)
  .alignItems(VerticalAlign.Center)
  .backgroundColor($r('app.color.white'))
}
```

2. 放置在Scroll组件的同级，因为我们希望底部按钮是不滚动的

```
build() {  
  Column() {  
    HmNavBar({ title: '任务详情' })  
    Scroll(this.scroller) {  
      Column() {  
        HmToggleCard({ title: '基本信息' }) {  
          this.getBaseContent()  
        }  
  
        HmToggleCard({ title: '车辆司机信息' }) {  
          this.getDriverContent()  
        }  
  
        HmToggleCard({ title: '运输路线' }) {  
          this.getTransLineContent()  
        }  
  
        HmToggleCard({ title: '提货信息' }) {  
          this.getPickUpContent()  
        }  
      }.padding({  
        bottom: 20  
      })  
      .layoutWeight(1)  
    }  
    this.getBottomBtn() // 底部按钮结构  
  }.backgroundColor($r('app.color.background_page')).height('100%')  
}
```



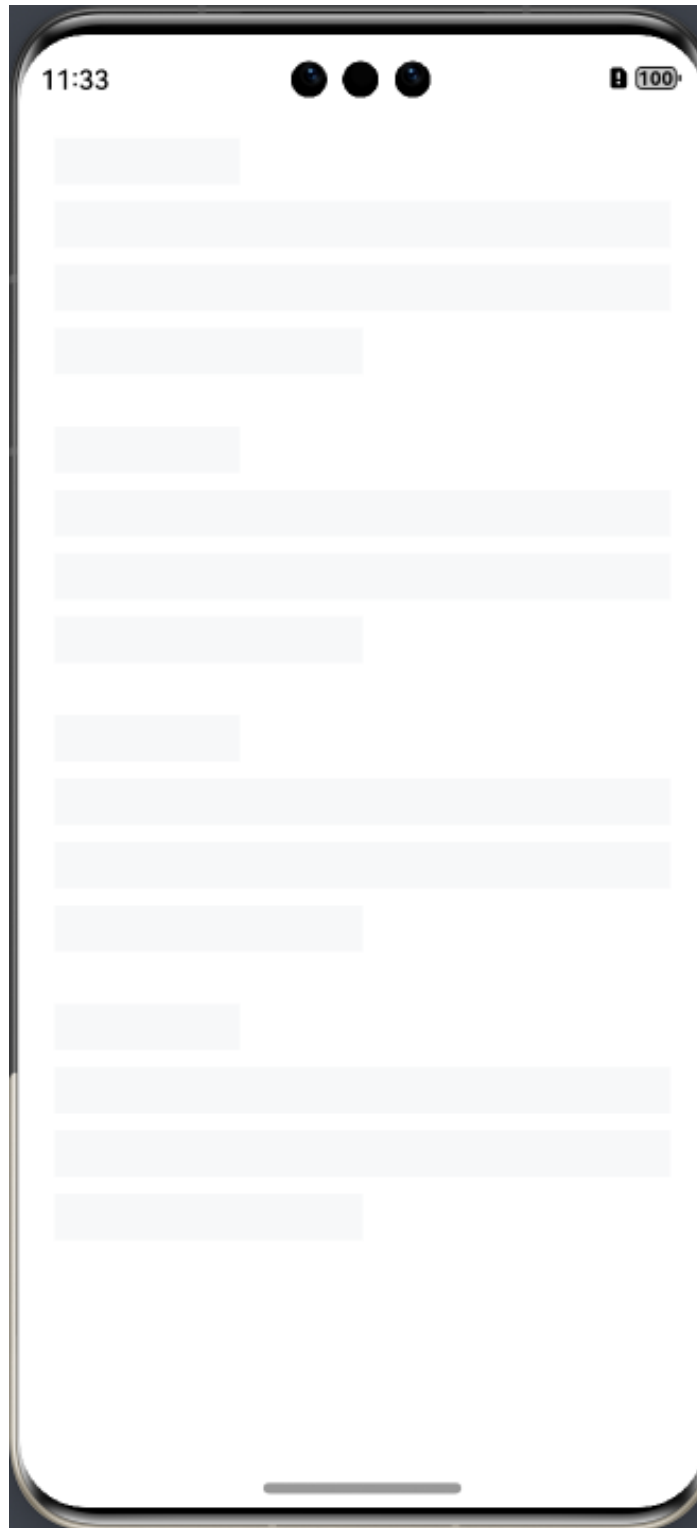
:::info  
总结  
提交代码  
:::

## 添加骨架屏判断

```
build() {  
  Column () {  
    if(this.taskDetailData.id) {  
      HmNavBar({ title: '任务详情' })  
      Scroll(this.scroller) {  
        Column() {  
          HmToggleCard({ title: '基本信息' }) {  
            this.getBaseContent()  
          }  
        }  
      }  
    }  
  }  
}
```

```
    }
    HmToggleCard({ title: '车辆司机信息' }) {
      this.getDriverContent()
    }
    HmToggleCard({ title: '运输路线' }) {
      this.getTransLineContent()
    }
    HmToggleCard({ title: '提货信息' }) {
      this.getPickUpContent()
    }
  }
}
.padding({
  bottom: 20
})
.layoutWeight(1)
this.getBottomBtn() // 底部按钮结构
}else {
  HmSkeleton({ count: 4, showAvatar: false })
}

}.backgroundColor($r('app.color.background_page')).height('100%')
}
```



提交代码

## 在其他项目上新建一个工具

目前两种手段可以读取相册

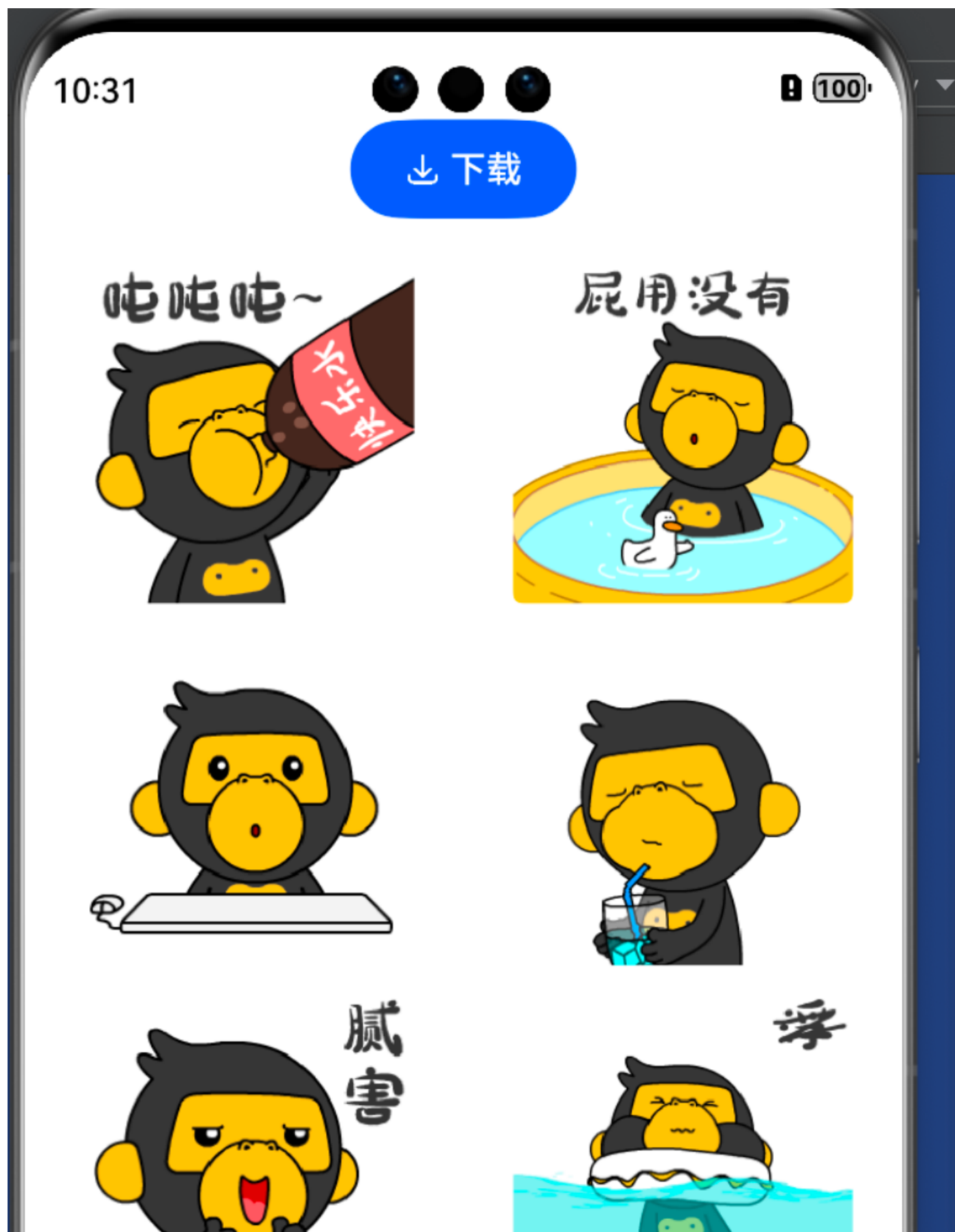
- 你需要向华为官方申请读取相册的权限-邮件申请-包 阐述需要这个权限的原因
- 可以用安全组件-SaveButton可以让你拥有10秒钟的时间读取相册

...info

因为模拟器不让拖动图片到相册目录，所以我们只能自己手动的将线上的图片下载到相册

...

- 在pages/Index.ets中实现





```
import { fileIo } from '@kit.CoreFileKit'
import { photoAccessHelper } from '@kit.MediaLibraryKit';
import { promptAction } from '@kit.ArkUI';
```

```
@Entry
```

```
@Component
```

```
struct FileCopy {
```

```
  @State
```

```
  list: Resource[] = [
```

```
    $r("app.media.001"),
```

```
    $r("app.media.002"),
```

```
    $r("app.media.003"),
```

```
    $r("app.media.004"),
```

```
    $r("app.media.005"),
```

```
    $r("app.media.006"),
```

```
    $r("app.media.007"),
```

```
    $r("app.media.008"),
```

```
    $r("app.media.009"),
```

```
    $r("app.media.010")
```

```
  ]
```

```
// 保存沙箱图片到相册
```

```

async saveImgToAssets() {
  try {
    let index = 0
    while (index < this.list.length) {
      let context = getContext();
      let phAccessHelper = photoAccessHelper.getPhotoAccessHelper(context);
      // Creating a Media File
      let uri = await phAccessHelper.createAsset(photoAccessHelper.PhotoType.IMAGE,
'jpg');
      // Open the created media file and read the local file and convert it to
ArrayBuffer for easy filling.
      let file = await fileIo.open(uri, fileIo.OpenMode.READ_WRITE);
      let buffer =
getContext(this).resourceManager.getMediaContentSync(this.list[index].id);
      // Write the read ArrayBuffer to the new media file.
      let writeLen = await fileIo.write(file.fd, buffer.buffer);
      await fileIo.close(file);
      index++
    }
    promptAction.showToast({ message: '下载成功' })

  } catch (err) {
    AlertDialog.show({ message: err.message })
  }
}

build() {
  Column({ space: 10 }) {
    Row() {
      SaveButton()
        .onClick((event, result: SaveButtonOnClickResult) => {
          if (result === SaveButtonOnClickResult.SUCCESS) {
            this.saveImgToAssets()
          }
        })
    }
    .justifyContent(FlexAlign.Center)
    .width('100%')
    GridRow({ columns: 2 }) {
      ForEach(this.list, (item: string) => {
        GridCol() {
          Image(item)
            .height(150)
            .height(150)
            .borderRadius(4)
        }
        .margin({
          top: 10
        })
      })
    }
  }
}

```



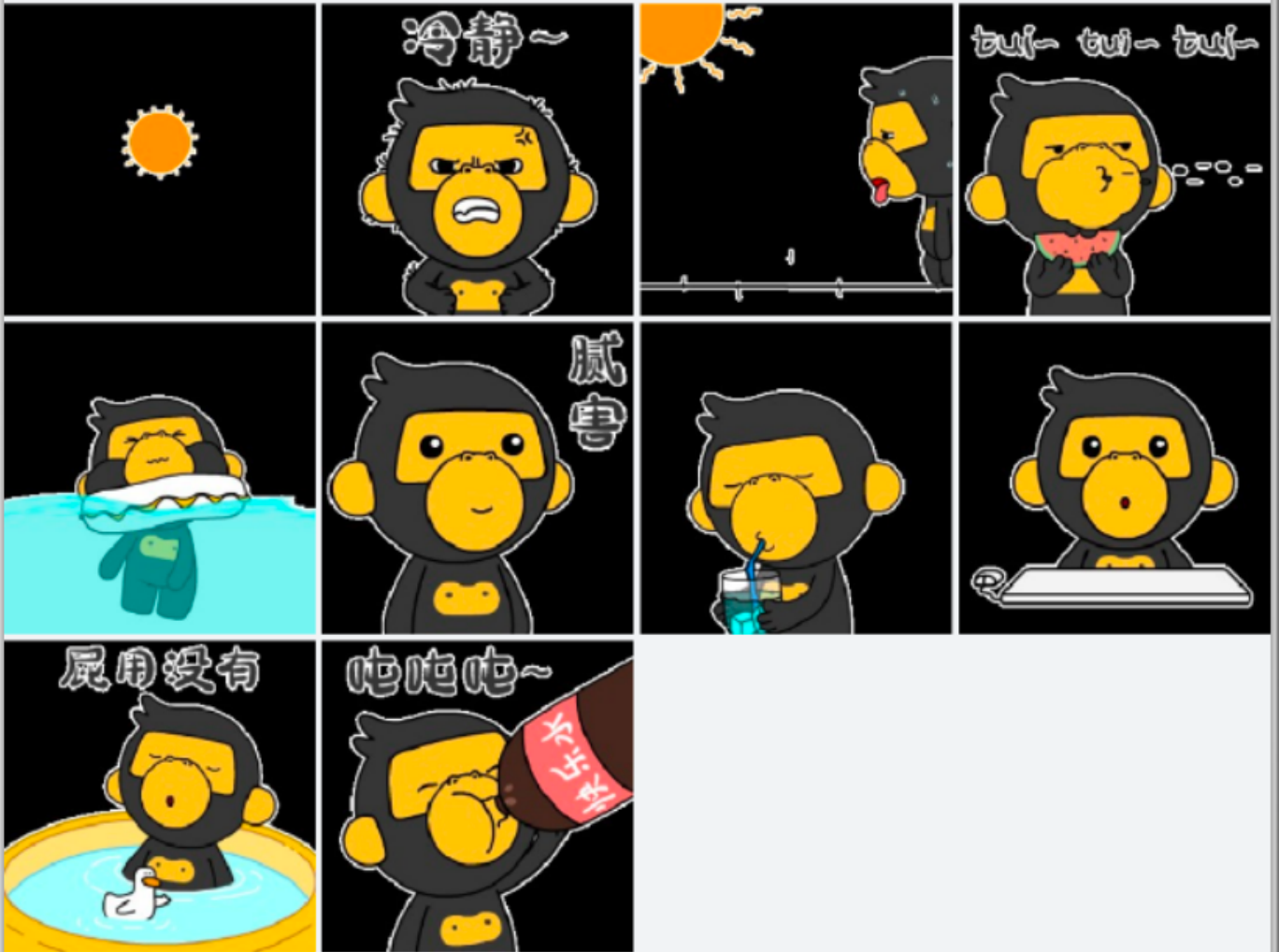
}

}

}

- 点击按钮拷贝图片

今天



:::success

项目仓库地址: [https://gitee.com/shuiruohanyu/copy\\_image](https://gitee.com/shuiruohanyu/copy_image)

...

# 上传功能-唤出相册- 拿到选择图片

## basic模块

1. 点击上传区域，弹出图片选择器

```
// 弹出相册选择器
selectImage () {
  let photoPicker = new picker.PhotoViewPicker();
  photoPicker.select({
    MimeType: picker.PhotoViewMIMETypes.IMAGE_TYPE,
    maxSelectNumber: 3
  })
}
```

```
Row() {
  Image($r("app.media.ic_add_img")).width(30).height(30)
}
.width(95)
.height(95)
.backgroundColor('#F2F2F2')
.alignItems(VerticalAlign.Center)
.justifyContent(FlexAlign.Center).onClick(() => {
  this.selectImage()
})
})
```

2. 将图片选择数量设置为可选

```

struct HmUpload {
    title: string = ""
    maxNumber: number = 3
    // 弹出相册选择器
    selectImage () {
        let photoPicker = new picker.PhotoViewPicker();
        photoPicker.select({
            MIMEType: picker.PhotoViewMIMETypes.IMAGE_TYPE,
            maxSelectNumber: this.maxNumber
        })
    }
}

build() {

```

此时可以在使用HmUpload时，自由传入maxNumber属性来控制数量

### 3. 获取选择后的图片列表

接收返回的选择图片列表

```

// 弹出相册选择器
async selectImage () {
    let photoPicker = new picker.PhotoViewPicker();
    const result = await photoPicker.select({
        MIMEType: picker.PhotoViewMIMETypes.IMAGE_TYPE,
        maxSelectNumber: this.maxNumber
    })
    AlertDialog.show({
        message: JSON.stringify(result.photoUris)
    })
}

```



:::info

总结

1. 通过file.picker的图片选择器选择图片，得到了一些临时路径
2. 临时路径要传到我们自己的服务器-待实现

### 3. 提交代码

...

## 拿到图片先将图片显示到预览区

- 声明一个数组来管理选择的图片

```
@State
imgList: ImageList[] = []
```

- 将选择的图片赋值给状态

```
// 循环数组
if (result.photoUris?.length) {
    this.imgList = result.photoUris.map(url => {
        return { url } as ImageList
    })
}
```

- 循环渲染图片

```
// 渲染图片
Grid() {
    ForEach(this.imgList, (item: ImageList) => {
        GridItem() {
            Image(item.url)
                .width(95)
                .height(95)
                .borderRadius(4)
        }
    })
    if (this.imgList.length < this.maxNumber) {
        GridItem() {
            Row() {
                Image($r("app.media.ic_add_img")).width(30).height(30)
            }
            .width(95)
            .height(95)
            .backgroundColor($r("app.color.upload_panel"))
            .alignItems(VerticalAlign.Center)
            .justifyContent(FlexAlign.Center)
            .onClick(() => {
                this.selectImage()
            })
        }
    }
}
.height(Math.ceil(this.maxNumber / 3 ) * 105)
.columnsTemplate("1fr 1fr 1fr")
.columnsGap(10)
```

- 发现问题

...success

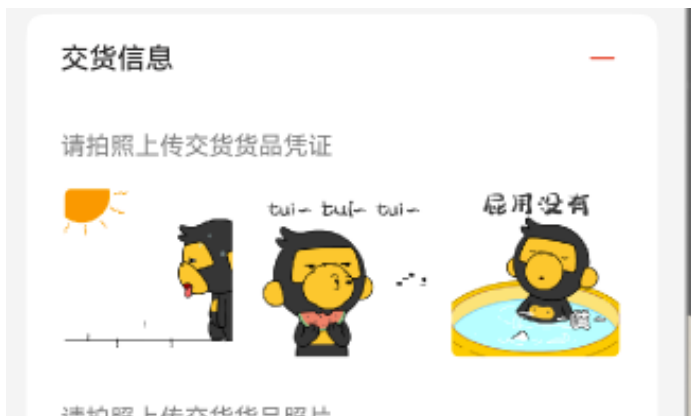
当选择了两张之后，再选一张，发现只剩最后一次选的一张图片了，这是因为我们之前用的直接替换，随意这里我们对图片的选择的图片张数做一下限制

...

```
const photoPicker = new picker.PhotoViewPicker()
const result = await photoPicker.select({
  MimeType: picker.PhotoViewMIMETypes.IMAGE_TYPE,
  maxSelectNumber: this.maxNumber - this.imgList.length
})
```

- 然后进行追加

```
// 循环数组
if (result.photoUris?.length) {
  this.imgList = this.imgList.concat(result.photoUris.map(url => {
    return { url } as ImageList
  })))
}
```



...success

提交代码

...

## 封装图片预览HmPreview

basic模块

...info

分析，当我们上传的图片时，我们希望能去放大预览该图片



并且图片还支持多张一起，比如上传了三张，点第二张可以预览，左右滑可以切到第一张和第三张  
...

#### 1. 新建components/HmPreview.ets文件

```
@CustomDialog
@Component
struct HmPreview {
    controller: CustomDialogController
    // 支持多张图片预览 给多张地址 给一个需要预览的索引
    urls: string[] = [] // 多张地址
    selectIndex: number = -1 // 当前索引
    build() {
        Column() {
            Swiper() {
                ForEach(this.urls, (url: string) => {
                    Image(url)
                        .width("100%") // 只给宽度 不给高度 让自己撑开
                        .onClick(() => {
                            this.controller.close()
                        })
                })
            }
            .indicator(false) // 去掉点的显示
            .index(this.selectIndex) // 当前要看的是第几张图片
        }
        .justifyContent(FlexAlign.Center)
        .width("100%")
        .height("100%")
        .backgroundColor($r("app.color.black"))
    }
}
```

```
export { HmPreview }
```

在components/index.ets导出

```
export * from './HmPreview'
```

- 在HmUpload中引入，并初始化dialogController

```
import { HmPreview } from './HmPreview'

preview: CustomDialogController = new CustomDialogController({
  builder: HmPreview({
    urls: this.imgList.map(item => item.url),
    selectIndex: this.index
  }),
  customStyle: true, // 自定义样式
})
```

- 在HmUpload中声明index属性

```
index: number = -1
```

- 切换点击图片时，切换索引, 打开弹层

```
Image().onClick(() => {
    this.index = index
    this.preview.open()
  })
```



:::info  
提交代码  
:::

## 接收父组件传入



...SUCCESS

因为图片的来源有可能是已经有的图片，所以我们需要外部传入该图片列表

...

- 将State修饰符变成Prop修饰符

```
@Prop
imgList: ImageList[] = []
```

## entry模块

- 在提货详情的页面中进行传入

```
@Builder
getPickUpContent() {
    HmUpload({ title: '请拍照上传货品凭证',
        imgList: this.taskDetailData.cargoPickUpPictureList || []
    })
    HmUpload({ title: '请拍照上传货品照片',
        imgList: this.taskDetailData.cargoPictureList || []
    })
}
```

...SUCCESS

当图片地址发生变化时，通知父组件更新

...

## basic模块

- 在HmUpload中定义一个外部传入的函数

```
onListChange: (list: ImageList[]) => void = () => {}
```

- 在selectImage方法中选择完毕图片之后调用该方法

```
// 循环数组
if (result.photoUris?.length) {
    this.imgList = this.imgList.concat(result.photoUris.map(url => {
        return { url } as ImageList
    }))
    this.onListChange(this.imgList)
}
```

- 父组件传入函数更新对应的字段

## entry模块

```
// 专门负责提货信息的结构
@Builder
getPickUpContent() {
```

```

HmUpload({ title: '请拍照上传货品凭证',
  imgList: this.taskDetailData.cargoPickUpPictureList || [],
  onListChange: (list: ImageList[]) => {
    this.taskDetailData.cargoPickUpPictureList = list
  }
})
HmUpload({ title: '请拍照上传货品照片',
  imgList: this.taskDetailData.cargoPictureList || [],
  onListChange: (list: ImageList[]) => {
    this.taskDetailData.cargoPictureList = list
  }
})
}

```

- 根据提货图片存在判断是否可点击按钮

```

// 获取提货的状态
getPickUpState() {
  return this.taskDetailData.cargoPickUpPictureList?.length > 0 &&
    this.taskDetailData.cargoPictureList?.length > 0 &&
    this.taskDetailData.cargoPickUpPictureList.every(item => !!item.url) &&
    this.taskDetailData.cargoPictureList.every(item => !!item.url)
}

```

- 设置状态

```

Button("提货", { type: ButtonType.Capsule })
  .backgroundColor(this.getPickUpState() ? $r('app.color.primary') :
    $r('app.color.primary_disabled'))
  .fontColor($r('app.color.white'))
  .height(50)
  .flexGrow(1)
  .margin({ left: 13 })
  .enabled(this.getPickUpState())

```

:::success

提交代码

...

## 沙箱文件拷贝

:::info

官网上传案例-[链接](#)

1. 上传文件我们需要使用官方的uploadFile方法

...

:::success

我们需要在点击提货按钮时，将所有的图片进行上传，并且监听上传进度，显示在页面上

...

# request.uploadFile<sup>9+</sup>

uploadFile(context: BaseContext, config: UploadConfig): Promise<UploadTask>

上传，异步方法，使用promise形式返回结果。通过on('fail')<sup>7+</sup>可获取任务上传时的错误信息。

需要权限：ohos.permission.INTERNET

系统能力: SystemCapability.MiscServices.Upload

参数:

| 参数名     | 类型           | 必填 | 说明          |
|---------|--------------|----|-------------|
| context | BaseContext  | 是  | 基于应用程序的上下文。 |
| config  | UploadConfig | 是  | 上传的配置信息。    |

返回值:

:::info  
第一个参数 context可以直接使用 getContext(this)获得  
第二个参数的config参数为

| 名称     | 类型                 | 必填 | 说明                                  |
|--------|--------------------|----|-------------------------------------|
| url    | string             | 是  | 资源地址。                               |
| header | Object             | 是  | 添加要包含在上传请求中的HTTP或HTTPS标志头。          |
| method | string             | 是  | 请求方法：POST、PUT。缺省为POST。              |
| files  | Array<File>        | 是  | 要上传的文件列表。请使用 multipart/form-data提交。 |
| data   | Array<RequestData> | 是  | 请求的表单数据。                            |

其中File的参数为

| 名称       | 类型     | 必填 | 说明                                                                                        |
|----------|--------|----|-------------------------------------------------------------------------------------------|
| filename | string | 是  | multipart提交时，请求头中的文件名。                                                                    |
| name     | string | 是  | multipart提交时，表单项目的名称，缺省为file。                                                             |
| uri      | string | 是  | 文件的本地存储路径。仅支持“internal”协议类型，“internal://cache/”为必填字段，示例：internal://cache/path/to/file.txt |
| type     | string | 是  | 文件的内容类型，默认根据文件名或路径的后缀获取。                                                                  |

...

综上所述，我们需要得到File中的uri这个属性，uri属性仅支持“internal”协议类型

但是我们通过弹窗发现，前面所得到的图片路径为

["datashare:///media/image/356",  
"datashare:///media/image/349"]

...::info  
格式不匹配！！  
需要转化  
怎么转？  
我们需要将选择的图片列表 一个个的拷贝到cache目录下，得到cache目录后新的目标路径  
[应用沙箱文件官网介绍](#)

...

**basic**模块

- 在HmUpload中导出一个上传方法

```
const UploadFile = (list: ImageList[]) => {}  
export { HmUpload, UploadFile }
```

// 上传方法

```
const UploadFile = (list: ImageList[]) => {  
  // 因为上传文件只能从沙箱文件中拷贝所以 我们需要把传过来的所有的图片拷贝到沙箱  
  const saveDir = getContext().cacheDir // 存储的目录  
  const fileParams: request.File[] = [] // 要提交的参数  
  list.forEach(item => {  
    const file = fileIo.openSync(item.url, fileIo.OpenMode.READ_ONLY) // 读取相册的文件  
    // 将文件拷贝到沙箱目录  
    // 相册的地址  
    const uniqueName = util.generateRandomUUID() + ".jpg"  
    fileIo.copyFileSync(file.fd, saveDir + "/" + uniqueName) // 将相册文件拷贝到沙箱  
    // 需要生成参数  
    fileParams.push({  
      filename: uniqueName, // 文件名称  
      name: 'file', // 接口的参数名称  
      type: 'jpg', // 文件后缀  
      uri: `internal://cache/${uniqueName}` // 应该是文件放到cache目录下 如果是cache协议 它会自  
      动找这个文件  
    })  
    fileIo.closeSync(file.fd)  
  })  
  AlertDialog.show({ message: JSON.stringify(fileParams) })  
}
```

- 得到对应的路径



#### 总结

- 文件上传必须先把文件拷贝到沙箱文件中
- 沙箱文件最终要上传时需要需要internal://cache目录 + 自己的文件路径

提交代码

## 封装上传文件的api

:::info

现在上传组件已经准备好了参数，我们封住一个统一的上传api

...

在api下新建upload.ets

```
import request from '@ohos.request'
import { BASE_URL, TOKEN_KEY } from '../constants'
import { ImageList, ResponseData } from '../models'

// 返回一个列表 ImageList[]
export const uploadImage = async (context: Context, files: request.File[]) => {
  let config: request.UploadConfig = {
    url: BASE_URL + '/files/imageUpload', // 拼接上传完整地址
    method: 'POST',
    header: {
      Authorization: AppStorage.get(TOKEN_KEY) || "", // 应用中的token
      "Content-Type": "multipart/form-data" // 上传文件的参数类型
      // application/json { name: '张三', age: 18 }
      // application/x-www-form-urlencoded 拼接 name=张三&age=18
      // text/xml 结构
    }
  }
```

```

    // multipart/form-data 专门传文件的结构
  },
  files,
  data: [] // 批量上传需要携带的参数 用不上
}
return new Promise<ImageList[]>(async (resolve, reject) => {
  try {
    let arr: ImageList[] = []
    const task = await request.uploadFile(context, config) // 成功执行并不意味着 上传成功 只是任务创建成功
    task.on("fail", () => {
      // 上传失败
      AlertDialog.show({
        message: "上传失败"
      })
      reject(new Error("上传失败"))
    })
    // 每上传成功一次就会进来
    task.on("headerReceive", (headers: object) => {
      if (headers["body"]) {
        const result = JSON.parse(headers["body"]) as ResponseData<string>
        if (result.code === 200) {
          arr.push({
            url: result.data as string
          })
        }
      }
    })
    task.on("complete", () => {
      // 上传完成
      // 也不能拿到上传成功的地址
      // 走到这里认为 要返回的结构已经ok了
      resolve(arr)
    })
    // 4.0 上传完成之后 用文件的一个标识 又去后台查了一下 当时是最优方案
  } catch (error) {
    AlertDialog.show({
      message: error.message // 提示错误消息
    })
    reject(error)
  }
})
}

```

- 在api/index.ets中导出

```
export * from './upload'
```

- 在HmUpload组件的UploadFile方法中调用

```
// 上传方法
const UploadFile = async (list: ImageList[]) => {
  // 因为上传文件只能从沙箱文件中拷贝所以 我们需要把传过来的所有的图片拷贝到沙箱
  const saveDir = getContext().cacheDir // 存储的目录
  const fileParams: request.File[] = [] // 要提交的参数
  list.forEach(item => {
    const file = fileIo.openSync(item.url, fileIo.OpenMode.READ_ONLY) // 读取相册的文件
    // 将文件拷贝到沙箱目录
    // 相册的地址
    const uniqueName = util.generateRandomUUID() + ".jpg"
    fileIo.copyFileSync(file.fd, saveDir + "/" + uniqueName) // 将相册文件拷贝到沙箱
    // 需要生成参数
    fileParams.push({
      filename: uniqueName, // 文件名称
      name: 'file', // 接口的参数名称
      type: 'jpg', // 文件后缀
      uri: `internal://cache/${uniqueName}` // 应该是文件放到cache目录下 如果是cache协议 它会自
    })
    fileIo.closeSync(file.fd)
  })
  // 将参数进行上传
  return await uploadImage(getContext(), fileParams) // 调用上传接口 // 将上传的结果再返回上一层
}
```

:::info  
提交代码  
:::

## 提货时上传

### entry模块

#### 1. 拷贝接口类型-[接口文档](#)

- 新建一个pickup提货的ets类型声明文件- models/pickup.ets

```
import { ImageList } from '@hm/basic'
export interface PickUpParams {
  /** 提货凭证照片数组 */
  cargoPickUpPictureList: ImageList[];
  /** 提货照片数组 */
  cargoPictureList: ImageList[];
  /** 司机作业id */
  id: string;
}
export class PickUpParamsModel implements PickUpParams {
  cargoPickUpPictureList: ImageList[] = []
  cargoPictureList: ImageList[] = []
  id: string = ''
}
```



```

constructor(model: PickupParams) {
  this.cargoPickUpPictureList = model.cargoPickUpPictureList
  this.cargoPictureList = model.cargoPictureList
  this.id = model.id
}
}

```

- 导出统一的文件

```
export * from './pick_up'
```

- 封装提货接口

```

import { PickupParamsModel } from '../models'

// 提货
export const pickUp = (data: PickupParamsModel) => {
  return Request.post<null>("/tasks/takeDelivery", data)
}

```

### 3. 提货点击事件-调用api

```

// 去提货
async btnPickUp() {
  const cargoPickUpPictureList = await
UploadFile(this.taskDetailData.cargoPickUpPictureList) // 传入提货凭证
  const cargoPictureList = await UploadFile(this.taskDetailData.cargoPictureList) // 货
品照片
  // 提货操作
  await pickUp(new PickupParamsModel({
    id: this.taskDetailData.id,
    cargoPickUpPictureList,
    cargoPictureList
  }))
  // 只需要重新获取数据
  // 重新获取数据
  this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据

  this.scroller.scrollToEdge(Edge.Top)
  // 滚动到顶部
  promptAction.showToast({ message: '提货成功' })
}

```

点击事件

```
Button("提货")
....
.onClick(() => {
    this.btnPickUp()
})
```

:::info

总结

提交代码

...

## 实现删除图片



1. 在图片中放入右上角的图标

```
ForEach(this.imgList, (item: ImageList, index: number) => {
    GridItem() {
        Stack({ alignContent: Alignment.TopEnd }) {
            Image(item.url)
                .width(95)
                .height(95)
                .borderRadius(4)
                .onClick(() => {
                    this.index = index
                    this.preview.open()
                })
            Image($r('app.media.ic_btn_delete')).width(30).height(30)
                .margin({ right: 15, bottom: 10 })
        }
    }
})
```

2. 注册点击事件-处理数据更新

```
Image($r('app.media.ic_btn_delete')).width(30).height(30)
    .onClick(() => {
        this.imgList.splice(index, 1) // 移除索引
        this.onListChange(this.imgList) // 通知父组件更新
    })
```

:::info

测试并提交代码

...

## 加载进度遮罩

### basic模块

- 封装一个HmLoading的组件，用过弹层

```
@CustomDialog
@Component
@Preview
struct HmLoading {
    controller: CustomDialogController
    title: string = '数据处理中..'

    build() {
        Row({ space: 6 }) {
            Text(this.title)
                .fontSize(16)
                .fontColor($r("app.color.text_primary"))
            LoadingProgress()
                .width(30)
                .height(30)
                .color($r("app.color.primary"))
        }
        .justifyContent(FlexAlign.Center)
        .width('100%')
        .height('100%')
        .backgroundColor("rgba(0,0,0,0.1)")
    }
}

export { HmLoading }
```

- 在TaskDetail中定义loading弹出层的对象

```
loading: CustomDialogController = new CustomDialogController({
    builder: HmLoading(),
    customStyle: true
})
```

- 加载前后打开和关闭

```
async getTaskDetail(id: string) {  
  this.layer.open()  
  this.taskDetailData = await getTaskDetail(id)  
  this.layer.close()  
}
```

- 提货前后打开和关闭弹窗

```
async toPickUp () {  
  this.loading.open()  
  const cargoPictureList = await UploadFile(this.taskDetailData.cargoPictureList)  
  const cargoPickUpPictureList = await  
UploadFile(this.taskDetailData.cargoPickUpPictureList)  
  await pickUp(new PickupParamsModel({  
    id: this.taskDetailData.id,  
    cargoPickUpPictureList: cargoPictureList,  
    cargoPictureList: cargoPickUpPictureList  
  })))  
  this.getTaskDetail(this.taskDetailData.id) // 重新加载数据  
  this.scroller.scrollToEdge(Edge.Top)  
  this.loading.close()  
  promptAction.showToast({ message: '提货成功' })  
}
```

...success

提交代码

...

## 交货列表加载

分析：

一个任务，如果已经提货，那么下一步应该司机去送货，完成交货，上一节我们完成了提货，那么完成的这个任务应该在在途里面，所以我来先把在途的数据进行查询一下





无在途任务



任务



消息



我的

- 在途和待提货的区别就是查询的状态不同，刚刚好，我们前面定义了枚举，所以这里可以直接复用前面提货的TaskList的组件

```

build() {
    Stack({ alignContent: Alignment.Top }) {
        Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex, controller:
this.tabController }) {
            ForEach(this.tabsData, (item: TabClass) => {
                TabContent(){
                    if(item.name === "waiting") {
                        TaskList()
                    }
                    else if(item.name === 'line') {
                        TaskList({ queryParams: new TaskListParamsModel({
                            page: 1,
                            pageSize: 5,
                            status: TaskTypeEnum.Line
                        }) as TaskListParams })
                    }
                    else {
                        Text(item.title)
                    }
                }
            })
        }.backgroundColor($r('app.color.background_page')).animationDuration(300)
        Row ({ space: 30 }) {
            ForEach(this.tabsData, (item: TabClass) => {
                this.getTabBar(item)
            })
        }
        .padding({
            left: 40,
            right: 40
        })
        .width('100%')
        .height(50)
        .backgroundColor($r("app.color.white"))
    }
}

```

因为我们TaskList默认是查的提货的数据-所以不用传参数了，传了和原来的参数也是一样的



发现List只有一条数据时，List的内容是位于容器的中间位置，可以给TaskList中的HmList再加一层容器，设置高度为100%

```
build() {  
  HmList({  
    onLoad: async () => {  
      await this.getTaskList(true)  
    },  
    onRefresh: async () => {  
      await this.onRefresh()  
    },  
    dataSource: $taskListData,  
    renderItem: this.renderItem,  
    finished: this.allPage < this.queryParams.page,  
    finishText: '没啦没啦',  
    loadingText: '拼命加载中'  
  }).height('100%')
```

}

- 实现效果



接下来，根据状态去调整按钮的显示和文本内容

- 当任务的状态为待提货并且可提货时，按钮显示提货并且可用
- 当任务的状态为待交货时，按钮显示提货并且可用
- 当任务的状态为待回车登记时，按钮显示回车登记并且可用
- 给之前的任务状态枚举再加一个属性



```
export enum TaskTypeEnum {
  Waiting = 1, // 待提货
  Line = 2, // 在途待交货
  Delivered = 4, // 已交货待回车登记
  Finish = 6 // 已完成
}
```

- 在TaskItemCard组件中根据状态获取按钮的文本及可用状态

如果enablePickup为true 表示可提货

如果status 为 2 表示为待交货 显示交货

如果status 为 4表示 待回车登记

```
// 获取按钮可用性
getBtnEnable() {
  const value = this.taskItemData.status.toString()
  if (this.taskItemData.enablePickUp) {
    return true
  }
  switch (value) {
    case TaskTypeEnum.Line.toString():
    case TaskTypeEnum.Delivered.toString():
      return true
    default:
      return false
  }
}

// 获取按钮显示文本
getBtnText() {
  const value = this.taskItemData.status.toString()
  switch (value) {
    case TaskTypeEnum.Waiting.toString():
      return "提货"
    case TaskTypeEnum.Line.toString():
      return "交货"
    case TaskTypeEnum.Delivered.toString():
      return "回车登记"
    default:
      return ""
  }
}
```

- 按钮显示文本

```
Button(this.getBtnText(), { type: ButtonType.Capsule })
    .backgroundColor(this.getBtnEnable() ? $r('app.color.primary') :
$r('app.color.primary_disabled'))
    .fontColor($r("app.color.white"))
    .fontSize(14)
    .height(32)
    .enabled(this.getBtnEnable())
    .onClick(() => {
        this.toPickUp()
    })
```

⚠：这里的去提货的逻辑不用发生任何变化，因为交货的业务也是到详情页去提货，完成可以复用一模一样的逻辑



提交代码

## 交货详情内容控制

1. 根据状态显示不同的按钮

|                                              |                                 |      |    |
|----------------------------------------------|---------------------------------|------|----|
| <code>startProvince</code>                   | <code>string</code>             | 起始省份 | 必需 |
| <code>status</code>                          | <code>integer</code>            | 作业状态 | 必需 |
| 1为待提货)、2为在途)、3为改派)、4为已交付)、5为已作废、6为已完成(已回车登记) |                                 |      |    |
| <code>&gt; certificatePictureList</code>     | <code>array [object {1}]</code> | 回单凭证 | 必需 |
| <code>&gt; exceptionList</code>              | <code>array [object {6}]</code> |      | 必需 |

:::info

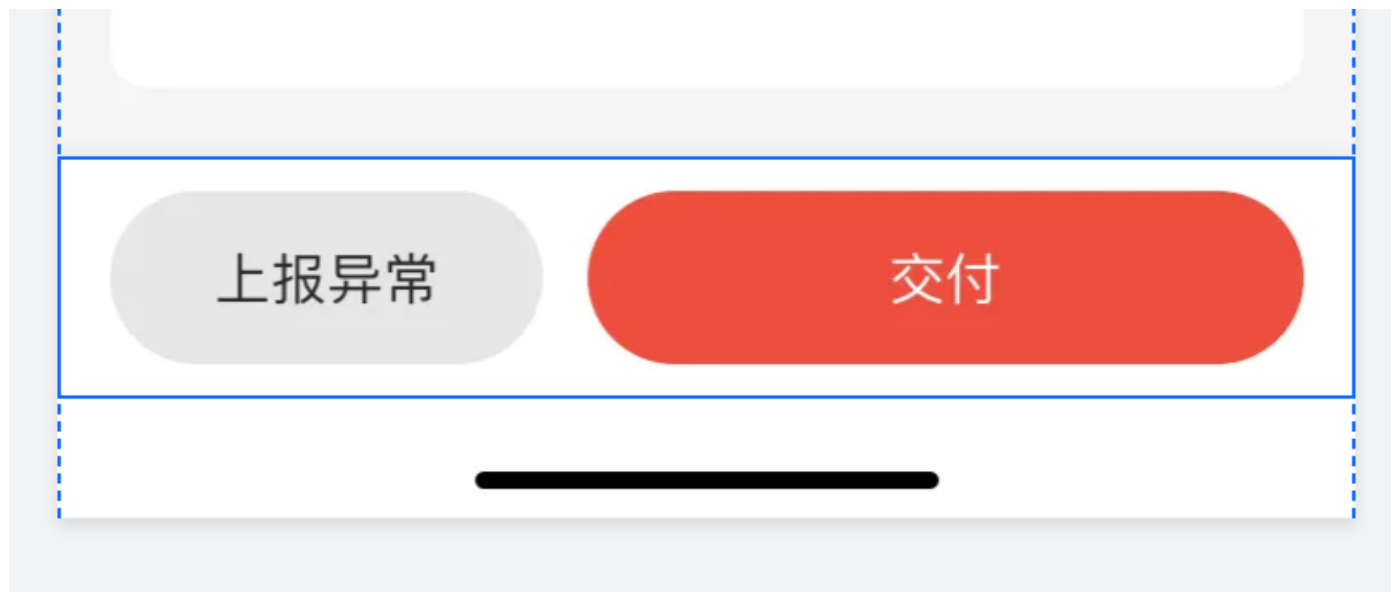
当一个任务提完货后，status应该是Line(在途)，如果是Waiting(待提货)的话显示提货按钮，如果是Line（在途），显示交货按钮

:::

#### 1. 提货



#### 2. 交货



// 底部按钮结构

```
@Builder
getBottomBtn() {
  //已完成不显示任何按钮
  Row() {
    if(this.taskDetailData.status === TaskTypeEnum.Waiting) {
      Button("延迟收货", { type: ButtonType.Capsule })
        .backgroundColor($r('app.color.btn_gray'))
        .fontColor($r('app.color.text_primary'))
        .fontSize(16)
    }
  }
}
```

```

        .height(50)
        .width(125)
        Button("提货", { type: ButtonType.Capsule })
            .backgroundColor($r('app.color.primary'))
            .fontColor($r('app.color.white'))
            .height(50)
            .flexGrow(1)
            .margin({ left: 13 })
            .enabled(this.getPickUpState())
            .onClick(() => {
                this.btnPickUp()
            })
    }
    else if(this.taskDetailData.status === TaskTypeEnum.Line) {
        Button("上报异常", { type: ButtonType.Capsule })
            .backgroundColor($r('app.color.btn_gray'))
            .fontColor($r('app.color.text_primary'))
            .fontSize(16)
            .height(50)
            .width(125)
        Button("交货", { type: ButtonType.Capsule })
            .backgroundColor($r('app.color.primary'))
            .fontColor($r('app.color.white'))
            .height(50)
            .flexGrow(1)
            .margin({ left: 13 })

    }

}

}
.width('100%')
.padding({ left: 15, right: 15 })
.height(70)
.justifyContent(FlexAlign.SpaceBetween)
.alignItems(VerticalAlign.Center)
.backgroundColor($r('app.color.white'))
}

```

2. 交货时，隐藏提货上传组件，显示交货上传组件



- 实现一个Builder函数来放置交货的上传

// 专门负责交货信息的结构

```
@Builder
getDeliverContent() {
  HmUpload({
    title: '请拍照上传交货凭证',
    imgList: this.taskDetailData.deliverPictureList || [],
    onListChange: (list: ImageList[]) => {
      this.taskDetailData.deliverPictureList = list
    }
  })
  HmUpload({
    title: '请拍照上传交货货品照片',
    imgList: this.taskDetailData.certificatePictureList || [],
    onListChange: (list: ImageList[]) => {
      this.taskDetailData.certificatePictureList = list
    }
  })
}
```

```
// 提货时显示提货照片
    if(this.taskDetailData.status === TaskTypeEnum.Waiting) {
        HmToggleCard({ title: '提货信息' }) {
            this.getPickUpContent()
        }
    }
    // 交货时显示交货照片
    if(this.taskDetailData.status === TaskTypeEnum.Line) {
        HmToggleCard({ title: '交货信息' }) {
            this.getDeliverContent()
        }
    }
}
```

:::info  
提交代码  
...

## 交货状态控制

```
// 获取交货状态
getDeliverState () {
    return this.taskDetailData.deliverPictureList?.length > 0 &&
        this.taskDetailData.certificatePictureList?.length > 0
        && this.taskDetailData.deliverPictureList.every(item => !!item.url)
        && this.taskDetailData.certificatePictureList.every(item => !!item.url)
}
```

```
Button("交付", { type: ButtonType.Capsule })
    .backgroundColor(this.getDeliverState() ? $r('app.color.primary') :
$r('app.color.primary_disabled'))
    .fontColor($r('app.color.white'))
    .height(50)
    .flexGrow(1)
    .margin({ left: 13 })
    .enabled(this.getDeliverState())
```



:::info  
提交代码  
...

## 交货

### Entry模块

:::info  
标准流程

1. 定义类型封装api
2. 引入api
3. 注册事件-调用api重新加载  
...
4. 定义请求参数类型

```
import { ImageList } from '@hm/basic'
export interface DeliverParamsType {
  /** 交付凭证列表 */
  certificatePictureList: ImageList[];
  /** 交付图片列表 */
  deliverPictureList: ImageList[];
  /** 司机作业id */
  id: string;
}
export class DeliverParamsTypeModel implements DeliverParamsType {
  certificatePictureList: ImageList[] = []
```



```

deliverPictureList: ImageList[] = []
id: string = ''

constructor(model: DeliverParamsType) {
  this.certificatePictureList = model.certificatePictureList
  this.deliverPictureList = model.deliverPictureList
  this.id = model.id
}
}

```

- 在models/index.ets统一导出

```
export * from './deliver'
```

## 2. 封装API

```

// 交货
export const deliver = (data: DeliverParamsTypeModel) => {
  return Request.post<null>("/tasks/deliver", data)
}

```

## 3. 调用

```

// 交货方法
async btnDeliver() {
  this.loading.open()
  this.taskDetailData.deliverPictureList = await
UploadFile(this.taskDetailData.deliverPictureList)
  this.taskDetailData.certificatePictureList = await
UploadFile(this.taskDetailData.certificatePictureList)
  await deliver(new DeliverParamsTypeModel({
    id: this.taskDetailData.id,
    certificatePictureList: this.taskDetailData.certificatePictureList,
    deliverPictureList: this.taskDetailData.deliverPictureList
  }))
  promptAction.showToast({
    message: '交货成功'
  })
  // 重新加载一下
  this.getTaskDetail(this.taskDetailData.id)
  this.loading.close()
}

```

```

        Button("交付", { type: ButtonType.Capsule })
            .backgroundColor(this.getDeliverState() ? $r('app.color.primary') :
$r('app.color.primary_disabled'))
            .fontColor($r('app.color.white'))
            .height(50)
            .flexGrow(1)
            .enabled(this.getDeliverState())
            .margin({ left: 13 }).onClick(() => {
                this.btnDeliver()
            })

```

:::info

提交

...

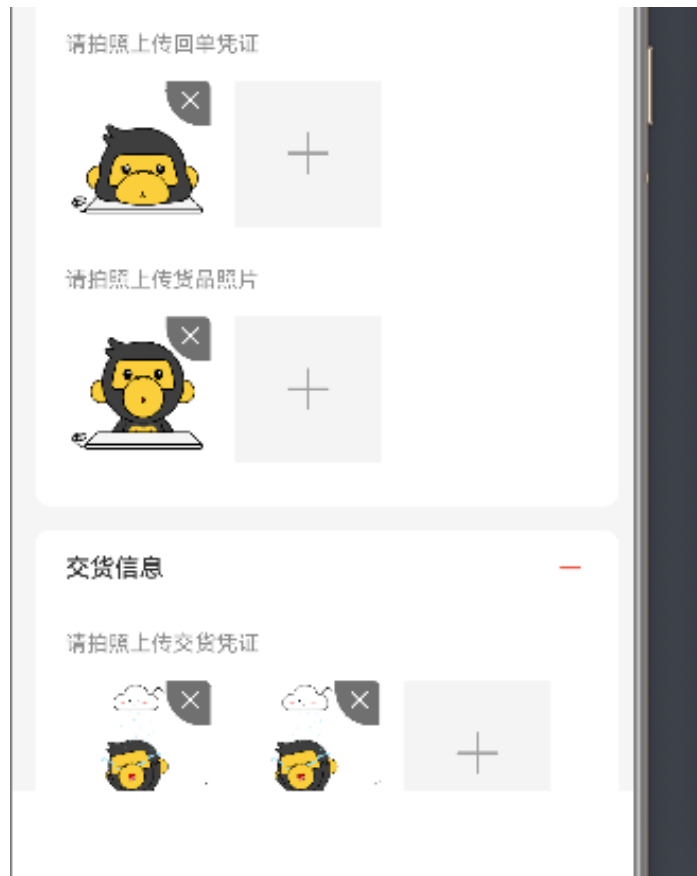
## 底部显示回车登记按钮

在已经交货的情况下，显示提货信息-交货信息- 和底部的回车登记按钮

```

// 提货时显示提货照片
    if(this.taskDetailData.status === TaskTypeEnum.Waiting ||
this.taskDetailData.status === TaskTypeEnum.Delivered) {
        HmToggleCard({ title: '提货信息' }) {
            this.getPickUpContent()
        }
    }
// 交货时显示交货照片
    if(this.taskDetailData.status === TaskTypeEnum.Line ||
this.taskDetailData.status === TaskTypeEnum.Delivered) {
        HmToggleCard({ title: '交货信息' }) {
            this.getDeliverContent()
        }
    }
}

```



- 显示底部回车登记按钮

```
else if(this.taskDetailData.status === TaskTypeEnum.Delivered) {
  Row() {
    // 已交付显示回车登记
    Button("回车登记", { type: ButtonType.Capsule })
      .backgroundColor($r('app.color.primary'))
      .fontColor($r('app.color.white'))
      .height(50)
      .width('80%')
  }.width('100%').justifyContent(FlexAlign.Center)
}
```



:::info  
提交代码  
:::

## 回车登记模式下-上传组件只读

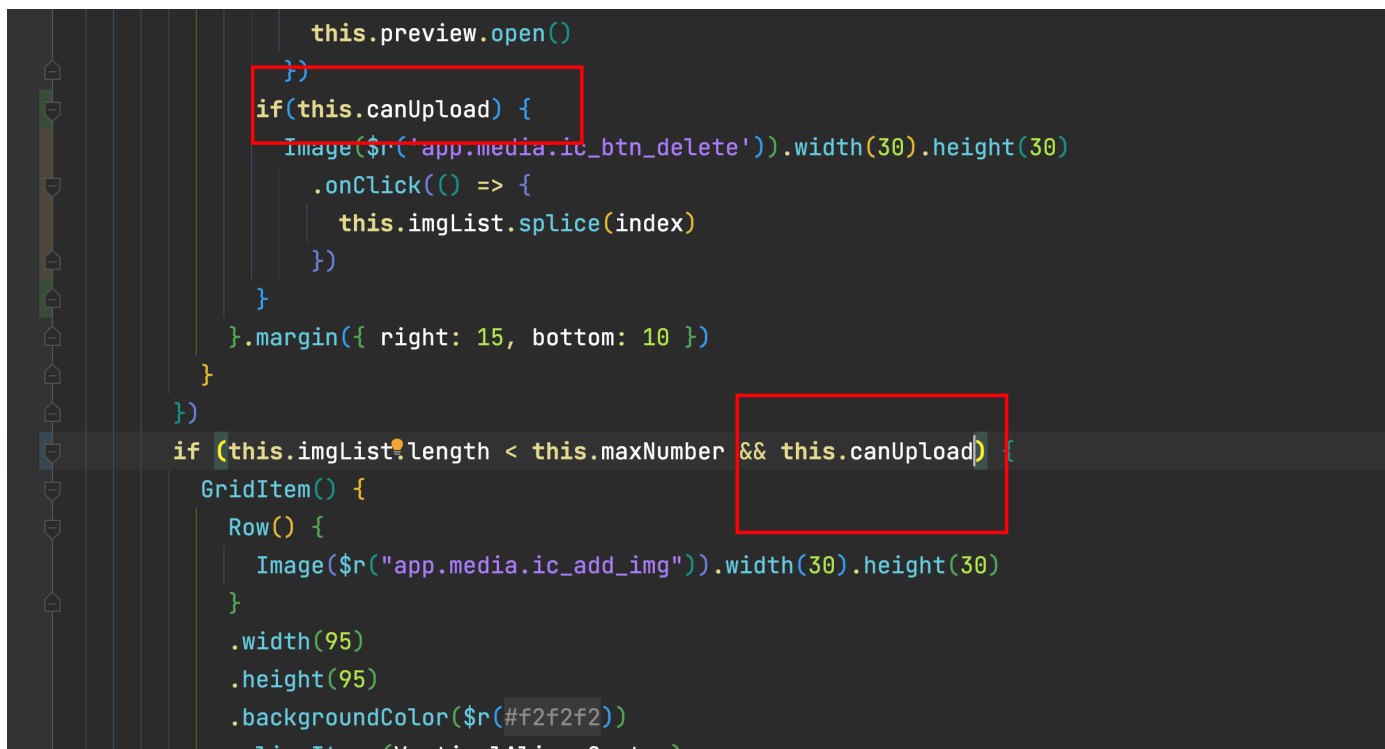
:::info  
因为货已经交了，所以回车登记模式下，此时只可以看，不能再传图片了  
:::

### basic模块

1. 给HmUpload一个属性来控制是否可上传和删除

```
@Prop
canUpload:boolean = true
```

2. 通过该属性来控制上传部分的显示和隐藏



```
        this.preview.open()
    })
    if(this.canUpload) {
        Image($r("app.media.ic_btn_delete")).width(30).height(30)
            .onClick(() => {
                this.imgList.splice(index)
            })
    }
    }.margin({ right: 15, bottom: 10 })
}
})
if (this.imgList.length < this.maxNumber && this.canUpload) {
    GridItem() {
        Row() {
            Image($r("app.media.ic_add_img")).width(30).height(30)
        }
        .width(95)
        .height(95)
        .backgroundColor($r("#f2f2f2"))
        .clipItems(VerticalAlign.Center)
```

3. 在TaskDetail中传入该属性

```
// 专门负责提货信息的结构
@Builder
getPickUpContent() {
    HmUpload({
        title: '请拍照上传货品凭证',
        canUpload: this.taskDetailData.status === TaskTypeEnum.Waiting,
        imgList: this.taskDetailData.cargoPickUpPictureList || [],
        onListChange: (list: ImageList[]) => {
            this.taskDetailData.cargoPickUpPictureList = list
        }
    })
}
```

```

    }
  })
  HmUpload({
    title: '请拍照上传货品照片',
    canUpload: this.taskDetailData.status === TaskTypeEnum.Waiting,
    imgList: this.taskDetailData.cargoPictureList || [],
    onListChange: (list: ImageList[]) => {
      this.taskDetailData.cargoPictureList = list
    }
  })
}

```

// 专门负责交货信息的结构

```

@Builder
getDeliverContent() {
  HmUpload({
    title: '请拍照上传交货货品凭证',
    canUpload: this.taskDetailData.status === TaskTypeEnum.Line,
    imgList: this.taskDetailData.deliverPictureList || [],
    onListChange: (list: ImageList[]) => {
      this.taskDetailData.deliverPictureList = list
    }
  })
  HmUpload({
    title: '请拍照上传交货货品照片',
    canUpload: this.taskDetailData.status === TaskTypeEnum.Line,
    imgList: this.taskDetailData.certificatePictureList || [],
    onListChange: (list: ImageList[]) => {
      this.taskDetailData.certificatePictureList = list
    }
  })
}

```



:::info  
提交代码  
:::

## 回车登记页面

### entry模块

- 声明回车登记类型

```
import { ImageList } from '@hm/basic'
export interface CarRecordType {
  /** 事故说明，类型为“其他”时填写 */
  accidentDescription: string | null;
  /** 事故图片列表 */
}
```

```

    accidentImagesList: ImageList[] | null;
    /** 事故类型, 1-直行事故, 2-追尾事故, 3-超车事故, 4-左转弯事故, 5-右转弯事故, 6-弯道事故, 7-坡道事故,
    8-会车事故, 9-其他, */
    accidentType: string | null;
    /** 违章说明, 类型为“其他”时填写 */
    breakRulesDescription: string | null;
    /** 违章类型, 1-闯红灯, 2-无证驾驶, 3-超载, 4-酒后驾驶, 5-超速行驶, 6-其他,可用 */
    breakRulesType: string | null;
    /** 扣分数数据 */
    deductPoints: number | null;
    /** 回车时间, 回车时间, 格式yyyy-MM-dd HH:mm:ss, 比如: 2023-07-18 17:00:00 */
    endTime: string;
    /** 故障说明, 类型为“其他”时填写 */
    faultDescription: string | null;
    /** 故障图片列表 */
    faultImagesList: ImageList[] | null;
    /** 故障类型, 1-发动机启动困难, 2-不着车, 3-漏油, 4-漏水, 5-照明失灵, 6-有异响, 7-排烟异常, 8-温度异常,
    9-其他,可用 */
    faultType: string | null;
    /** 运输任务id */
    id: string;
    /** 是否出现事故 */
    isAccident: boolean | null;
    /** 车辆是否可用 */
    isAvailable: boolean | null;
    /** 车辆是否违章 */
    isBreakRules: boolean | null;
    /** 车辆是否故障 */
    isFault: boolean | null;
    /** 罚款金额 */
    penaltyAmount: string | null;
    /** 出车时间, 出车时间, 格式yyyy-MM-dd HH:mm:ss, 比如: 2023-07-18 17:00:00 */
    startTime: string;
}

export class CarRecordTypeModel implements CarRecordType {
    accidentDescription: string | null = null
    accidentImagesList: ImageList[] | null = null
    accidentType: string | null = null
    breakRulesDescription: string | null = null
    breakRulesType: string | null = null
    deductPoints: number | null = null
    endTime: string = ''
    faultDescription: string | null = null
    faultImagesList: ImageList[] | null = null
    faultType: string | null = null
    id: string = ''
    isAccident: boolean | null = null
    isAvailable: boolean | null = null
    isBreakRules: boolean | null = null
    isFault: boolean | null = null
    penaltyAmount: string | null = null
    startTime: string = ''

```

```

constructor(model: CarRecordType) {
  this.accidentDescription = model.accidentDescription
  this.accidentImagesList = model.accidentImagesList
  this.accidentType = model.accidentType
  this.breakRulesDescription = model.breakRulesDescription
  this.breakRulesType = model.breakRulesType
  this.deductPoints = model.deductPoints
  this.endTime = model.endTime
  this.faultDescription = model.faultDescription
  this.faultImagesList = model.faultImagesList
  this.faultType = model.faultType
  this.id = model.id
  this.isAccident = model.isAccident
  this.isAvailable = model.isAvailable
  this.isBreakRules = model.isBreakRules
  this.isFault = model.isFault
  this.penaltyAmount = model.penaltyAmount
  this.startTime = model.startTime
}
}

```

- 在models/index.ets中导出

```
export * from './car_record'
```

- 新建回车登记页面 pages/CardRecord/Record



9:41

<

回车登记

出车时间

2022.05.04 13:00

回车时间

请选择 >

车辆违章

☐

是

☒

否

车辆故障

☐

是

☒

否

车辆事故

☐

是

☒

否

交车

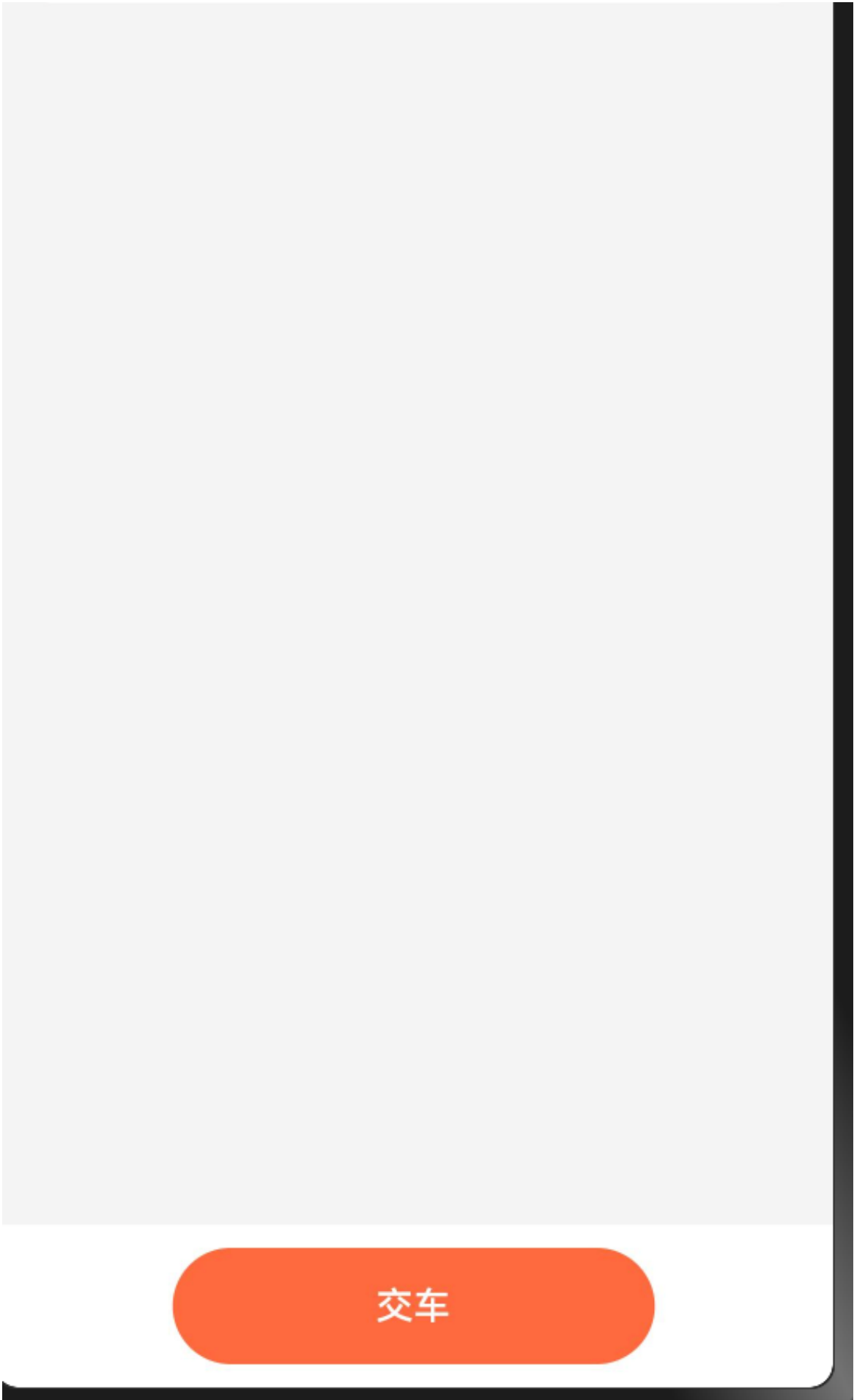
```
import { HmNavBar, HmCard, HmCardItem } from '@hm/basic'
@Entry
@Component
struct CarRecord {
  build() {
```

```

Column() {
  HmNavBar({ title: '回车登记' })
  Scroll() {
    Column() {
      HmCard(){
        HmCardItem({
          leftTitle: '出车时间',
          rightText: '2022.05.04 13:00'
        })
        HmCardItem({
          leftTitle: '回车时间',
          rightText: '请选择',
          showBottomBorder: false
        })
      }
    }
    .height('100%')
  }
  .layoutWeight(1)
  // 底部内容
  Row() {
    Button("交车",{ type: ButtonType.Capsule })
      .backgroundColor($r('app.color.primary'))
      .width(207)
      .height(50)
  }
  .backgroundColor($r('app.color.white'))
  .height(70)
  .width('100%')
  .justifyContent(FlexAlign.Center)
  .alignItems(VerticalAlign.Center)
}
.backgroundColor($r('app.color.background_page'))
.height('100%')
}
}

```





- TaskDetail页面跳转到回车登记

```

Button("回车登记", { type: ButtonType.Capsule })
    .backgroundColor($r('app.color.primary'))
    .fontColor($r('app.color.white'))
    .height(50)
    .width('80%')
    .onClick(() => {
        router.pushUrl({
            url: 'pages/CarRecord/CarRecord',
            params: {
                id: this.taskDetailData.id
            }
        })
    })
})

```

- 在CarRecord接收参数id，并且获取任务的详情，赋值出车时间

```

@State
taskDetailData: TaskDetailInfoModel = new TaskDetailInfoModel({} as TaskDetailInfo)
aboutToAppear() {
    const params = router.getParams() as CommonRouterParams
    if(params && params.id) {
        this.getTaskDetail(params.id)
    }
}
async getTaskDetail (id: string) {
    this.taskDetailData = await getTaskDetail(id)
}

```

- 赋值出车时间

```

HmCardItem({
    leftTitle: '出车时间',
    rightText: this.taskDetailData.actualDepartureTime
})

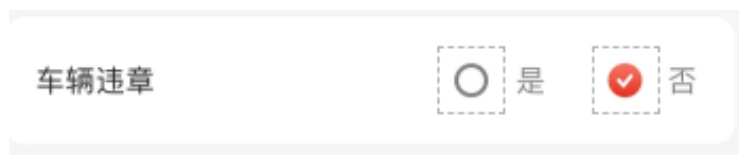
```



:::info  
提交代码  
:::

## 封装选择组件-HmCheckBox

### basic模块



### 分析

- 可以控制显示文本
  - 右侧的是否文本可以控制
  - 值改变时通知外部组件
  - 这个组件通用性不是那么大，简单封装一下
- 在components下新建HmCheckBox.ets

```
@Preview
@Component
struct HmCheckBox {
    title: string = "测试"
    confirmText: string = "是"
    cancelText: string = '否'
    @Prop
    value: boolean = true // 决定左侧还是右侧\
    checkChange: (value: boolean) => void = () => {}
    build() {
        Row () {
            Row () {
                Text(this.title)
                    .fontSize(14)
                    .fontColor($r("app.color.text_primary"))
```

```
// 右侧内容
Row ({ space: 10 }) {
  Row () {
    Image(this.value ? $r("app.media.ic_radio_true") :
$r("app.media.ic_radio_false"))
      .width(32)
      .aspectRatio(1)
    Text(this.confirmText)
  }
  .onClick(() => {
    this.value = true
    this.checkChange(this.value)
  })
  Row () {
    Image(!this.value ? $r("app.media.ic_radio_true") :
$r("app.media.ic_radio_false"))
      .width(32)
      .aspectRatio(1)
    Text(this.cancelText)
  }
  .onClick(() => {
    this.value = false
    this.checkChange(this.value)
  })
}
}
.width("100%")
.borderRadius(10)
.height(60)
.padding({
  left: 15,
  right: 15
})
.justifyContent(FlexAlign.SpaceBetween)
.backgroundColor($r("app.color.white"))
}
.width("100%")
.padding({
  left: 15,
  right: 15
})
.margin({
  top: 15
})
}
}
export { HmCheckBox }
```

- 在components/index.ets中导出

```
export * from './HmCheckBox'
```

- 在回车登记中放置三个组件

```
HmCheckBox({ value: false, title: '车辆违章' })  
HmCheckBox({ value: false, title: '车辆故障' })  
HmCheckBox({ value: false, title: '车辆事故' })
```

entry模块



## 回车登记

出车时间



回车时间

请选择 >

车辆违章



是



否

车辆故障



是



否

车辆事故



是



否

提交代码

## 使用官方的时间弹窗选择器来选择时间

- 使用官方的选择时间弹窗



```
HmCardItem({ leftTitle: '回车时间',
  rightText: "请选择", showBottomBorder: false,
  onRightClick: () => {
    DatePickerDialog.show({
      showTime: true,
      useMilitaryTime: true,
      onDateAccept: (value: Date) => {

      }
    })
  }
})
```

<

回车登记

出车时间

回车时间

请选择 >

车辆违章

☐ 是 ☒ 否

车辆故障

☐ 是 ☒ 否

车辆事故

☐ 是 ☒ 否

2024年4月27日星期六 ▾

|       |    |    |
|-------|----|----|
| 4月25日 | 15 | 48 |
| 4月26日 | 16 | 49 |
| 4月27日 | 17 | 50 |
| 4月28日 | 18 | 51 |
| 4月29日 | 19 | 52 |

取消

确定

## 回车登记提交

- 定义提交数据类型
- 绑定数据到数据
- 提交回车登记

- 在CarRecord中定义数据

```
@State
carRecord: CarRecordTypeModel = new CarRecordTypeModel({} as CarRecordType)
```

- 手写转时间格式的方法

```
transTimeFormat(value: Date) {
    // 2023-12-23 05:12
    return value.getFullYear() + "-" + (value.getMonth() + 1).toString().padStart(2, "0")
+ '-'
    + value.getDate().toString().padStart(2, "0") + " " +
    value.getHours().toString().padStart(2, "0") + ":"
    + value.getMinutes().toString().padStart(2, "0")
}
```

- 回车时间绑定对应的字段

```
HmCardItem({
    leftTitle: '回车时间',
    rightText: this.carRecord.endTime || "请选择", showBottomBorder: false,
    onRightClick: () => {
        DatePickerDialog.show({
            showTime: true,
            useMilitaryTime: true,
            onDateAccept: (value: Date) => {
                // 结束时间
                this.carRecord.endTime = this.transTimeFormat(value)
            }
        })
    }
})
```

- 绑定车辆违章-车辆故障-车辆事故的属性

```
HmCheckBox({ value: !!this.carRecord.isBreakRules, title: '车辆违章',
    checkChange: (value) => {
        this.carRecord.isBreakRules = value
    } })
HmCheckBox({ value: !!this.carRecord.isFault, title: '车辆故障',
    checkChange: (value) => {
        this.carRecord.isFault = value
    } })
HmCheckBox({ value: !!this.carRecord.isAccident, title: '车辆事故',
    checkChange: (value) => {
        this.carRecord.isAccident = value
    } })
```

- 封装交车api

```
export const carRecord = (data: CarRecordTypeModel) => {  
  return Request.post<null>("/tasks/truckRegistration", data)  
}
```

- 点击交车调用接口

**id** string 运输任务id 必需

**startTime** string 出车时间 必需

出车时间, 格式yyyy-MM-dd HH:mm:ss, 比如: 2023-07-18 17:00:00

**endTime** string 回车时间 必需

回车时间, 格式yyyy-MM-dd HH:mm:ss, 比如: 2023-07-18 17:00:00

有三个字段是必须要传的, id来自任务详情中的transportTaskId, startTime来自任务详情中的actualDepartureTime, endTime是我们自己选择的时间

- 提交

```
// 检查再提交  
async btnCarRecord() {  
  if (!this.carRecord.endTime) {  
    promptAction.showToast({ message: '请选择回车时间' })  
    return  
  }  
  this.carRecord.startTime = this.taskDetailData.actualDepartureTime  
  // 运输任务id  
  this.carRecord.id = this.taskDetailData.transportTaskId // 注意注意再注意  
  await carRecord(this.carRecord)  
  promptAction.showToast({ message: '回车登记成功' })  
  router.clear() // 清空页面栈  
  router.replaceUrl({ url: 'pages/Index/Index' })  
}
```

同学们到现在, 我们已经将神领物流中的 提货-交货-回车登记业务跑通, 剩下的将完成 对整个项目的一些核心点的优化

## 实现延迟收货业务-基本布局

entry模块

9:41



延迟提货

原定时间

2022.05.04 13:00

延迟时间

不可超过2个小时 >

请输入延迟提货原因

0/50

提交

## 1. 新建pages/Delay/Delay.ets -(Page)

- 快速布局

```
import { HmNavBar, HmCard, HmCardItem } from '@hm/basic'
@Entry
@Component
```

```

struct Delay {
  build() {
    Column() {
      HmNavBar({ title: '延迟提货' })
      HmCard(){
        HmCardItem({ leftTitle: '原定时间', rightText: '', showRightIcon: false })
        HmCardItem({ leftTitle: '延迟时间', rightText: '' })
        TextArea({ placeholder: '请输入延迟提货原因'
      })
    }.backgroundColor($r('app.color.background_page')).margin({
      top: 20

    }).borderRadius(8).height(130).placeholderColor($r('app.color.text_secondary')).fontSize(14)

      Text('0/50').margin({
        top: -30
      }).textAlign(TextAlign.End).width('100%').padding({ right: 15
    }).fontColor($r('app.color.text_secondary'))
    Row() {
      Button("提交").height(50).width(207).
        backgroundColor($r('app.color.primary_disabled'))
    }.justifyContent(FlexAlign.Center).padding({
      top: 20,
      bottom: 20
    })
  }
}
}.height('100%').backgroundColor($r('app.color.background_page'))
}
}

```

:::info  
 提交代码  
 :::

## 延迟收货-接收参数-显示

:::info  
 TaskDeail中点击延迟收货跳转到延迟收货，传入参数  
 :::

1. 点击延迟收货，跳转，传入参数

这里偷个懒直接把要的参数id和时间传递过去，利用原来封装的公共路由参数

**basic**模块

```

export class CommonRouterParams {
  id?: string = ''
  oldTime?: string = ''
}

```

```
Button("延迟收货", { type: ButtonType.Capsule })
    .backgroundColor($r('app.color.btn_gray'))
    .fontColor($r('app.color.text_primary'))
    .fontSize(16)
    .height(50)
    .width(125)
    .onClick(() => {
        router.pushUrl({
            url: 'pages/Delay/Delay',
            params: {
                id: this.taskDetailData.id,
                oldTime: this.taskDetailData.planDepartureTime
            }
        })
    })
})
```

## 2. 在延迟收货接收该参数

- 定义延迟提货提交类型参数

```
export interface DelayParamsType {
    /** 延迟原因 */
    delayReason: string;
    /** 延迟时间, 格式: yyyy-MM-dd HH:mm */
    delayTime: string;
    /** 司机作业单id */
    id: string;
}
export class DelayParamsTypeModel implements DelayParamsType {
    delayReason: string = ''
    delayTime: string = ''
    id: string = ''

    constructor(model: DelayParamsType) {
        this.delayReason = model.delayReason
        this.delayTime = model.delayTime
        this.id = model.id
    }
}
```

- 在models/index.ets中导出

```
export * from './delay'
```

- 接收赋值id和原有时间

```

@State
delayForm: DelayParamsTypeModel = new DelayParamsTypeModel({} as DelayParamsType)
@State
oldTime: string = ""
aboutToAppear() {
  const params = router.getParams() as CommonRouterParams
  if(params && params.id && params.oldTime) {
    this.delayForm.id = params.id
    this.oldTime = params.oldTime
  }
}

```

- 赋值原有时间

```
HmCardItem({ leftTitle: '原定时间', rightText: this.oldTime, showRightIcon: false })
```

<

延迟提货

原定时间

2023-11-17 20:00:00

延迟时间

>

请输入延迟提货原因

0/50

提交

:::info  
 提交代码  
 ...



# 延迟收货-日期选择

- 在basic模块新建一个公共的时间格式化工具

```
export const DateFormat = (value: Date) => {  
  // 2023-12-23 05:12  
  return value.getFullYear() + "-" + (value.getMonth() + 1).toString().padStart(2, "0") +  
    '-' +  
    + value.getDate().toString().padStart(2, "0") + " " +  
    value.getHours().toString().padStart(2, "0") + ":" +  
    + value.getMinutes().toString().padStart(2, "0")  
}
```

- 在utils/index.ets导出

```
export * from './format'
```

- 右侧点击弹出

```
HmCardItem({  
  leftTitle: '延迟时间', rightText: this.delayForm.delayTime || '',  
  onRightClick: () => {  
    DatePickerDialog.show({  
      useMilitaryTime: true,  
      showTime: true,  
      onDateAccept: (value) => {  
        this.delayForm.delayTime = DateFormat(value)  
      }  
    })  
  })  
})
```



:::info  
提交代码  
:::

# 延迟收货-字数控制

1. 声明状态

```
@State
maxSizeNumber:number = 50
```

## 2. 双向绑定原因字段

:::warning

\$\$不支持嵌套对象里面的属性绑定

...

```
TextArea({ placeholder: '请输入延迟提货原因', text: this.delayForm.delayReason, controller:
this.controller })
.maxLength(this.maxSizeNumber)
```

## 3. 监听onChange事件

```
.onChange((value) => {
    this.delayForm.delayReason = value
})
```

## 4. 设置显示文本

```
Text(`${this.delayForm.delayReason?.length || 0}/${this.maxSizeNumber}`)
    .margin({
        top: -30
    })
    .textAlign(TextAlign.End)
    .width('100%')
    .padding({ right: 15 })
    .fontColor($r('app.color.text_secondary'))
```

延迟时间

>

ghhhhhhhhhhg  
ggggggggggggg

50/50

提交

:::info  
提交代码  
:::

## 控制按钮状态及颜色

### 1. 封装方法检查必填项

```
getBtnEnable() {  
    return !(this.delayForm.id && this.delayForm.delayReason && this.delayForm.delayTime)  
}
```

### 2. 控制按钮及颜色

```
Button("提交").height(50).width(207).backgroundColor(  
    this.getBtnEnable() ? $r('app.color.primary') :  
    $r('app.color.primary_disabled')  
) .enabled(this.getBtnEnable()).onClick(() => {  
  
    })
```

原定时间

2023-11-17 20:00:00

延迟时间

2023-12-04 03:15 >

tyl

2/50

提交

:::info  
提交代码  
:::

## 提交延迟收货

### 1. 封装api

```
// 延迟收货
export const delay = (data: DelayParamsTypeModel) => {
  return Request.put<null>("/tasks/delay", data)
}
```

### 2. 调用api, 返回页面

```
async delay() {
  await delay(this.delayForm)
  promptAction.showToast({
    message: '延迟收货成功'
  })
  router.back()
}
```

:::info  
提交代码  
:::

# 上报异常页面基础布局

解决接口异常 返回值的问题

```
async getTaskList(append: boolean) {  
  const result = await getTaskList(this.queryParams)  
  // 追加数据  
  // this.taskListData = this.taskListData.concat(result.items) // 拿到返回的数组  
  if (append) {  
    this.taskListData.push(...result.items || []) // 延展运算符的写法  
  } else {  
    this.taskListData = result.items // 直接赋值  
  }  
  
  this.allPage = result.pages // 总页数  
  this.queryParams.page++ // 下次请求的页码  
}
```

**items有可能为null**

**需要添加短路表达式**

entry模块



新建上报异常页面-pages/ExceptionReport/ExceptionReport.ets

```
import { HmCard, HmCardItem, HmNavBar, HmUpload } from '@hm/basic'

@Entry
@Component
struct ExceptionReport {
```

```

build() {
  Column() {
    HmNavBar({ title: '上报异常' })
    Scroll() {
      Column() {
        HmCard() {
          HmCardItem({ leftTitle: '异常时间', rightText: '请选择' })
          HmCardItem({ leftTitle: '上报位置', rightText: '请选择' })
          HmCardItem({ leftTitle: '异常类型', rightText: '请选择' })
          HmCardItem({ leftTitle: '异常描述', rightText: '', showRightIcon: false,
showBottomBorder: false })
          TextArea({
            placeholder: '请输入异常描述'

          }).height(130).borderRadius(8).placeholderColor($r('app.color.text_secondary')).fontSize(14)

          Text(`0/50`)
            .margin({
              top: -30
            })
            .textAlign(TextAlign.End)
            .width('100%')
            .padding({ right: 15 })
            .fontColor($r('app.color.text_secondary'))
          Row().height(20)

        }

        HmCard() {
          HmUpload({
            title: '上传图片(最多6张)',
            imgList: []
            , canUpload: true
          })
          Row().height(20)
        }
      }.padding({
        bottom: 80
      })
      .layoutWeight(1)

      Row() {
        Button("提交").height(50).width(207).backgroundColor($r('app.color.primary_disabled'))
      }
      .position({
        y: '100%'
      })
      .height(70)
      .translate({

```



```

        y: -70
    })
    .width('100%')
    .justifyContent(FlexAlign.Center)
    .backgroundColor($r('app.color.white'))
}
.height('100%').backgroundColor($r('app.color.background_page'))
}
}

```

:::info

提交代码

:::

## 上报异常页面选择时间

entry模块

- 点击上报异常进入-TaskDetail

```

Button("上报异常", { type: ButtonType.Capsule })
    .backgroundColor($r('app.color.btn_gray'))
    .fontColor($r('app.color.text_primary'))
    .fontSize(16)
    .height(50)
    .width(125).onClick(() => {
        router.pushUrl({
            url: 'pages/Exception/ExceptionReport',
            params: {
                id: this.taskDetailData.transportTaskId
            }
        })
    })
})

```

- 声明数据类型-新建models/exception.ets

```

import { ImageList } from '@hm/basic'
export interface ExceptionParamsType {
    /** 异常描述, 200字以内 */
    exceptionDescribe: string | null;
    /** 异常图片 */
    exceptionImagesList: ImageList[] | null;
    /** 上报异常位置 */
    exceptionPlace: string;
    /** 异常时间, 精确到分钟 */
    exceptionTime: string;
    /** 异常类型(传中文), 发动机启动困难, 不着车, 漏油, 漏水, 照明失灵, 有异响, 排烟异常, 温度异常, 其他 */
    exceptionType: string;
    /** 运输任务id */
    transportTaskId: string;
}

```

```

}
export class ExceptionParamsTypeModel implements ExceptionParamsType {
  exceptionDescribe: string | null = null
  exceptionImagesList: ImageList[] | null = null
  exceptionPlace: string = ''
  exceptionTime: string = ''
  exceptionType: string = ''
  transportTaskId: string = ''

  constructor(model: ExceptionParamsType) {
    this.exceptionDescribe = model.exceptionDescribe
    this.exceptionImagesList = model.exceptionImagesList
    this.exceptionPlace = model.exceptionPlace
    this.exceptionTime = model.exceptionTime
    this.exceptionType = model.exceptionType
    this.transportTaskId = model.transportTaskId
  }
}

```

- 在models/index.ets中导出

```
export * from './exception'
```

- 接收id

```

@State
exceptionForm: ExceptionParamsTypeModel = new ExceptionParamsTypeModel({} as
ExceptionParamsType)
aboutToAppear() {
  const params = router.getParams() as CommonRouterParams
  if(params && params.id) {
    this.exceptionForm.transportTaskId = params.id
  }
}

```

- 点击右侧弹出选择框

```
HmCardItem({
    leftTitle: '异常时间', rightText: this.exceptionForm.exceptionTime || '请选择',
    onRightClick: () => {
        DatePickerDialog.show({
            showTime: true,
            useMilitaryTime: true,
            onDateAccept: (value) => {
                this.exceptionForm.exceptionTime = DateFormat(value)
            }
        })
    }
})
```



## 上报异常

异常时间

请选择 >

上报位置

请选择 >

异常类型

请选择 >

异常描述

请输入异常描述

0/50

|      |      |    |
|------|------|----|
| 取消   | 选择时间 | 完成 |
| 月日   | 时    | 分  |
| 1月3日 | 08   | 53 |
| 1月4日 | 09   | 54 |
| 1月5日 | 10   | 55 |
| 1月6日 | 11   | 56 |
| 1月7日 | 12   | 57 |

:::info  
提交代码  
:::

## 封装HmSelectCard组件

basic模块

异常类型

发动机启动困难

选择类型



发动机启动困难



不着车，漏油



照明失灵



有异常响动



排烟异常、温度异常



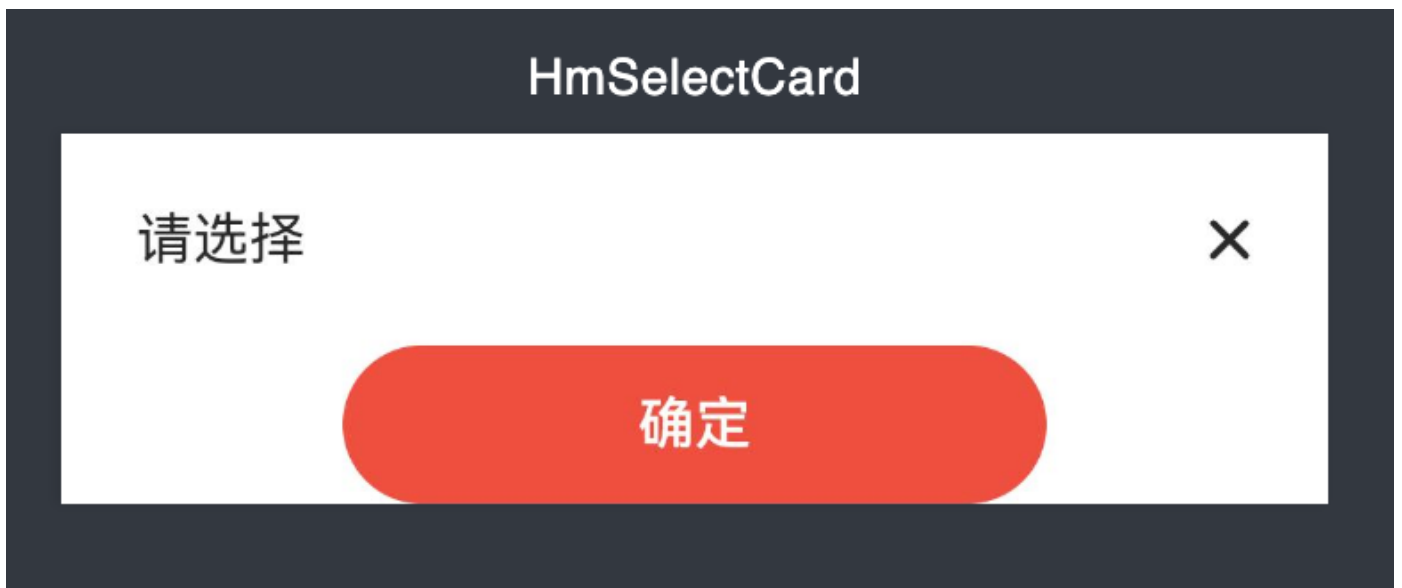
其他问题



确定

- 顶部内容可以设置标题，可以设置关闭按钮显示隐藏

- 可以设置顶部按钮显示隐藏，可以设置底部按钮的文本和颜色
  - 可以自定义传入的结构
  - 支持自定义Dialog的形式弹出
- 在components下新建HmSelectCard.ets组件



粘贴静态结构

```
@Preview
@CustomDialog
@Component
struct HmSelectCard {
    controller: CustomDialogController
    build() {
        Column() {
            Row() {
                Text("请选择").fontSize(16).fontColor($r('app.color.text_primary'))
                Image($r("app.media.ic_btn_close")).width(13).height(13)
            }
            .width('100%')
            .justifyContent(FlexAlign.SpaceBetween)
            .alignItems(VerticalAlign.Center)
            .height(60)
            .borderRadius({
                topLeft: 16,
                topRight: 16
            })
            // 渲染内容

            Row() {
                Button('确定', { type: ButtonType.Capsule
            }).width(200).backgroundColor($r('app.color.primary')).height(45)
            }.width('100%').justifyContent(FlexAlign.Center)

        }.backgroundColor($r('app.color.white')).justifyContent(FlexAlign.SpaceBetween).padding({
```

```

        left: 21.36,
        right: 21.36
    })
}
}

export { HmSelectCard }

```

- 定义开放属性，用于设置组件的内容

```

title: string = "请选择" // 显示标题
showClose: boolean = true // 显示关闭按钮
showButton: boolean = true // 显示底部按钮
buttonText: string = "确定" // 底部按钮文本
confirm: () => void = () => {}
@BuilderParam
cardContent: () => void

```

- 控制内容显示

```

@Preview
@CustomDialog
@Component
struct HmSelectCard {
    controller: CustomDialogController
    title: string = "请选择" // 显示标题
    showClose: boolean = true // 显示关闭按钮
    showButton: boolean = true // 显示底部按钮
    buttonText: string = "确定" // 底部按钮文本
    confirm: () => void = () => {}
    @BuilderParam
    cardContent: () => void
    build() {
        Column() {
            Row() {
                Text(this.title).fontSize(16).fontColor($r('app.color.text_primary'))
                if (this.showClose) {
                    Image($r("app.media.ic_btn_close")).width(13).height(13)
                        .onClick(() => {
                            this.controller.close()
                        })
                }
            }
        }
        .width('100%')
        .justifyContent(FlexAlign.SpaceBetween)
        .alignItems(VerticalAlign.Center)
        .height(60)
        .borderRadius({
            topLeft: 16,
            topRight: 16
        })
    }
}

```

```

// 渲染内容
if(this.cardContent) {
    this.cardContent()
}
if (this.showButton) {
    Row() {
        Button(this.buttonText, { type: ButtonType.Capsule })
            .width(200)
            .backgroundColor($r('app.color.primary'))
            .height(45)
            .onClick(() => {
                this.confirm()
            })
    }.width('100%').justifyContent(FlexAlign.Center)
}

}.backgroundColor($r('app.color.white')).justifyContent(FlexAlign.SpaceBetween).padding({
    left: 21.36,
    right: 21.36
})
}
}

export { HmSelectCard }

```

在components/index.ets导出

```
export * from './HmSelectCard'
```

## entry模块

在上报异常中使用

```

// 异常类型弹层
typeDialog: CustomDialogController = new CustomDialogController({
    builder: HmSelectCard({ cardContent: () => { this.getCardContent } }),
    autoCancel: false,
    customStyle: true,
    alignment: DialogAlignment.Bottom
})
@Builder
getCardContent () {}

```

- 这里必须得使用箭头函数来给cardContent赋值，否则在当前组件中无法正确使用this

点击右侧内容打开



```
HmCardItem({ leftTitle: '异常类型', rightText: '请选择', onRightClick: () => {  
    this.typeDialog.open()  
} })
```



## 渲染异常内容数据

|           |                                  |
|-----------|----------------------------------|
| 发动机启动困难   | <input checked="" type="radio"/> |
| 不着车，漏油    | <input type="radio"/>            |
| 照明失灵      | <input type="radio"/>            |
| 有异常响动     | <input type="radio"/>            |
| 排烟异常、温度异常 | <input type="radio"/>            |
| 其他问题      | <input type="radio"/>            |

- 定义循环项

```
@State
exceptErrorList: string[] = ["发动机启动困难", "不着车、漏油", "照明失灵", "有异常响动", "排烟异常、温度异常", "其他问题"]
```

- 使用一个轻量Builder来实现单个的渲染

```
// 单个的渲染选项
@Builder
getSingleItem() {
  Row() {
    Text("发动机漏油").fontSize(14).fontWeight(400).fontColor($r('app.color.text_primary'))
    Row() {
      Image($r("app.media.ic_check_true")).width(32).height(32)
    }
  }
  .justifyContent(FlexAlign.SpaceBetween)
  .alignItems(VerticalAlign.Center)
  .border({
    width: {
      bottom: 1
    },
    color: $r('app.color.background_divider')
  })
  .width('100%')
  .height(60)
}
```

- 根据数据动态生成N个的选项

```
@Builder
getCardContent() {
  ForEach(this.exceptionErrorList, () => {
    this.getSingleItem()
  })
}
```

- 定义一个class用来给Builder传递参数

```
class ExceptionItemClass {
  name: string = ""
  index: number = 0
}
```

- 给Builder传递class参数

```
@Builder
getSingleItem(item: string, index: number, showBorder: boolean) {
  Row() {
    Text(item)
      .fontSize(14)
      .fontColor($r("app.color.text_primary"))
  }
```

```

        Image(this.selectIndex === index ? $r("app.media.ic_check_true") :
$r("app.media.ic_check_false"))
            .width(32)
            .height(32)
    }
    .onClick(() => {
        this.selectIndex = index // 赋值索引
    })
    .height(60)
    .width("100%")
    .justifyContent(FlexAlign.SpaceBetween)
    .border({
        color: $r("app.color.background_divider"),
        width: {
            bottom: showBorder ? 1 : 0
        }
    })
})
}

// 传入HmSelectCard组件
@Builder
getCardContent() {
    ForEach(this.exceptionList, (item: string, index: number) => {
        this.getSingleItem(item, index, index !== this.exceptionList.length - 1)
    })
}
}

```

异常时间

请选择 >

上报位置

请选择 >

异常类型

请选择 >

异常描述

请选择



发动机启动困难



不着车、漏油



照明失灵



有异常响动



排烟异常、温度异常



其他问题



确定

- 定义一个状态索引，根据该索引判断当前选项是否被选中

```
@State
selectIndex: number = 0
```

```
@Builder
getSingleItem(item: string, index: number, showBorder: boolean) {
  Row() {
    Text(item)
      .fontSize(14)
      .fontColor($r("app.color.text_primary"))
    Image(this.selectIndex == index ? $r("app.media.ic_check_true") :
$r("app.media.ic_check_false"))
      .width(32)
      .height(32)
  }
  .onClick(() => {
    this.selectIndex = index // 赋值索引
  })
  .height(60)
  .width("100%")
  .justifyContent(FlexAlign.SpaceBetween)
  .border({
    color: $r("app.color.background_divider"),
    width: {
      bottom: showBorder ? 1 : 0
    }
  })
}

// 传入HmSelectCard组件
@Builder
getCardContent() {
  ForEach(this.exceptionList, (item: string, index: number) => {
    this.getSingleItem(item, index, index != this.exceptionList.length - 1)
  })
}
```

请选择



发动机启动困难



不着车、漏油



照明失灵



有异常响动



排烟异常、温度异常



其他问题



确定

# 选择类型

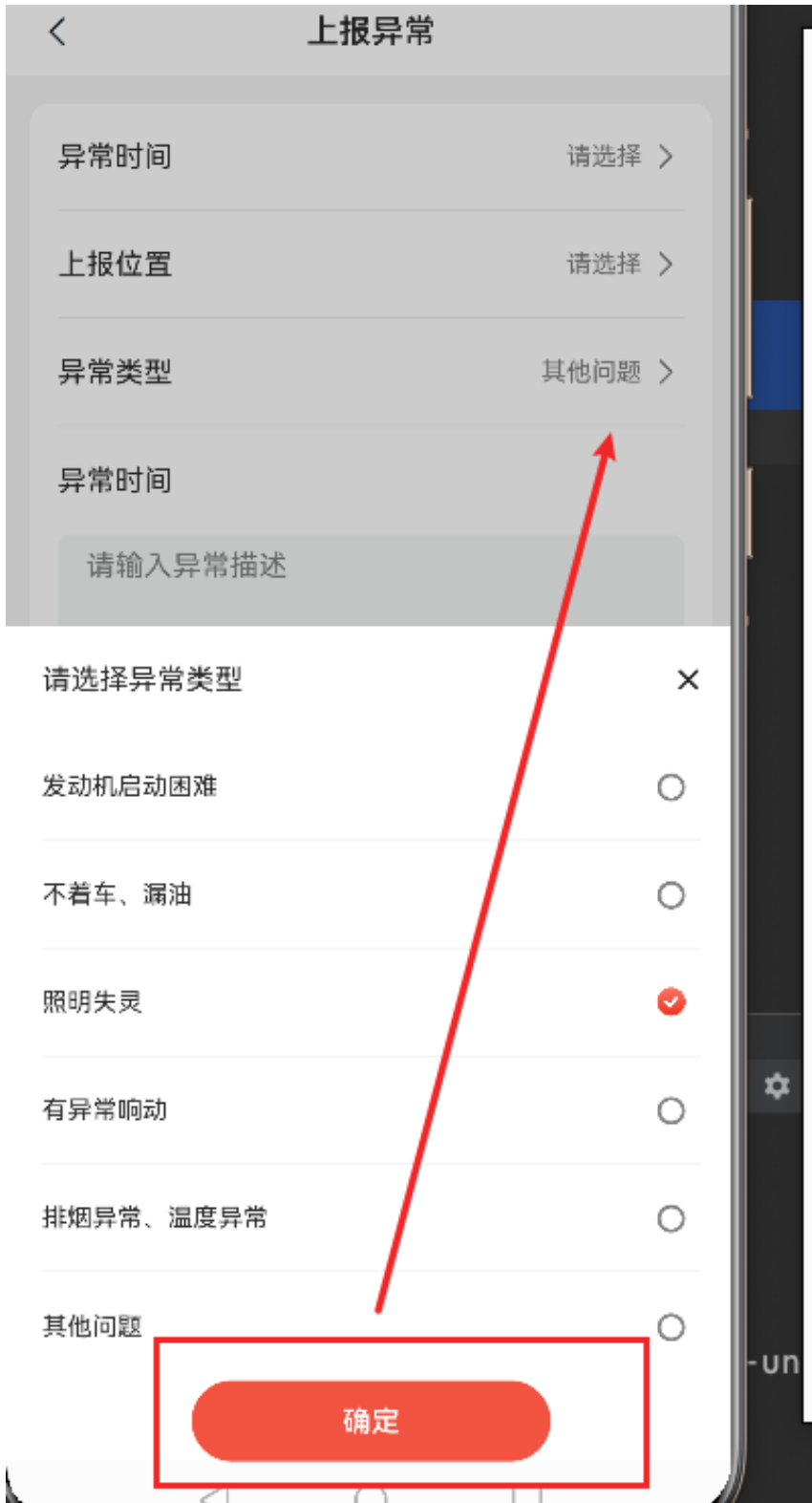
- 点击确定时，将选择的类型赋值给当前的类型属性

```
// 选择类型的弹层
selectTypeDialog: CustomDialogController = new CustomDialogController({
  builder: HmSelectCard({
    cardContent: () => {
      // 必须调用一个builder的函数
      this.getCardContent()
    },
    confirm: () => {
      if (this.selectIndex > -1) {
        this.exceptionForm.exceptionType = this.exceptionList[this.selectIndex]
      }
      this.selectTypeDialog.close()
    }
  }),
  customStyle: true,
  alignment: DialogAlignment.Bottom
})
```

- 绑定数据到CardItem组件

```
HmCardItem({ leftTitle: '异常类型', rightText: this.exceptionForm.exceptionType || '请选择',
showBottomBorder: false,
  onRightClick: () => {
    this.typeDialog.open()
  }
})
```





...info  
提交代码  
...

## ·上报异常-跳转当前位置页面



## 上报异常

异常时间

请选择 >

上报位置

请选择 >

异常类型

其他问题 >

异常时间

请输入异常描述

0/50

上传图片(最多6张)



提交



## 当前位置



小区/写字楼/学校等



2号

5号楼



回龙观街道

100m以内

金艳龙科研楼

200m

春野画室

300m

:::info

需要在点击上报位置时，跳转到选择位置页面

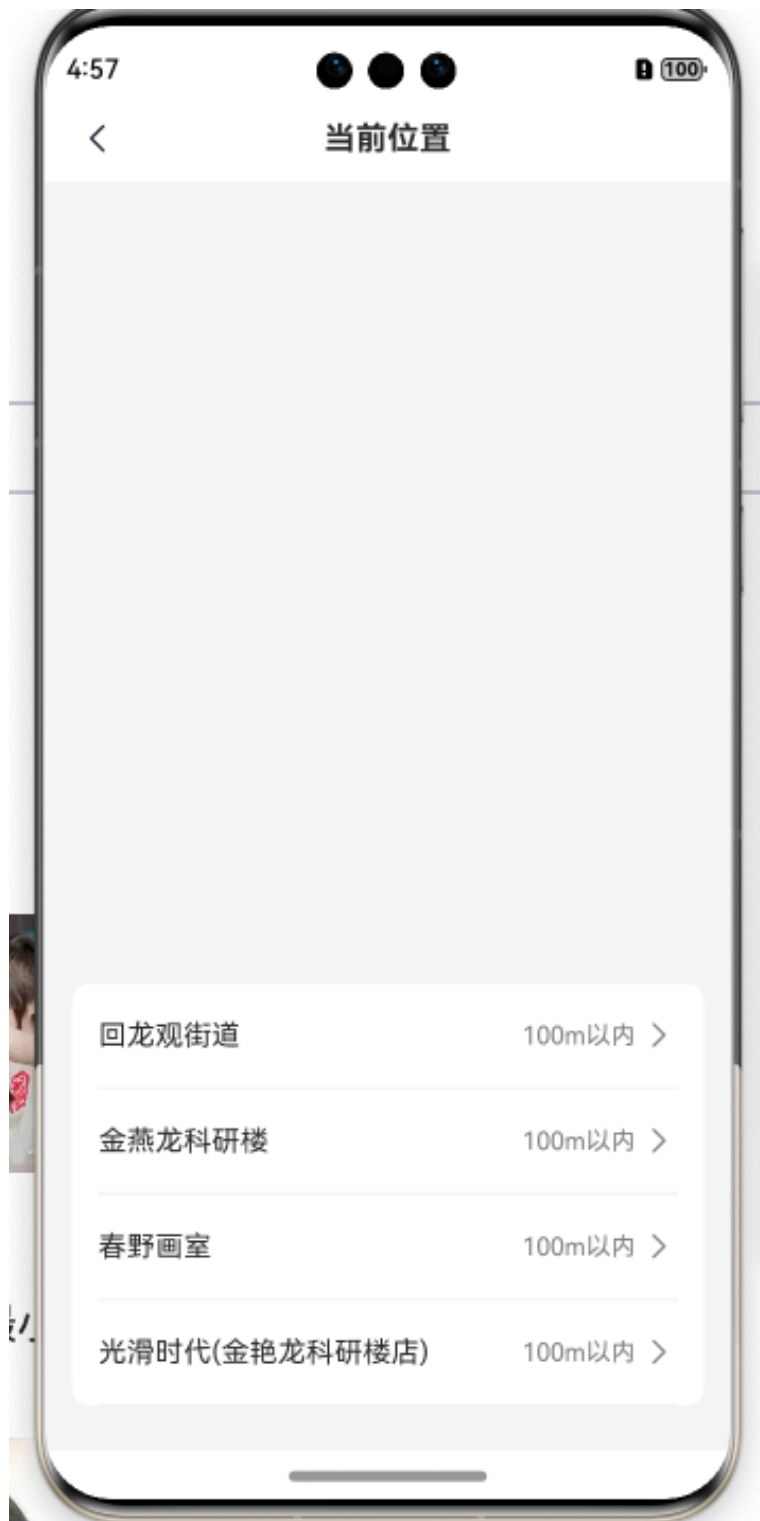
:::

### 1. 新建pages/SelectLocation/SelectLocation页面

```
import { HmNavBar, HmCard, HmCardItem } from '@hm/basic'
import { MapComponent } from '@hms.core.map.MapComponent'

@Entry
@Component
struct SelectLocation {
  build() {
    Column() {
      HmNavBar({ title: '当前位置' })
      Stack({ alignContent: Alignment.Bottom }) {
        // 地图区域
        MapComponent({
          mapOptions: {
            position: {
              target: {
                latitude: 39.9,
                longitude: 116.4
              },
            },
            zoom: 10
          },
        },
        mapCallback: () => {}
      })
      .width('100%')
      .height('100%')
    }
    Column() {
      HmCard() {
        HmCardItem({ leftTitle: '回龙观街道', rightText: '100m以内' })
        HmCardItem({ leftTitle: '金燕龙科研楼', rightText: '100m以内' })
        HmCardItem({ leftTitle: '春野画室', rightText: '100m以内' })
        HmCardItem({ leftTitle: '光滑时代(金艳龙科研楼店)', rightText: '100m以内' })
      }
    }
    .padding({
      bottom: 60
    })
  }
}
```

```
}  
  .height('100%').backgroundColor($r('app.color.background_page'))  
}  
}
```



- 点击上报位置跳转到选择位置

```
HmCardItem({ leftTitle: '上报位置', rightText: '请选择', onRightClick: () => {
  router.pushUrl({
    url: 'pages/SelectLocation/SelectLocation'
  })
}})
```

:::info  
提交代码  
...

## 配置地图

按照微信案例中的签名规则配置如下签名

p12  
p7b  
cer  
csr

| 名称                                                                                    |               |
|---------------------------------------------------------------------------------------|---------------|
|      | fast.cer      |
|      | fast.csr      |
|     | fast.p12      |
|    | fastDebug.p7b |
| >  | material      |
|                                                                                       |               |
|                                                                                       |               |

- 在agc中开启地图使用



定位服务



混合定位帮助您的应用快速精准获取用户位置信息。[展开](#)



位置服务



帮助您的用户更加方便的使用位置相关的服务。[展开](#)



地图服务



助力全球开发者实现个性化地图呈现与交互，全面提升您应用的LBS 体验。[展开](#)

- 在module.json5中配置client\_id

应用

SDK配置： 下载最新的配置文件（如果您修改了项目、应用信息或者更改了某个开发服务设置，可能需要更新该文件）

⌵

agconnect-services.json

☐ 不包含密钥 ?

包名： com.itheima.fastdriver

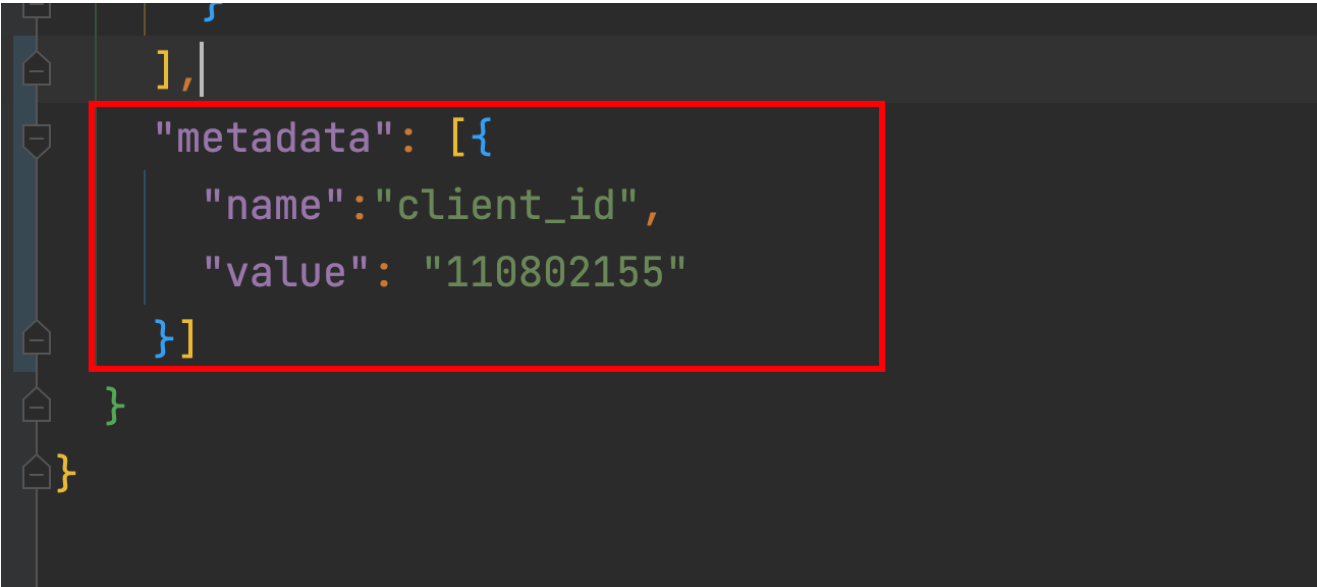
APP ID： 5765880207854187755

SHA256证书/公钥指纹： ? [添加证书指纹](#) [添加公钥指纹 \(HarmonyOS API 9及以上\)](#)

OAuth 2.0客户端ID（凭据）： Client ID 110802155  
Client Secret e7f8d8040936e915dc54e596dd962068a28a638da1f3d1fcea9...

回调地址： ?

删除应用



• 添加公钥指纹

⌵

agconnect-services.json

☐ 不包含密钥 ?

包名： com.itheima.fastdriver

APP ID： 5765880207854187755

SHA256证书/公钥指纹： ? [添加证书指纹](#) [添加公钥指纹 \(HarmonyOS API 9及以上\)](#)

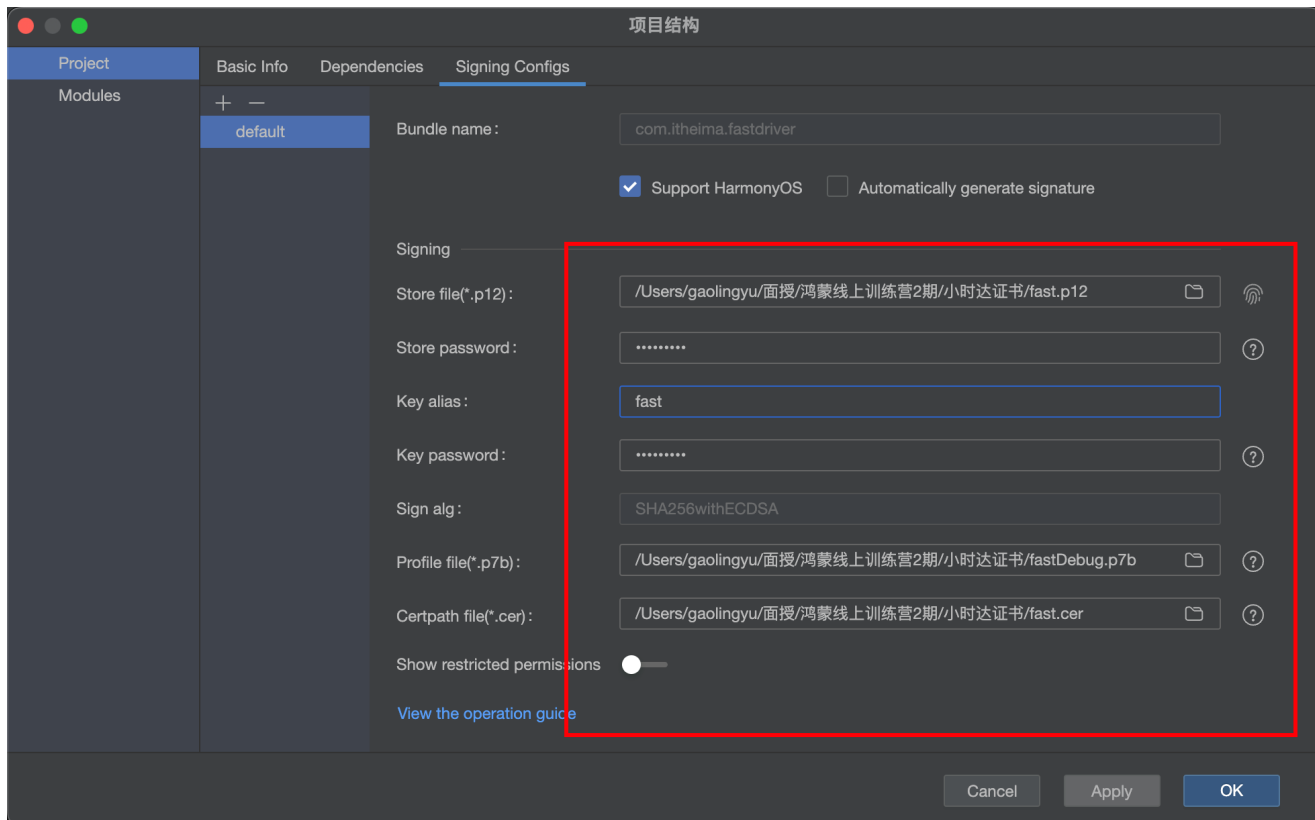
OAuth 2.0客户端ID（凭据）： Client ID 110802155  
Client Secret e7f8d8040936e915dc54e596dd962068a28a638da1f3d1fcea9...

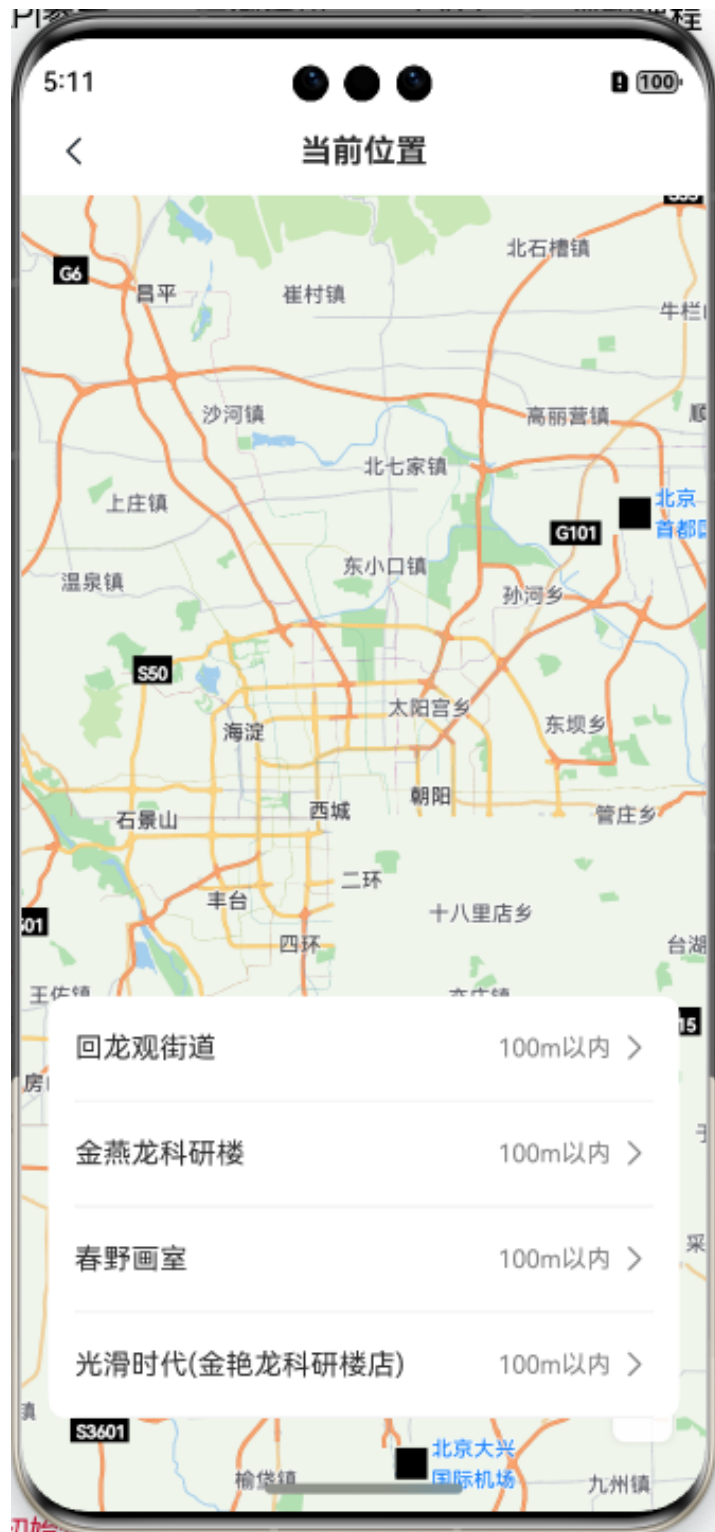
回调地址： ?

删除应用

• 配置手动签名







## 获取当前位置的经纬度

[官方位置文档](#)

- 引入经纬度管理

```
import geoLocationManager from '@ohos.geoLocationManager';
```

- 初始化时获取经纬度

```
aboutToAppear() {  
    this.getLocation()  
}  
async getLocation () {  
    try {  
        const result = await geoLocationManager.getCurrentLocation()  
        AlertDialog.show({  
            message: JSON.stringify(result)  
        })  
    } catch (error) {  
        AlertDialog.show({  
            message: error.message  
        })  
    }  
}
```

...info

没反应???



- 获取用户的地理位置必须经过用户同意-需要在初始化UIAbility的时候就发起请求
- 并且需要在module.json5中配置地址位置的权限
- ...
- module.json5

```
{  
    "name": "ohos.permission.LOCATION",  
    "reason": "$string:LOCATION_REASON",  
}
```

```

        "usedScene": {
            "abilities": [ "EntryAbility" ],
            "when": "always"
        }
    },
    {
        "name": "ohos.permission.APPROXIMATELY_LOCATION",
        "reason": "$string:LOCATION_REASON",
        "usedScene": {
            "abilities": [ "EntryAbility" ],
            "when": "always"
        }
    },

```

- 在string.json中填写原因

```

{
    "name": "LOCATION_REASON",
    "value": "申请地理位置"
}

```

- src/main/ets/entryability/EntryAbility.ts

```

import abilityAccessCtrl from '@ohos.abilityAccessCtrl'

async onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): Promise<void> {
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onCreate');
    let manger = abilityAccessCtrl.createAtManager()
    await manger.requestPermissionsFromUser(this.context,
        [
            'ohos.permission.LOCATION',
            'ohos.permission.APPROXIMATELY_LOCATION',
        ])
}

```

```
{"latitude":40,"longitude":116,"altitude":43.5,"accuracy":5,"speed":0,"timeStamp":1701676043,"direction":45,"timeSinceBoot":123425476384560,"additions":"","additionSize":0,"isFromMock":false}
```

回龙观街道 100m以内 >

金燕龙科研楼 100m以内 >

春野画室 100m以内 >

光滑时代(金艳龙科研楼店) 100m以内 >

因为模拟器没有真实的位置，想要给一个真实的地址的话可以在模拟器位置心里填入一个经纬度如图



[高德地图坐标拾取器](#)





填入

纬度

40.060507

经度

116.343779

高度

43.5

m

方位

45.00



测试

```
{'latitude':40.06050491333008,'longitude':116.34378051757812,'altitude':43.5,'accuracy':5,'speed':0,'timeStamp':1704512430,'direction':45,'timeSinceBoot':85719924907539,'additions':"", 'additionSize':0,'isFromMock':false}
```

...info

提交代码

...

## 转化经纬度坐标



## 坐标系知识介绍

华为地图涉及2种坐标系：

- WGS84：一种大地坐标系，也是目前广泛使用的GPS全球卫星定位系统使用的坐标系。
- GCJ02：由中国国家测绘局制订的地理信息系统的坐标系统，是由WGS84坐标系经过加密后的坐标系。

## 华为地图使用的坐标类型 [🔗](#)

在国内站点，中国大陆、中国香港和中国澳门使用GCJ02坐标系，中国台湾和海外使用WGS84坐标系。

在海外站点，统一使用WGS84坐标系。

我们需要将GCJ02转成WGS84

您可以通过[map](#)命名空间下的[convertCoordinate](#)方法进行坐标转换：

- 转化代码

```
import { map, mapCommon } from '@kit.MapKit';
let wgs84Position: mapCommon.LatLng = {
  latitude: 30,
  longitude: 118
};
let gcj02Posion: mapCommon.LatLng = await
map.convertCoordinate(mapCommon.CoordinateType.WGS84,
mapCommon.CoordinateType.GCJ02,wgs84Position);
```

突发!!!

目前获取的地理定位华为改为了GCJ02, 如果我们是显示在花瓣地图上，我们不需要再转化了  
高德地图的定位其实需要逆向转化。

## 获取controller之后设置当前位置

- 声明一个controller

```
mapController: map.MapComponentController = new map.MapComponentController()
```

- 回调赋值controller

```
MapComponent({
  mapOptions: {
    position: {
      target: {
        latitude: 39.9,
        longitude: 116.4
      }
    }
  }
})
```

```

        },
        zoom: 10
    }
},
mapCallback: (err, controller) => {
    if(!err) {
        this.mapController = controller
        this.getLocation()
    }
}
})

```

- 获取位置转化坐标-调整照相机位置

```

async getLocation() {
    try {
        const rightPostion = await geoLocationManager.getCurrentLocation()
        // let wgs84Position: mapCommon.LatLng = {
        //     latitude: result.latitude,
        //     longitude: result.longitude
        // };
        // let rightPostion: mapCommon.LatLng = await
        map.convertCoordinate(mapCommon.CoordinateType.WGS84,
            // mapCommon.CoordinateType.GCJ02, wgs84Position);
        this.mapController.moveCamera(map.newCameraPosition({
            target: {
                longitude: rightPostion.longitude,
                latitude: rightPostion.latitude
            },
            zoom: 16 // 缩放级别
        )))
        this.mapController.addPointAnnotation({
            position: {
                longitude: rightPostion.longitude,
                latitude: rightPostion.latitude
            },
            titles: [{
                content: '您当前的位置'
            }],
        })
    } catch (error) {
        AlertDialog.show({
            message: error.message
        })
    }
}
}

```



## 根据坐标点搜索周围地址

- 声明一个class

```
class SiteClass {  
    name: string = ""  
    distance: number = 0  
}
```

- 获取方法

```
const res = await site.nearbySearch({
  location: {
    longitude: rightResult.longitude,
    latitude: rightResult.latitude
  },
  pageSize: 4,
  pageIndex: 1,
  radius: 50
})
this.list = res.sites?.slice(0,4) as SiteClass[] // 只拿4条数据
```

- 声明状态变量

```
@State
list: SiteClass[] = []
```

- 赋值变量

```
const res = await site.nearbySearch({
  location: {
    longitude: rightResult.longitude,
    latitude: rightResult.latitude
  },
  pageSize: 4,
  pageIndex: 1,
  radius: 50
})
this.list = res.sites?.slice(0,4) as SiteClass[] // 只拿4条数据
```

- 循环地址

```
ForEach(this.list, (item: SiteClass) => {
  HmCardItem({ leftTitle: item.name, rightText: `${item.distance}m` })
    .onClick(() => {
      router.back({
        url: "pages/ExceptionReport/ExceptionReport",
        params: {
          location: item.name
        }
      })
    })
})
})
```



## 点击返回地址

- 点击选择城市返回地址参数

```
HmCardItem({ leftTitle: item.name, rightText: `${item.distance}m` })
    .onClick(() => {
      router.back({
        url: "pages/ExceptionReport/ExceptionReport",
        params: {
          location: item.name
        }
      })
    })
  })
})
```

#### 4. 在上报异常中获取数据

##### basic模块

- 在公共路由参数中再添加一个address字段

```
export class CommonRouterParams {
  id?: string = ''
  oldTime?: string = ''
  location?: string = ''
}
```

##### entry模块

```
// aboutToAppear
onPageShow(): void {
  const params = router.getParams() as CommonRouterParams
  if (params && params.location) {
    this.exceptionForm.exceptionPlace = params.location
  }
  if (params && params.id) {
    this.exceptionForm.transportTaskId = params.id
  }
}
```

#### 5. 显示位置到CardItem组件上

```
HmCardItem({ leftTitle: '上报位置', rightText: this.exceptionForm.exceptionPlace || '请选择', onRightClick: () => {
  router.pushUrl({
    url: 'pages/SelectLocation/SelectLocation'
  })
}})
```



...info  
提交代码  
...

## 处理异常图片的赋值-控制按钮

- 上传图片的赋值

```
HmUpload({
    title: '上传图片(最多6张)',
    maxNumber: 6,
    canUpload: true,
    imgList: this.exceptionForm.exceptionImagesList || [],
    onChange: (list: ImageList[]) => {
        this.exceptionForm.exceptionImagesList = []
    }
})
```

- 处理描述的双向绑定

```
maxNumber: number = 50
```

```
TextArea({
    placeholder: '请输入异常描述',
    text: this.exceptionForm.exceptionDescribe!
}).height(130).borderRadius(8).placeholderColor($r('app.color.text_secondary')).fontSize(14).onChange((value) => {
    this.exceptionForm.exceptionDescribe = value
}).maxLength(this.maxNumber)
Text(`${this.exceptionForm.exceptionDescribe?.length || 0}/${this.maxNumber}`)
    .margin({
        top: -30
    })
    .textAlign(TextAlign.End)
    .width('100%')
    .padding({ right: 15 })
    .fontColor($r('app.color.text_secondary'))
```

- 处理按钮状态

```
getBtnEnable () {
    return !(this.exceptionForm.exceptionDescribe &&
        this.exceptionForm.exceptionPlace &&
        this.exceptionForm.exceptionTime &&
        this.exceptionForm.exceptionType &&
        this.exceptionForm.exceptionDescribe)
}
```

- 绑定按钮



```
Row() {  
  Button("提交").height(50).width(207).backgroundColor(  
    this.getBtnEnable() ? $r('app.color.primary') :  
    $r('app.color.primary_disabled')).  
  enabled(this.getBtnEnable())  
}
```

The screenshot shows a mobile application interface for reporting an abnormality. The title bar at the top is black with a white back arrow on the left and the title '上报异常' in white. The status bar above the title bar shows the time 7:31, signal strength, and battery level. The form is a light gray card with rounded corners. It contains four input fields, each with a label on the left and a value on the right followed by a right-pointing chevron: '异常时间' (2023-12-04 07:30), '上报位置' (回龙观镇黄土南), '异常类型' (发动机启动困难), and '异常时间' (empty). Below the fourth field is a large text area with a light gray background, containing the text 'kj' and a character count '2/50' at the bottom right. Below the text area is a section for uploading images, labeled '上传图片(最多6张)', which contains a large square button with a gray background and a white plus sign. At the bottom of the form is a red rounded rectangle button with the white text '提交'. The bottom of the screen shows the Android navigation bar with back, home, and recent apps icons.

7:31

< 上报异常

异常时间 2023-12-04 07:30 >

上报位置 回龙观镇黄土南 >

异常类型 发动机启动困难 >

异常时间

kj

2/50

上传图片(最多6张)

+

提交

:::info  
提交代码  
...

# 上报异常提交

## 1. 封装api

```
// 上报异常
export const exceptionReport = (data: ExceptionParamsTypeModel) => {
  return Request.post<null>("/tasks/reportException", data)
}
```

## 2. 接收详情页过来的id

将之前的逻辑合二为一，初始化逻辑和页面显示事件

```
onPageShow() {
  const params = router.getParams() as CommonRouterParams
  if (params && params.id) {
    this.exceptionForm.transportTaskId = params.id
  }
  if(params && params.address) {
    this.exceptionForm.exceptionPlace = params.address
  }
}
```

## 3. 按钮点击提交-返回上一个页面

```
async btnReport () {
  await exceptionReport(this.exceptionForm)
  promptAction.showToast({
    message: '上报异常成功'
  })
  router.back()
}
```



:::info

- 提交代码
- ...

根据上报异常数据进行显示

## 异常信息

上报时间 2023-12-04 07:42:00

异常类型 发动机启动困难

处理结果 继续运输

:::info

如果任务详情中，有上报异常数据，则显示控制

:::

- 直接拷贝现成的结构

```
// 获取异常信息
@Builder
getExceptionContent() {
    ForEach(this.taskDetailData.exceptionList, (item: ExceptionList) => {
        Row() {
            Column() {
                Row() {
                    Text("上报时间").fontSize(14).fontColor($r('app.color.text_primary'))
                    Text(item.exceptionTime).margin({ left: 20
                }).fontColor($r('app.color.text_secondary'))
                }.height(50).alignItems(VerticalAlign.Center).width('100%')

                Row() {
                    Text("异常类型").fontSize(14).fontColor($r('app.color.text_primary'))
                    Text(item.exceptionType).margin({ left: 20
                }).fontColor($r('app.color.text_secondary'))
                }.height(50).alignItems(VerticalAlign.Center).width('100%')

                Row() {
                    Text("处理结果").fontSize(14).fontColor($r('app.color.text_primary'))
                    Text("继续运输").margin({ left: 20 }).fontColor($r('app.color.text_secondary'))
                }.height(50).alignItems(VerticalAlign.Center).width('100%')
            }
            // 跳转到详情
            Image($r("app.media.ic_btn_more")).width(24).height(24)
        }
        .width('100%')
        .padding({ left: 15, right: 15 })
        .alignItems(VerticalAlign.Center)
        .justifyContent(FlexAlign.SpaceBetween)
    })
}
```

```
}
```

- 放置到任务详情的内容区

```
if (this.taskDetailData.exceptionList?.length > 0) {  
    HmToggleCard({ title: '异常信息' }) {  
        this.getExceptionContent()  
    }  
}
```



```
:::info  
提交代码  
:::
```

## 上报完异常-回到详情加载异常信息

### entry模块

1. 在上报异常页面传递一个标记，表示已经加了一条数据

```
async btnReport() {  
    await exceptionReport(this.exceptionForm)  
    promptAction.showToast({  
        message: '上报异常成功'  
    })  
    router.back({  
        url: 'pages/TaskDetail/TaskDetail',  
        params: {  
            addExcept: true  
        }  
    })  
}
```

在路由的公共参数添加一个属性add\_except的属性

### basic模块

```
export class CommonRouterParams {
  id?: string = ''
  oldTime?: string = ''
  address?: string = ''
  addExcept?: boolean = false
}
```

2. 在TaskDetail的onPageShow中判断此字段，然后重新加载数据，

注意！！！！只重新赋值异常的信息部分，否则之前上传的图片会被干掉，因为只有异常发生了变化，所以不能执行原来的获取执行逻辑

```
async onPageShow() {
  const params = router.getParams() as CommonRouterParams
  if(params && params.addExcept) {
    const result = await getTaskDetail(this.taskDetailData.id)
    this.taskDetailData.exceptionList = result.exceptionList
  }
}
```

#### 异常信息

上报时间 2023-12-04 07:59:00

异常类型 有异常响动



处理结果 继续运输

上报时间 2023-12-04 07:42:00

异常类型 发动机启动困难



处理结果 继续运输

:::info

提交代码

...

## 回显上报异常组件



需求

点击右侧的箭头-需要回到异常详情页去显示内容

这里我们剑走偏锋，因为没有查询异常详情的接口，我们可以把点击当前行的整行数据传到详情页

#### basic模块

- 在公共路由参数添加一个新的参数

```
export class CommonRouterParams {  
  id?: string = ''  
  oldTime?: string = ''  
  address?: string = ''  
  addExcept?: boolean = false  
  formData?: object  
}
```

#### entry模块

- 新建pages/Exception/ExceptionDetail.ets(page)

这里做过多操作已经无意，直接拷贝下方结构即可

- 直接拷贝结构

```
import { HmNavBar, HmCardItem, HmCard, CommonRouterParams, ImageList } from '@hm/basic'  
import router from '@ohos.router'  
import { ExceptionListModel, ExceptionList } from '../../models'  
  
@Entry  
@Component
```

```

struct ExceptDetail {
  @State submitForm: ExceptionListModel = new ExceptionListModel({} as ExceptionList)
  aboutToAppear() {
    const params = router.getParams() as CommonRouterParams
    if (params && params.formData) {
      // 检查formData
      this.submitForm = params.formData as ExceptionListModel
    }
  }
  @Builder
  getCardChildren() {
    HmCardItem({ leftTitle: '异常时间', rightText: this.submitForm.exceptionTime,
showRightIcon: false, })
    HmCardItem({ leftTitle: '上报位置', rightText: this.submitForm.exceptionPlace,
showRightIcon: false, })
    HmCardItem({ leftTitle: '异常类型', rightText: this.submitForm.exceptionType,
showRightIcon: false })
    HmCardItem({ leftTitle: '异常描述', showBottomBorder: false, showRightIcon: false,
rightText: '' })
    Row() {

Text(this.submitForm.exceptionDescribe).fontSize(14).fontColor($r('app.color.text_primary'))
    }.padding(15).justifyContent(FlexAlign.Start).width('100%')
  }
  @Builder
  getUpload() {
    if (this.submitForm.exceptionImagesList?.length) {
      Text("异常图片").width('100%').padding(10)
      Flex({ wrap: FlexWrap.Wrap, direction: FlexDirection.Row }) {
        ForEach(this.submitForm.exceptionImagesList, (item: ImageList) => {
          Image(item.url)
            .width(95)
            .height(95)
            .borderRadius(4)
            .margin({ right: 15 })
        })
      }
      .width('100%').margin({ top: 16.5, bottom: 16.5 })
    }
  }

  build() {
    Column() {
      HmNavBar({ title: '异常详情' })
      HmCard() {
        this.getCardChildren()
      }
      HmCard() {
        this.getUpload()
      }
    }.height('100%').backgroundColor($r('app.color.background_page'))
  }
}

```



```
}  
}  
  
export default ExceptDetail
```

## 2. 点击TaskDetail的右侧箭头跳转-传递formData

```
// 跳转到详情  
Image($r("app.media.ic_btn_more")).width(24).height(24).onClick(() => {  
  router.pushUrl({  
    url: "pages/ExceptionReport/ExceptionDetail",  
    params: {  
      formData: item  
    }  
  })  
})
```



...info  
提交代码  
...

## 导航功能



:::info

只在在途的状态下显示导航

:::

// 在途情况下 才显示导航

```
if (this.taskDetailData.status === TaskTypeEnum.Line) {  
  Column() {  
    Image($r("app.media.ic_navigation")).width(22).height(22)  
    Text("开始导航").fontSize(14).margin({ top: 10, bottom: 10 })  
  }.justifyContent(FlexAlign.SpaceBetween)  
  .margin({  
    top: 20  
  })  
}
```

- 点击开始导航

// 获取基础信息

// 开始导航

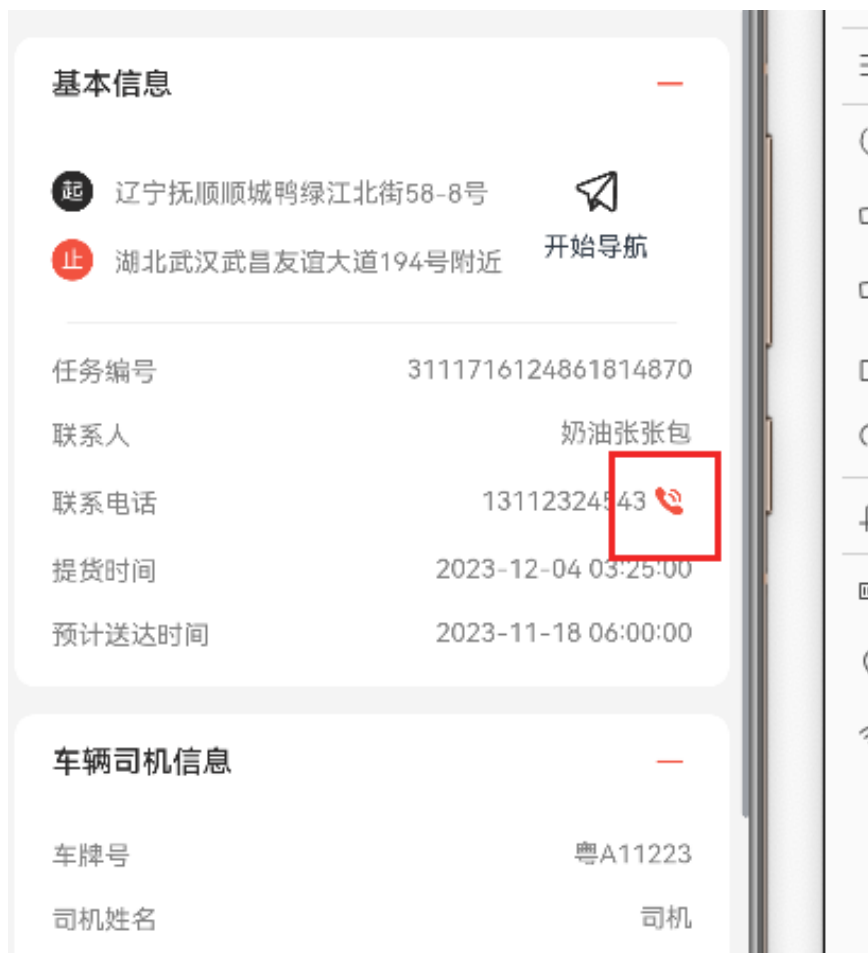
```
async beginNav() {  
  try {  
    let context = getContext(this) as common.UIAbilityContext  
    context.startAbility({  
      action: 'ohos.want.action.viewData',  
      entities: ['entity.system.browsable'],  
      uri: encodeURI('https://gaode.com/search?query=' + this.taskDetailData.endAddress)  
    })  
  } catch (error) {  
    AlertDialog.show({  
      message: JSON.stringify(error)  
    })  
  }  
}
```

```
}
```

目前高德地图的导航调用方式无法得知，无法打开导航路线，只能用浏览器唤起打开高德页面，传入我们的地址

```
:::info  
提交代码  
:::
```

## 打电话功能



### 1. 导入使用包

```
import call from '@ohos.telephony.call'
```

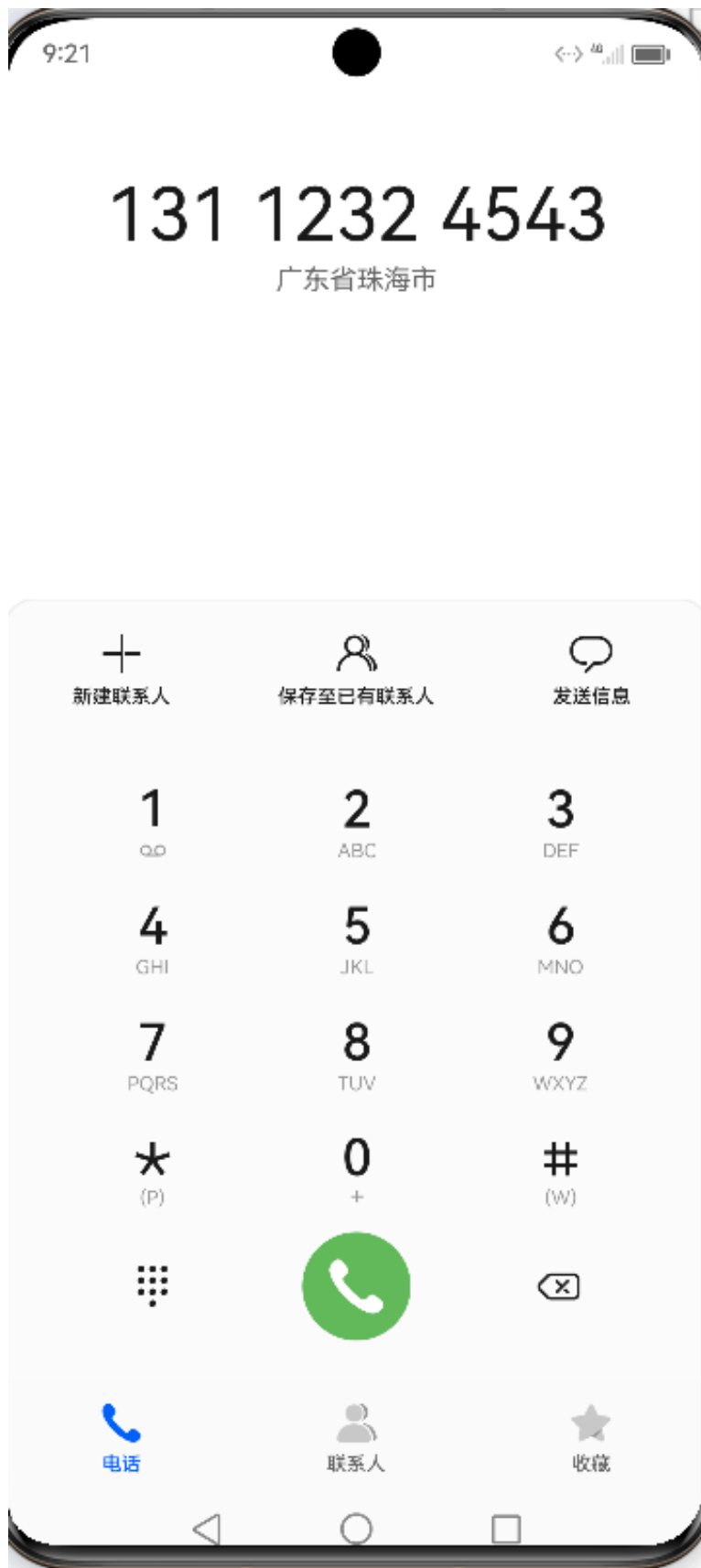
- 给之前的builder类型添加一个图标点击事件

```
class BaseBuilderClass {
    title: string = ""
    value: string = ""
    icon?: ResourceStr = ""
    iconClick?: () => void = () => {}
}
```

## 2. 点击图标打电话

```
@Builder
getBaseContentItem(item: BaseBuilderClass) {
    Row() {
        Text(item.title).fontSize(14).fontColor($r('app.color.text_secondary'))
            .lineHeight(20)
        Row() {
            Text(item.value).fontSize(14).fontColor($r('app.color.text_secondary'))
            if (item.icon) {
                Image(item.icon).width(24).height(24)
                    .onClick(() => {
                        item.iconClick?()
                    })
            }
        }
    }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
        top: 14
    })
}
```

```
this.getBaseContentItem({
    title: '联系电话',
    value: this.taskDetailData.startHandoverPhone,
    icon: $r('app.media.ic_phone'),
    iconClick: () => {
        call.makeCall(this.taskDetailData.startHandoverPhone);
    }
})
```



:::info  
提交代码  
:::

已完成任务列表加入搜索条件结构

 请输入任务编号

开始时间

至

结束时间

筛选

```
build() {
  Column() {
    if(this.queryParams.status === TaskTypeEnum.Finish) {
      this.getSearchForm()
    }
    HmList({
      onLoad: async () => {
        await this.getTaskList(true)
      },
      onRefresh: async () => {
        await this.onRefresh()
      },
      dataSource: $taskListData,
      renderItem: this.renderItem,
      finished: this.allPage < this.queryParams.page,
      finishText: '没啦没啦',
      loadingText: '拼命加载中'
    })
  }
  .height('100%')
}
```

搜索条件builder

```
@Builder
getSearchForm() {
  Column() {
    Row() {
      Search({ placeholder: '请输入任务编号'
    }).backgroundColor($r('app.color.background_page')).height(32)
    }
    .justifyContent(FlexAlign.Center)
    .padding({ left: 15, right: 15, bottom: 5 })

    Row() {
      // 完成搜索页需要测试点击之后键盘和弹层同时弹出的情况
      Button('开始时间')
        .fontSize(14)
    }
  }
}
```

```

        .width(106)
        .height(32)
        .padding({ left: 0, right: 0 })
        .fontColor('#999')
        .backgroundColor($r('app.color.background_page'))

Text("至")
Button('结束时间')
    .fontSize(14)
    .width(110)
    .height(32)
    .padding({ left: 0, right: 0 })
    .fontColor('#999')
    .backgroundColor($r('app.color.background_page'))
Button("筛选")
    .backgroundColor($r('app.color.primary_disabled'))
    .height(32)
    .width(60)

}.width('100%').alignItems(VerticalAlign.Center).justifyContent(FlexAlign.SpaceAround)
}
.backgroundColor($r('app.color.white'))
.padding(15)
.justifyContent(FlexAlign.Center)
.width('100%')
}

```

- 在TaskTabs中复用TaskList

```

else if(item.name === "finish") {
    TaskList({ queryParams: new TaskListParamsModel({
        page: 1,
        pageSize: 5,
        status: TaskTypeEnum.Finish
    }) as TaskListParams })
}

```

- 处理TaskItemCard在已完成状态下的按钮显示



```

if (parseInt(this.taskItemData.status) !== TaskTypeEnum.Finish) {
  Button(this.getBtnText(), { type: ButtonType.Capsule })
    .backgroundColor($r('app.color.primary'))
    .fontColor($r("app.color.white"))
    .fontSize(14)
    .height(32)
    .enabled(this.getBtnEnable())
    .onClick(() => {
      this.toPickUp()
    })
}

```

- 点击卡片，当已完成时，可以进去查看详情

```

.onClick(() => {
  if (parseInt(this.taskItemData.status) === TaskTypeEnum.Finish) {
    router.pushUrl({
      url: 'pages/TaskDetail/TaskDetail',
      params: {
        id: this.taskItemData.id
      }
    })
  }
})

```

- TaskDetail中当已完成时，可以展示图片

```

}
if (this.taskDetailData.status === TaskTypeEnum.Waiting
|| this.taskDetailData.status === TaskTypeEnum.Delivered
|| this.taskDetailData.status === TaskTypeEnum.Finish
) {
  HmToggleCard({ title: '提货信息' }) {
    this.getPickUpContent()
  }
}
// 交货时显示交货照片
if (this.taskDetailData.status === TaskTypeEnum.Line
|| this.taskDetailData.status === TaskTypeEnum.Delivered
|| this.taskDetailData.status === TaskTypeEnum.Finish
) {
  HmToggleCard({ title: '交货信息' }) {
    this.getDeliverContent()
  }
}
}

```

- 当已完成时，不显示底部内容

```
if(this.taskDetailData.status !== TaskTypeEnum.Finish) {  
    this.getBottomBtn() // 底部按钮结构  
}
```



...info  
提交代码  
...

# 已完成搜索-选择日期

entry模块

Q

请输入任务编号

开始时间

至

结束时间

筛选

任务编号：3111716124861814870

起

辽宁抚顺顺城鸭绿江北街58-8号

止

湖北武汉武昌友谊大道194号附近

提货时间

2023-12-04 03:25:00

任务编号：3111716124860458266

起

辽宁抚顺顺城鸭绿江北街58-8号

2023年12月4日星期一

2021年

10月

2日

2022年

11月

3日



```
.onClick(() => {  
    DatePickerDialog.show({  
        selected: new Date(),  
        onAccept: (value: DatePickerResult) => {  
  
            }  
        })  
    })  
})
```

- 补零函数

```
// 日期补零  
addZero(value: number) {  
    return value.toString().padStart(2, "0")  
}
```

- 赋值查询参数

```
@Builder  
getSearchForm() {  
    Column() {  
        Row() {  
            Search({ placeholder: '请输入任务编号'  
        }).backgroundColor($r('app.color.background_page')).height(32)  
        }  
        .justifyContent(FlexAlign.Center)  
        .padding({ left: 15, right: 15, bottom: 5 })  
  
        Row() {
```

```

// 完成搜索页需要测试点击之后键盘和弹层同时弹出的情况
Button(this.queryParams.startTime || '开始时间')
    .fontSize(14)
    .width(106)
    .height(32)
    .padding({ left: 0, right: 0 })
    .fontColor('#999')
    .backgroundColor($r('app.color.background_page'))
    .onClick(() => {
        DatePickerDialog.show({
            selected: new Date(),
            onAccept: (value: DatePickerResult) => {
                this.queryParams.startTime = `${value.year}-${this.addZero(value.month! +
1)}-${this.addZero(value.day)}~`
            }
        })
    })

Text("至")
Button(this.queryParams.endTime || '结束时间')
    .fontSize(14)
    .width(110)
    .height(32)
    .padding({ left: 0, right: 0 })
    .fontColor('#999')
    .backgroundColor($r('app.color.background_page'))
    .onClick(() => {
        DatePickerDialog.show({
            selected: new Date(),
            onAccept: (value: DatePickerResult) => {
                this.queryParams.endTime = `${value.year}-${this.addZero(value.month! +
1)}-${this.addZero(value.day)}~`
            }
        })
    })

Button("筛选")
    .backgroundColor($r('app.color.primary_disabled'))
    .height(32)
    .width(60)

}.width('100%').alignItems(VerticalAlign.Center).justifyContent(FlexAlign.SpaceAround)
}
.backgroundColor($r('app.color.white'))
.padding(15)
.justifyContent(FlexAlign.Center)
.width('100%')
}

```

实现一个方法控制筛选按钮可点

// 控制筛选按钮可点

```
getSearchEnable() {  
  return !(this.queryParams.startTime && this.queryParams.endTime)  
}
```

```
Button("筛选")  
  .backgroundColor(this.getSearchEnable() ? $r('app.color.primary') :  
$r('app.color.primary_disabled'))  
  .height(32)  
  .width(60)  
  .enabled(this.getSearchEnable())
```

- 加入loading进度条

```
loading: CustomDialogController = new CustomDialogController({  
  builder: HmLoading({  
    title: '搜索查询中'  
  }),  
  customStyle: true,  
  autoCancel: false,  
  alignment: DialogAlignment.Center  
})
```

- 查询弹层

```
Button("筛选")  
  .backgroundColor(this.getSearchEnable() ? $r('app.color.primary') :  
$r('app.color.primary_disabled'))  
  .height(32)  
  .width(60)  
  .enabled(this.getSearchEnable())  
  .onClick(async() => {  
    this.layer.open()  
    this.allPage = 1  
    this.queryParams.page = 1  
    await this.getTaskList(false)  
    this.layer.close()  
  })
```

待提货    在途    已完成

🔍 请输入任务编号

2023-12-04

至

2023-12-04

筛选

任务编号: 3111716124861814870

起

辽宁抚顺顺城鸭绿江北街58-8号

止

湖北武汉武昌友谊大道194号附近

提货时间

2023-12-04 03:25:00

:::info  
提交代码  
...

## 清空状态控制

目前我们发现一个问题，选择完开始时间和结束时间，没有办法取消了，所以我们定义一个状态来控制下

- 定义一个状态

```
@State
reset: boolean = false // 用于控制重置状态
```

- 如果查询完一次之后，将状态设置为true

```
Button(this.reset ? "重置" : "筛选")
    .backgroundColor(this.getSearchEnable() ? $r('app.color.primary') :
$r('app.color.primary_disabled'))
    .height(32)
    .width(60)
    .enabled(this.getSearchEnable())
    .onClick(async() => {
        if(this.reset) {
            this.queryParams.startTime = ''
            this.queryParams.endTime = ''
            this.reset = false
        }else {
            this.reset = true
        }
        this.loading.open()
        this.allPage = 1
        this.queryParams.page = 1
        await this.getTaskList(false)
        this.loading.close()
    })
```

## 输入单号搜索



🔍 3111716124861814870



开始时间

至

结束时间

筛选

任务编号: 3111716124861814870

起

辽宁抚顺顺城鸭绿江北街58-8号

止

湖北武汉武昌友谊大道194号附近

提货时间

2023-12-04 03:25:00

输入单号不影响分页及其他，只覆盖数据，不影响下拉刷新的逻辑

- 接口查询有个bug, 就是单号必须是数字的字符串，否则查不出内容，所以需要特殊处理下

```
Search({ placeholder: '请输入任务编号', value: this.queryParams.transportTaskId || ''
}).backgroundColor($r('app.color.background_page')).height(32)
  .onSubmit(async value => {
    this.loading.open()
    this.queryParams.page = 1
    if(value) {
      this.queryParams.startTime = ''
      this.queryParams.endTime = ''
    }
  })
```

重复

```
        this.reset = false
        this.allPage = 0 // 有值意味着只有一条 因为是根据单号传的，所以不让他继续往后查 否则会

        this.queryParams.transportTaskId = isNaN(parseInt(value)) ? "0" : value
    }else {
        this.allPage = 1 // 没值意味着查不到 重新加载
        this.queryParams.transportTaskId = ""
    }
    const result = await getTaskList(this.queryParams)
    this.taskListData = result.items || []
    this.loading.close()
  })
}
```

- 这里逻辑捋一捋
- 如果有值，那么意味着即使查也只能查出一条，那么不能触发上拉加载，所以有值的情况下，要把allPage设置为0， 否则设置为1，
- 有值情况下，如果已经有开始时间和结束时间，要清空，如果有重置，则恢复
- 最终将数据赋值进行设置

整体代码

```
import { HmList, HmLoading } from '@hm/basic/Index'
import { getTaskList } from '../../api'
import { TaskInfoItem, TaskInfoItemModel, TaskListParams, TaskListParamsModel, TaskTypeEnum } from '../../models'
import TaskItemCard from './TaskItemCard'
import { promptAction } from '@kit.ArkUI'

// 待提货
@Component
struct TaskList {
  loading: CustomDialogController = new CustomDialogController({
    builder: HmLoading(),
    customStyle: true
  })
  @State
  queryParams: TaskListParamsModel = new TaskListParamsModel(
    {
      status: TaskTypeEnum.Waiting, // 待提货的类型
      page: 1, // 第几页
      pageSize: 5 // 每页几条数据
    } as TaskListParams
  )
  @State
  taskListData: TaskInfoItem[] = []
  @State
  allPage: number = 1 // 默认只有一页
  @State
  reset: boolean = false // 用来控制重置状态
```

```

async getTaskList(append: boolean) {
  const result = await getTaskList(this.queryParams)
  // 追加数据
  // this.taskListData = this.taskListData.concat(result.items) // 拿到返回的数组
  if (append) {
    this.taskListData.push(...result.items || []) // 延展运算符的写法
  } else {
    this.taskListData = result.items // 直接赋值
  }

  this.allPage = result.pages // 总页数
  this.queryParams.page++ // 下次请求的页码
}

@Builder
renderItem(item: object) {
  TaskItemCard({ taskItemData: item as TaskInfoItemModel })
}

// 下拉刷新函数
async onRefresh() {
  // 重新请求第一页数据
  this.queryParams.page = 1 // 重置第一页
  await this.getTaskList(false) // 直接赋值
}

// 补零函数
addZero(value: number) {
  return value.toString().padStart(2, "0")
}

// 获取筛选按钮是否可用
getSearchEnable() {
  return !(this.queryParams.startTime && this.queryParams.endTime)
}

@Builder
getSearchForm() {
  Column() {
    Row() {
      Search({ placeholder: '请输入任务编号', value: this.queryParams.transportTaskId ||
"" })

      .backgroundColor($r('app.color.background_page'))
      .height(32)
      .onSubmit(async (value) => {
        // 点击键盘的右下角的提交
        // 4042715413936324752
        this.loading.open()
        this.allPage = 1 // 总页数为1
        this.queryParams.page = 1 // 查第一页
        if (value) {
          // 有单号的情况 如果有只有一条记录

```

```

        // 后台业务缺陷- 按道理来说 如果传单号的了 应该开始时间和结束时间自动忽略
        this.queryParams.startTime = ""
        this.queryParams.endTime = ""
        this.queryParams.transportTaskId = value
    } else {
        // 没有单号的情况下恢复之前的查询
        this.queryParams.transportTaskId = ""
    }
    await this.getTaskList(false) // 不追加数据
    this.loading.close()
  })
}
.justifyContent(FlexAlign.Center)
.padding({ left: 15, right: 15, bottom: 5 })

Row() {
  // 完成搜索页需要测试点击之后键盘和弹层同时弹出的情况
  Button(this.queryParams.startTime || '开始时间')
    .fontSize(14)
    .width(106)
    .height(32)
    .padding({ left: 0, right: 0 })
    .fontColor('#999')
    .backgroundColor($r('app.color.background_page'))
    .onClick(() => {
      DatePickerDialog.show({
        selected: new Date(),
        onDateAccept: (value) => {
          this.queryParams.startTime =
            `${value.getFullYear()}-${this.addZero(value.getMonth() + 1)}-${
              this.addZero(value.getDate())
            }`
        }
      })
    })
})

Text("至")
Button(this.queryParams.endTime || '结束时间')
  .fontSize(14)
  .width(110)
  .height(32)
  .padding({ left: 0, right: 0 })
  .fontColor('#999')
  .backgroundColor($r('app.color.background_page'))
  .onClick(() => {
    DatePickerDialog.show({
      selected: new Date(),
      onDateAccept: (value) => {
        this.queryParams.endTime =
          `${value.getFullYear()}-${this.addZero(value.getMonth() + 1)}-${
            this.addZero(value.getDate())
          }`
      }
    })
  })
})

```

```

    })
    Button(this.reset ? "重置" : "筛选")
      .backgroundColor($r('app.color.primary'))
      .height(32)
      .width(60)
      .enabled(this.getSearchEnable())
      .onClick(async () => {
        if (this.reset) {
          // 表示现在需要重置 将开始时间和结束时间清空
          this.queryParams.startTime = ""
          this.queryParams.endTime = ""
        }
        this.reset = !this.reset // 状态取反

        this.loading.open()
        // 搜索
        // 处理总页数 处理第几页
        this.allPage = 1 // 默认有一页
        this.queryParams.page = 1
        // 获取数据
        await this.getTaskList(false) // true是追加 false是不追加

        this.loading.close()
      })
  })

}.width('100%').alignItems(VerticalAlign.Center).justifyContent(FlexAlign.SpaceAround)
}
.backgroundColor($r('app.color.white'))
.padding(15)
.justifyContent(FlexAlign.Center)
.width('100%')
}

build() {
  Column() {
    // 是否显示搜索条件
    if (this.queryParams.status === TaskTypeEnum.Finish) {
      // 显示搜索条件
      this.getSearchForm()
    }
    HmList({
      dataSource: this.taskListData, // 数据源
      finished: this.allPage < this.queryParams.page, // 是否还有下一页
      // 上拉加载的函数
      onLoad: async () => {
        // 上拉加载
        await this.getTaskList(true) // 追加逻辑
      },
      onRefresh: async () => {
        // 下拉刷新
        await this.onRefresh()
      }
    })
  }
}

```

```

    },
    renderItem: (item: object) => {
      // 如果需要的是builderParams的参数 可以用普通函数包裹一个Builder的函数
      this.renderItem(item)
    },
    loadingText: '拼命加载中',
    finishText: '没啦没啦'
  }).height("100%")
}

}

}

export default TaskList

```

:::info

- 提交代码
- ...

## 多线程处理图片压缩

性能优化

- 资源-图片不要全用原图
- 请求-尽可能延伸-最好不要把所有请求都放在入口处
- 组件-尽可能采用拆分组件-延展组件- 低开门-高入户
- 设计-多层架构-高内聚低耦合-公用har-包共用hsp
- 多任务处理-多线程-耗时任务-IO操作-文件压缩-解压缩-裁剪-位移
- 主线程-子线程处理耗时任务

1. 建立多线程 子线程
2. 将主线程拿到的图片给到子线程
3. 子线程进行图片的压缩
4. 压缩完成 将完成的图片信息回传给主线程
5. 子线程自动关闭销毁
6. 主线程继续上传操作

- 新建一个UploadWorker

```

import { ImageList } from '@hm/basic/Index';
import { ErrorEvent, MessageEvents, ThreadWorkerGlobalScope, util, worker } from
 '@kit.ArkTS';
import { fileIo } from '@kit.CoreFileKit';
import { image } from '@kit.ImageKit';

const workerPort: ThreadWorkerGlobalScope = worker.workerPort;

```

```

export class PostParams {
  files: ImageList[] = []
  filePath: string = "" // 沙箱目录
}

/**
 * Defines the event handler to be called when the worker thread receives a message sent
 * by the host thread.
 * The event handler is executed in the worker thread.
 *
 * @param e message data
 */
workerPort.onmessage = async (e: MessageEvents) => {
  // 处理耗时任务e
  const params = e.data as PostParams
  // 进行图片压缩
  if (params && params.files) {
    // 拿到原图
    // 对原图进行压缩
    // 需要将原图压缩成新的图片 存到一个位置
    const imagePackerAPI = image.createImagePacker() // 创建图片压缩API
    let packOpts: image.PackingOption = { format: "image/jpeg", quality: 20 };
    let arr: ImageList[] = []
    while (params.files.length) {
      const obj = params.files.pop() // 每次取一条
      const sourceFile = fileIo.openSync(obj?.url, fileIo.OpenMode.READ_ONLY) // 打开来源文
      件
      // 对来源文件进行压缩
      const newFileName = params.filePath + "/" + util.generateRandomUUID() + ".jpg"
      const targetFile = fileIo.openSync(newFileName, fileIo.OpenMode.CREATE |
fileIo.OpenMode.READ_WRITE)
      // 目标文件的fd
      await imagePackerAPI.packToFile(image.createImageSource(sourceFile.fd),
targetFile.fd, packOpts)
      fileIo.closeSync(sourceFile.fd) // 关闭来源文件
      fileIo.closeSync(targetFile.fd) // 关闭目标文件
      arr.push({
        url: newFileName
      })
    }
    // 图片压缩完毕 将图片传出去
    // 这里不需要序列化
    workerPort.postMessage({ files: arr }) // 子线程往主线程发消息
    workerPort.close() // 结束子线程
  }
}

/**

```

```

    * Defines the event handler to be called when the worker receives a message that cannot
    be deserialized.
    * The event handler is executed in the worker thread.
    *
    * @param e message data
    */
workerPort.onmessageerror = (e: MessageEvents) => {
}

/**
    * Defines the event handler to be called when an exception occurs during worker
    execution.
    * The event handler is executed in the worker thread.
    *
    * @param e error message
    */
workerPort.onerror = (e: ErrorEvent) => {
}

```

- 提货交货时处理图片压缩

```

// 去提货
async btnPickUp() {
    this.loading.open()
    // 建立子线程
    const imageCompress1 = new worker.ThreadWorker("entry/ets/workers/UploadWorker.ets")
// 子线程1
    const imageCompress2 = new worker.ThreadWorker("entry/ets/workers/UploadWorker.ets")
// 子线程2
    let cargoPickUpPictureList: ImageList[] = []
    let cargoPictureList: ImageList[] = []

    imageCompress1.onmessage = (e: MessageEvents) => {
        const params = e.data as PostParams
        cargoPickUpPictureList = params.files // 已经压缩完毕的图片
        checkFile() // 检查是否都完成了
    }
    imageCompress2.onmessage = (e: MessageEvents) => {
        const params = e.data as PostParams
        cargoPictureList = params.files // 已经压缩完毕的图片
        checkFile()
    }
    imageCompress1.postMessage({
        files: [...this.taskDetailData.cargoPickUpPictureList],
        filePath: getContext().filesDir
    }) // 给子线程发消息
    imageCompress2.postMessage({
        files: [...this.taskDetailData.cargoPictureList],
        filePath: getContext().filesDir
    }) // 给子线程发消息
}

```



```

const checkFile = async () => {
  if (cargoPickUpPictureList.length && cargoPictureList.length) {
    const result = await Promise.all([UploadFile(cargoPickUpPictureList),
UploadFile(cargoPictureList)])
    // 提货操作
    await pickUp(new PickUpParamsModel({
      id: this.taskDetailData.id,
      cargoPickUpPictureList: result[0],
      cargoPictureList: result[1]
    }))
    this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据
    this.scroller.scrollEdge(Edge.Top)
    this.loading.close()
    promptAction.showToast({ message: '提货成功' })
  }
}

// const cargoPickUpPictureList = await
UploadFile(this.taskDetailData.cargoPickUpPictureList) // 传入提货凭证
// const cargoPictureList = await UploadFile(this.taskDetailData.cargoPictureList)
// 货品照片
// 提货操作
// await pickUp(new PickUpParamsModel({
//   id: this.taskDetailData.id,
//   cargoPickUpPictureList,
//   cargoPictureList
// })))
// // 只需要重新获取数据
// // 重新获取数据
// this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据
//
// this.scroller.scrollEdge(Edge.Top)
// this.loading.close()
// // 滚动到顶部
// promptAction.showToast({ message: '提货成功' })

}

// 交货
async btnDeliver() {
  this.loading.open()
  // 建立子线程
  const imageCompress1 = new worker.ThreadWorker("entry/ets/workers/UploadWorker.ets")
// 子线程1
  const imageCompress2 = new worker.ThreadWorker("entry/ets/workers/UploadWorker.ets")
// 子线程2
  let certificatePictureList: ImageList[] = []
  let deliverPictureList: ImageList[] = []
  imageCompress1.onmessage = (e: MessageEvents) => {
    const params = e.data as PostParams
    certificatePictureList = params.files // 已经压缩完毕的图片
  }
}

```

```

        checkFile() // 检查是否都完成了
    }
    imageCompress2.onmessage = (e: MessageEvents) => {
        const params = e.data as PostParams
        deliverPictureList = params.files // 已经压缩完毕的图片
        checkFile()
    }
    imageCompress1.postMessage({
        files: [...this.taskDetailData.certificatePictureList],
        filePath: getContext().filesDir
    }) // 给子线程发消息
    imageCompress2.postMessage({
        files: [...this.taskDetailData.deliverPictureList],
        filePath: getContext().filesDir
    }) // 给子线程发消息
    // 用来检查是否全部都压缩完毕
    const checkFile = async () => {
        if (certificatePictureList.length && deliverPictureList.length) {
            // 此时此刻才可以进行下一步操作
            const result = await Promise.all([UploadFile(certificatePictureList),
            UploadFile(deliverPictureList)])
            await deliver(new DeliverParamsTypeModel({
                id: this.taskDetailData.id,
                certificatePictureList: result[0],
                deliverPictureList: result[1]
            }))
            this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据
            this.scroller.scrollEdge(Edge.Top)
            promptAction.showToast({ message: '交货成功' })
            this.loading.close()
        }
    }
    // this.loading.open()
    // const certificatePictureList = await
    UploadFile(this.taskDetailData.certificatePictureList) // 传入提货凭证
    // const deliverPictureList = await UploadFile(this.taskDetailData.deliverPictureList)
    // 货品照片
    // // 交货操作
    // await deliver(new DeliverParamsTypeModel({
    //     id: this.taskDetailData.id,
    //     certificatePictureList,
    //     deliverPictureList
    // }))
    // // 只需要重新获取数据
    // // 重新获取数据
    // this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据
    //
    // this.scroller.scrollEdge(Edge.Top)
    // this.loading.close()
    // // 滚动到顶部
    // promptAction.showToast({ message: '交货成功' })

```

```
}
```

# 集成消息模块

:::info

消息模块内容较为简单

同学们将老师提供的模块导入到项目中结成即可，也可以自己尝试自己去编写

...

## entry模块

### 1. 新建models/message.ts

```
export interface MessageQueryType {
  /** 消息类型，200：司机端公告，201：司机端系统通知 */
  contentType: number;
  page: number;
  pageSize: number;
}

export interface MessageData {
  /** 总条目数 */
  counts: number;
  items: MessageItem[];
  /** 页码 */
  page: number;
  /** 总页数 */
  pages: number;
  /** 页尺寸 */
  pageSize: number;
}

/** 数据列表 */
export interface MessageItem {
  /** 1：用户端，2：司机端，3：快递员端，4：后台管理系统 */
  bussinessType: number;
  /** 消息内容 */
  content: string;
  /** 消息类型，300：快递员端公告，301：寄件相关消息，302：签收相关消息，303：快件取消消息，200：司机端公告，201：司机端系统通知 */
  contentType: number;
  /** 创建时间 */
  created: string;
  /** 创建者 */
  createUser: number;
  /** 主键 */
  id: number;
  /** 消息是否已读，0：未读，1：已读 */
  isRead: number;
  /** 读时间 */
}
```

```

readTime: string;
/** 相关id */
relevantId: number;
/** 消息标题 */
title: string;
/** 更新时间 */
updated: string;
/** 更新者 */
updateUser: number;
/** 消息接受者 */
userId: number;
}
interface DetailType {
  id: string
  content: string
  created: string
}

export class MessageQueryTypeModel implements MessageQueryType {
  contentType: number = 0
  page: number = 0
  pageSize: number = 0

  constructor(model: MessageQueryType) {
    this.contentType = model.contentType
    this.page = model.page
    this.pageSize = model.pageSize
  }
}

export class MessageDataModel implements MessageData {
  counts: number = 0
  items: MessageItem[] = []
  page: number = 0
  pages: number = 0
  pageSize: number = 0

  constructor(model: MessageData) {
    this.counts = model.counts
    this.items = model.items
    this.page = model.page
    this.pages = model.pages
    this.pageSize = model.pageSize
  }
}

export class MessageItemModel implements MessageItem {
  bussinessType: number = 0
  content: string = ''
  contentType: number = 0
  created: string = ''
  createUser: number = 0
  id: number = 0
  isRead: number = 0
}

```

```

readTime: string = ''
relevantId: number = 0
title: string = ''
updated: string = ''
updateUser: number = 0
userId: number = 0

constructor(model: MessageItem) {
  this.bussinessType = model.bussinessType
  this.content = model.content
  this.contentType = model.contentType
  this.created = model.created
  this.createUser = model.createUser
  this.id = model.id
  this.isRead = model.isRead
  this.readTime = model.readTime
  this.relevantId = model.relevantId
  this.title = model.title
  this.updated = model.updated
  this.updateUser = model.updateUser
  this.userId = model.userId
}
}

export class DetailTypeModel implements DetailType {
  id: string = ''
  content: string = ''
  created: string = ''

  constructor(model: DetailType) {
    this.id = model.id
    this.content = model.content
    this.created = model.created
  }
}

```

- 在models/index.ts中导出

```
export * from './message'
```

- 新建api/message.ets

```

import { Request } from '@hm/basic'
import { MessageQueryTypeModel, MessageDataModel } from '../models'

// 获取信息
export const getMessage = (params: MessageQueryTypeModel) => {
  return Request.get<MessageDataModel>("/messages/page", params)
}

// 标记已读

```

```

export const readMessage = (id: string) => {
  return Request.put<null>(`/messages/${id}`)
}

// 全部已读
export const readAllMessage = (contentType: string) => {
  return Request.put<null>(`/messages/readAll/${contentType}`)
}

```

- 在api/index.ets导出

```
export * from './message'
```

- 在pages/Index下新建Message/Message.ets组件

```

import { HmNavBar, TabClass } from '@hm/basic'
import MessageTemplate from './MessageTemplate'
@Component
struct Message {
  @State currentTabName: string = 'information'
  @State tabBarData: TabClass[] = [{
    title: '公告',
    name: 'information'
  }, {
    title: '任务通知',
    name: 'notice'
  }]
  @Builder
  TabBuilder(item: TabClass) {
    Column() {
      Text(item.title)
        .fontSize(16)
        .fontColor(this.currentTabName === item.name ? $r('app.color.text_primary') :
$r('app.color.text_secondary'))
        .fontWeight(this.currentTabName === item.name ? 500 : 400)
        .lineHeight(50)
        .height(50)
      Divider()
        .strokeWidth(4)
        .color($r('app.color.primary'))
        .opacity(this.currentTabName === item.name ? 1 : 0)
        .width(this.currentTabName === item.name ? 23 : 0)
        .animation({
          duration: 300,
          curve: Curve.EaseOut,
          iterations: 1,
          playMode: PlayMode.Normal
        })
    }
  }
}

```

```

build() {
  Column() {
    HmNavBar({ title: '消息', showBackIcon: false })
    Tabs({
      barPosition: BarPosition.Start,
      controller: new TabsController()
    }) {
      ForEach(this.tabBarData, (item: TabClass) => {
        TabContent() {
          Flex() {
            MessageTemplate({ type: item.name || '' })
              .height('100%').backgroundColor($r('app.color.background_page'))
              .tabBar(this.TabBuilder(item))
          }
        }.animationDuration(300).onChange((index) => {
          this.currentTabName = index === 0 ? 'information' : 'notice'
        })
      }.width('100%').height('100%')
    }
  }
}

export default Message

```

- 在Message.ets旁新建MessageTemplate.ets

```

import { getMessage, readAllMessage, readMessage } from '../../api'
import { MessageQueryTypeModel, MessageItemModel } from '../../models'
import router from '@ohos.router';
import { HmList } from '../../components'
import promptAction from '@ohos.promptAction';

@Component
struct MessageTemplate {
  @Prop
  type: string
  @State
  queryParams: MessageQueryTypeModel = new MessageQueryTypeModel({
    page: 1,
    pageSize: 10,
    contentType: this.type === "information" ? 200 : 201
  })
  @State
  allPage: number = 1
  @State
  messageList: MessageItemModel[] = []

  async getMessage(append: boolean) {
    if (this.allPage < this.queryParams.page) {
      return
    }
    const result = await getMessage(this.queryParams)

```

```

    if (append) {
        this.messageList = this.messageList.concat(result.items) // 追加数据
    } else {
        this.messageList = result.items // 覆盖数据
    }
    this.queryParams.page++
    this.allPage = result.pages
}

async refreshData() {
    this.allPage = 1 // 只要下拉就默认有一页
    this.queryParams.page = 1
    await this.getMessage(false)
    promptAction.showToast({ message: '刷新成功' })
}

@Builder
renderItem(item: object) {
    if(this.type === "information") {
        this.renderInformationItem(item as MessageItemModel)
    }
    if(this.type === "notice") {
        this.renderNoticeItem(item as MessageItemModel)
    }
}

// 读取
readSuccess() {
    this.messageList = this.messageList.map(item => {
        item.isRead = 1
        return item
    })
}

@Builder
renderInformationItem (item: MessageItemModel) {
    Row() {
        Row() {
            if (item.isRead === 0) {
                Text("").width(8).height(8).backgroundColor($r('app.color.primary')).borderRadius(4)
            }
            Text(item.content)
                .fontSize(14)
                .fontColor($r("app.color.text_primary"))
                .margin({ left: 6 })
                .textOverflow({ overflow: TextOverflow.Ellipsis })
                .maxLines(1)
        }.width(230)

        Text(item.created).fontSize(12).fontColor($r("app.color.text_secondary"))
    }
    .justifyContent(FlexAlign.SpaceBetween)
    .alignItems(VerticalAlign.Center)
    .padding({ left: 22, right: 22 })
}

```



```

.height(60)
.backgroundColor($r('app.color.white'))
.border({ width: { bottom: 1 }, color: $r("app.color.background_page") })
.width('100%')
.onClick(() => {
    // 先更改数据状态
    item.isRead = 1
    this.messageList = [...this.messageList]
    router.pushUrl({
        url: 'pages/Index/Message/MessageDetail',
        params: {
            content: item.content,
            created: item.created,
            id: item.id
        }
    })
})
}
}

@Builder
renderNoticeItem(message: MessageItemModel) {
    Column() {
        Row() {
            Text("您有新的运输任务")
            if (message.isRead === 0) {
                Text("")
                    .width(8)
                    .height(8)
                    .backgroundColor($r('app.color.primary'))
                    .borderRadius(4)
                    .margin({ left: 10 })
            }
        }

        Divider().color($r('app.color.background_page')).margin({ top: 13 })
        Text(message.content.replace(new RegExp("/\n/"), ""))
            .margin({ top: 11, bottom: 22.5 })
            .fontSize(13)
            .fontColor($r('app.color.text_secondary'))
            .maxLines(2)
            .textOverflow({ overflow: TextOverflow.Ellipsis })
            .lineHeight(22)
        Row() {
            Text(message.updated).fontSize(12).fontColor($r('app.color.text_secondary'))
            Button("查看详情", { type: ButtonType.Capsule })
                .fontColor($r('app.color.primary'))
                .border({ width: 1, color: $r('app.color.primary') })
                .height(24)
                .width(76)
                .backgroundColor('#fff')
                .onClick(() => {
                    readMessage(message.id + "")
                    // 跳转到任务详情
                })
            }
        }
    }
}

```

```

        router.pushUrl({
            url: 'pages/TaskDetail/TaskDetail',
            params: {
                id: message.relevantId
            }
        })
    })
})

}.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).width('100%')
}
.width('100%')
.backgroundColor($r('app.color.white'))
.borderRadius(10)
.padding(16)
.margin({ bottom: 15 })
.alignItems(HorizontalAlign.Start)
}

build() {
    Column() {
        Row() {
            Image($r("app.media.ic_yidu")).width(16).height(16)
            Text("全部已读").fontSize(14).fontColor($r('app.color.text_secondary')).margin({
                left: 9
            }).onClick(async () => {
                this.queryParams.contentType && await
readAllMessage(this.queryParams.contentType.toString())
                promptAction.showToast({ message: '全部已读' })
                this.readSuccess && this.readSuccess()
            })
        }.padding({
            left: 17,
            right: 17,
            top: 14,
            bottom: 14
        }).width('100%') // 全部已读
        // 公告内容
        HmList({
            finished: this.allPage < this.queryParams.page,
            dataSource: $messageList,
            onRefresh: async () => {
                await this.refreshData()
            },
            onLoad: async () => {
                await this.getMessage(true)
            },
            renderItem: (item: object) => {
                this.renderItem(item)
            }
        })
    }.height('100%')
}
}

```

```

}

export default MessageTemplate

```

- 在Index/Index.ets中引入Message

```

import Message from './Message/Message'

build() {
  Tabs({ barPosition: BarPosition.End }) {
    ForEach(this.tabsData, (item: TabClass) => {
      TabContent() {
        if(item.name === 'task') {
          TaskTabs()
        }
        else if(item.name === 'message') {
          Message()
        }
        else {
          My()
        }
      }.tabBar(this.getTabBar(item))
    })
  }
  .onChange(index => {
    this.currentName = this.tabsData[index].name
  })
  .animationDuration(300)
}

```

- 在pages/Index/Message下新建MessageDetail.ets(page)

```

import { HmNavBar } from '@hm/basic'
import router from '@ohos.router';
import { readMessage } from '../../api/message'
import { DetailTypeModel } from '../../models'
@Entry
@Component
struct MessageDetail {
  @State detailForm: DetailTypeModel = new DetailTypeModel({
    content: '',
    created: '',
    id: ''
  })

  async aboutToAppear() {
    const params = router.getParams()
    this.detailForm = params as DetailTypeModel
    readMessage(this.detailForm.id)
  }
}

```

```

build() {
  Flex({ direction: FlexDirection.Column }) {
    HmNavBar({ title: '详情' })
    Column() {
      Text("系统公告").fontSize(16).fontColor($r('app.color.text_primary')).lineHeight(22)

      Text(this.detailForm.created).fontSize(12).fontColor($r('app.color.text_secondary')).lineHeight(17).margin({
        top: 7,
        bottom: 17
      })
      Text(this.detailForm.content).fontSize(14).lineHeight(22)
    }.alignItems(HorizontalAlign.Start).padding({
      top: 20,
      left: 16,
      right: 16
    })
  }.height('100%').backgroundColor($r('app.color.white'))
}
}

export default MessageDetail

```

## 集成车辆信息

### entry模块

在models/car.ets中加入车辆信息类型

```

import { ImageList } from '@hm/basic'
/** 响应数据 */
export interface UserCarDataType {
  /** 载重 */
  allowableLoad: string;
  /** 所属机构名称 */
  currentOrganName: string;
  /** 车辆编号 */
  id: string;
  /** 车牌号码 */
  licensePlate: string;
  /** 图片 */
  pictureList: ImageList[];
  /** 车辆类型名称 */
  truckType: string;
}
export class UserCarDataTypeModel implements UserCarDataType {
  allowableLoad: string = ''
  currentOrganName: string = ''

```

```

id: string = ''
licensePlate: string = ''
pictureList: ImageList[] = []
truckType: string = ''

constructor(model: UserCarDataType) {
  this.allowableLoad = model.allowableLoad
  this.currentOrganName = model.currentOrganName
  this.id = model.id
  this.licensePlate = model.licensePlate
  this.pictureList = model.pictureList
  this.truckType = model.truckType
}
}

```

- 在models/index.ets统一导出

```
export * from './car'
```

在api/user.ts中加入封装获取车辆信息api

```

// 获取用户车辆信息
export const getUserCarInfo = () => {
  return Request.get<UserCarDataTypeModel>("/users/truck")
}

```

新建pages/Car/Car.ets - Page

```

import { HmNavBar } from '@hm/basic'
import { getUserCarInfo } from '../../api'
import { UserCarDataTypeModel, UserCarDataType, ImageList } from '../../models'
@Entry
@Component
struct Car {
  @State
  userCarInfo: UserCarDataTypeModel = new UserCarDataTypeModel({} as UserCarDataType)
  aboutToAppear() {
    this.getUserCarInfo()
  }
  async getUserCarInfo() {
    this.userCarInfo = await getUserCarInfo()
  }
  @Builder
  getContentItem (item: CarItem) {
    Row() {

      Text(item.leftTitle).fontSize(14).fontWeight(400).fontColor($r('app.color.text_secondary'
    ))
    }
  }
}

```

```

        Text(item.rightValue).fontSize(14).fontColor($r('app.color.text_primary')).fontWeight(400)
    )

}.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).width('100%').height(40)
}
build() {
    Column() {
        HmNavBar({ title: '车辆信息' })
        Swiper(new SwiperController()) {
            ForEach(this.userCarInfo.pictureList, (item: ImageList) => {
                Row() {
                    Image(item.url).width('100%').height('100%').objectFit(ImageFit.Cover).
                        borderRadius(8)
                }.height(201).padding({ left: 15, right: 15, top: 15 })
            })
        }
        .loop(false)
        .cachedCount(3)
        .indicator(true)
        .curve(Curve.Linear)

        // 信息列表
        Column() {
            this.getContentItem({ leftTitle: '车辆编号', rightValue: this.userCarInfo.id })
            this.getContentItem({ leftTitle: '车辆号牌', rightValue:
this.userCarInfo.licensePlate })
            this.getContentItem({ leftTitle: '车型', rightValue: this.userCarInfo.truckType
})
            this.getContentItem({ leftTitle: '所属机构', rightValue:
this.userCarInfo.currentOrgName })
            this.getContentItem({ leftTitle: '载重', rightValue:
this.userCarInfo.allowableLoad })
        }
        .padding({
            top: 19.5,
            bottom: 19.5,
            left: 20,
            right: 20
        })
        .backgroundColor($r('app.color.white'))
        .margin(15)
        .borderRadius(8)
    }.height('100%').backgroundColor($r('app.color.background_page')).width('100%')
}
}

class CarItem {
    leftTitle: string = ""
    rightValue: string = ""
}

```

```
}  
export default Car
```

- 点击我的-车辆信息跳转过去

```
HmCardItem({ leftTitle: '车辆信息', rightText: '', onRightClick: () => {  
  router.pushUrl({  
    url: 'pages/Car/Car'  
  })  
} })
```



## 车辆信息



|      |                     |
|------|---------------------|
| 车辆编号 | 1575257934065905665 |
| 车辆号牌 | 粤A11223             |
| 车型   | 12.8米箱式货车hjfhjvhj   |
| 所属机构 | 沈阳分拣中心              |
| 载重   | 10.00               |

:::info  
提交代码  
:::

## 集成任务信息



:::info

获取任务数据之前已经封装过，直接使用即可

...

- 新建pages/UserTask/UserTask.ets

```
import { HmNavBar, HmLoading } from '@hm/basic'
import { UserTaskInfoModel, UserTaskInfo, UserTaskInfoParamsModel } from '../models'
import { getUserTaskInfo } from '../api'
@Entry
@Component
struct UserTask {
  @State TaskInfo: UserTaskInfoModel = new UserTaskInfoModel({} as UserTaskInfo)
  @State list: string[] = []
  @State queryParams: UserTaskInfoParamsModel = new UserTaskInfoParamsModel({
    year: new Date().getFullYear()+"",
    month: new Date().getMonth() + 1 +""
  })
  @State
  currentDate: Date = new Date()
  layer: CustomDialogController = new CustomDialogController({
    builder: HmLoading(),
    customStyle: true,
    alignment: DialogAlignment.Center
  })
  async aboutToAppear() {
    this.getUserTask()
  }
  async getUserTask () {
    this.layer.open()
    this.TaskInfo = await getUserTaskInfo(this.queryParams)
    this.layer.close()
  }
  build() {
    Column() {
      HmNavBar({ title: '任务数据' })

      // 本月任务
      Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Center,
        justifyContent: FlexAlign.SpaceAround }) {
        Text("- 本月任务 -")
          .fontSize(14).fontColor($r('app.color.text_secondary')).lineHeight(20)
        Row() {
          Column() {
            Text(this.TaskInfo.taskAmounts+
              "").fontSize(18).fontColor($r('app.color.text_primary')).lineHeight(25).margin({
                bottom: 17
              })
          }
          Text("任务总量")
            .fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
        }.justifyContent(FlexAlign.SpaceAround)
      }
    }
  }
}
```

```

        Column() {
            Text(this.TaskInfo.completedAmounts+"")
                .fontSize(18)
                .fontColor($r('app.color.text_primary'))
                .lineHeight(25)
                .margin({ bottom: 17 })
            Text("完成任务
量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
        }.justifyContent(FlexAlign.SpaceAround)

        Column() {
            Text(this.TaskInfo.transportMileage+"")
                .fontSize(18)
                .fontColor($r('app.color.text_primary'))
                .lineHeight(25)
                .margin({ bottom: 17 })
            Text("运输里程
(km)").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
        }.justifyContent(FlexAlign.SpaceAround)
        }.justifyContent(FlexAlign.SpaceEvenly).width('100%').flexGrow(1)
    }
    .backgroundColor($r('app.color.white'))
    .margin({ left: 14.5, right: 14.5 })
    .height(100).margin({
        top: 20,
        bottom: 20
    })
}
Row() {
    Button("切换月份")
        .backgroundColor($r('app.color.primary'))
        .onClick(() => {
            CalendarPickerDialog.show({
                selected: this.currentDate,
                onAccept: (value) => {
                    this.queryParams.year = value.getFullYear().toString()
                    this.queryParams.month = (value.getMonth() + 1).toString()
                    this.getUserTask()
                }
            })
        })
}
}.justifyContent(FlexAlign.Center)
.width('100%')

}
.height('100%').backgroundColor($r('app.color.background_page'))
}
}

```

- 在我的页面跳转过去

```
HmCardItem({ leftTitle: '任务设置', rightText: '',
  onRightClick: () => {
    router.pushUrl({
      url: 'pages/UserTask/UserTask'
    })
  }
})
```



:::info  
提交代码  
...

## 做一个截图效果

---

- 实现一个转发按钮

```
Row() {  
    Image($r("app.media.share"))  
        .width(20)  
        .height(20)  
        .fillColor($r("app.color.primary"))  
        .onClick(async () => {  
            this.snapImg = await componentSnapshot.get("detail")  
            this.showSnap = true  
        })  
}  
.width("100%")  
.justifyContent(FlexAlign.End)  
.padding(10)
```



- 定义变量

```
@State
snapImg: image.PixelMap | null = null
@State
showSnap: boolean = false
```

- 使用bindContentCover

```
.bindContentCover($$this.showSnap, this.getSnapContent(), {  
  modalTransition: ModalTransition.NONE  
})
```

- 实现弹出内容

```
@Builder  
getSnapContent () {  
  Column() {  
    Image(this.snapImg)  
      .width("100%")  
      .height("100%")  
      .objectFit(ImageFit.Auto)  
      .borderRadius(6)  
  }  
  .padding("10%")  
  .width("100%")  
  .height("100%")  
  .backgroundColor("rgba(0,0,0,0.2)")  
  .onClick(() => {  
    this.showSnap = false  
  })  
}
```



## LazyForEach应用

LazyForEach从提供的数据源中按需迭代数据，并在每次迭代过程中创建相应的组件。当在滚动容器中使用LazyForEach，框架会根据滚动容器可视区域按需创建组件，当组件滑出可视区域外时，框架会进行组件销毁回收以降低内存占用。

- 接口描述

```
LazyForEach(  
    dataSource: IDataSource, // 需要进行数据迭代的数据源  
    itemGenerator: (item: any, index: number) => void, // 子组件生成函数  
    keyGenerator?: (item: any, index: number) => string // 键值生成函数  
): void
```

本质上-LazyForEach和ForEach的用法基本一致，但是ForEach属于全量渲染，而LazyForEach属于只渲染可见区域

- 限制

- LazyForEach必须在容器组件内使用，仅有[List](#)、[Grid](#)、[Swiper](#)以及[WaterFlow](#)组件支持数据懒加载（可配置cachedCount属性，即只加载可视部分以及其前后少量数据用于缓冲），其他组件仍然是一次性加载所有的数据。
- LazyForEach在每次迭代中，必须创建且只允许创建一个子组件。
- 生成的子组件必须是允许包含在LazyForEach父容器组件中的子组件。
- 允许LazyForEach包含在if/else条件渲染语句中，也允许LazyForEach中出现if/else条件渲染语句。
- 键值生成器必须针对每个数据生成唯一的值，如果键值相同，将导致键值相同的UI组件渲染出现问题。
- **LazyForEach必须使用DataChangeListener对象来进行更新，第一个参数dataSource使用状态变量时，状态变量改变不会触发LazyForEach的UI刷新。**
- 为了高性能渲染，通过DataChangeListener对象的onDataChange方法来更新UI时，需要生成不同于原来的键值来触发组件刷新。

- 具体案例

[UI框架-List组件的使用之商品列表 \(ArkTS\) .zip](#)

## 吸顶效果

- List吸顶

```
// xxx.ets  
@Entry  
@Component  
struct ListItemGroupExample {  
    private timeTable: TimeTable[] = [  
        {  
            title: '星期一',  
            projects: ['语文', '数学', '英语']  
        },  
        {  
            title: '星期二',  
            projects: ['物理', '化学', '生物']  
        },  
        {  
            title: '星期三',  
            projects: ['历史', '地理', '政治']  
        },  
    ],
```



```

    {
        title: '星期四',
        projects: ['美术', '音乐', '体育']
    }
]

@Builder
itemHead(text: string) {
    Text(text)
        .fontSize(20)
        .backgroundColor(0xAABBCC)
        .width("100%")
        .padding(10)
}

@Builder
itemFoot(num: number) {
    Text('共' + num + "节课")
        .fontSize(16)
        .backgroundColor(0xAABBCC)
        .width("100%")
        .padding(5)
}

build() {
    Column() {
        List({ space: 20 }) {
            ForEach(this.timeTable, (item: TimeTable) => {
                ListItemGroup({ header: this.itemHead(item.title), footer:
this.itemFoot(item.projects.length) }) {
                    ForEach(item.projects, (project: string) => {
                        ListItem() {
                            Text(project)
                                .width("100%")
                                .height(100)
                                .fontSize(20)
                                .textAlign(TextAlign.Center)
                                .backgroundColor(0xFFFFFFFF)
                        }
                    }, (item: string) => item)
                }
                .divider({ strokeWidth: 1, color: Color.Blue }) // 每行之间的分界线
            })
        }
        .width('90%')
        .sticky(StickyStyle.Header | StickyStyle.Footer)
        .scrollBar(BarState.Off)
    }.width('100%').height('100%').backgroundColor(0xDCDCDC).padding({ top: 5 })
}
}

interface TimeTable {

```

```
title: string;  
projects: string[];  
}
```